



TEMA 02 - TypeScript y Angular

RA 2, RA 3, RA 5 [FRAMEWORKS] Y RA 7 [DWECC]

Parte 02

Angular es un framework de desarrollo frontend creado por Google que permite construir aplicaciones web de una **sola página (SPA) en el lado del cliente**. Aunque evolucionó a partir de AngularJS, Angular moderno (a partir de la versión 2) fue reescrito desde cero y utiliza TypeScript como base, lo que facilita un desarrollo más estructurado y escalable.

Su arquitectura se basa en **componentes reutilizables** y **módulos** que organizan el código de manera eficiente, permitiendo mantener aplicaciones grandes sin perder el control del proyecto.

Angular se apoya en llamadas a **APIs y microservicios** para manejar datos dinámicos de manera eficiente.

Los pilares fundamentales de Angular:

- Declarar UI con plantillas HTML: Crear interfaces usando templates HTML
- Vincular datos: Usar interpolación `{{ }}` y directivas como `@for`, `@if`
- Manejar eventos: Capturar clics, formularios y entrada del usuario con (click), (input)
- Obtener datos: Comunicarte con APIs usando Fetch, y HttpClient y Observables
- Navegar entre páginas: Crear rutas y navegación con el Router
- Proteger rutas: Implementar autenticación con Guards

Aunque actualmente React es más popular entre desarrolladores, Angular sigue siendo ampliamente utilizado en el entorno empresarial debido a su robustez, herramientas integradas (CLI, Router, HttpClient, etc.), y el **soporte a largo plazo** ofrecido por Google.

0. Instalaciones básicas	2
0.1. Guía de atajos	4
0.2. API Reference	5
1. Diferencias entre AngularJS y Angular moderno	7
2. Creación de un nuevo proyecto y estructura básica	9
2.1. Standalone Components	9
2.2. Módulos	13
2.3. Compilando y desplegando la aplicación en producción	19
3. Arquitectura de Angular	25
3.1. Guía rápida de Angular	28
3.2. Build your first Angular app	91
A. Pruebas en Angular	130

0. Instalaciones básicas

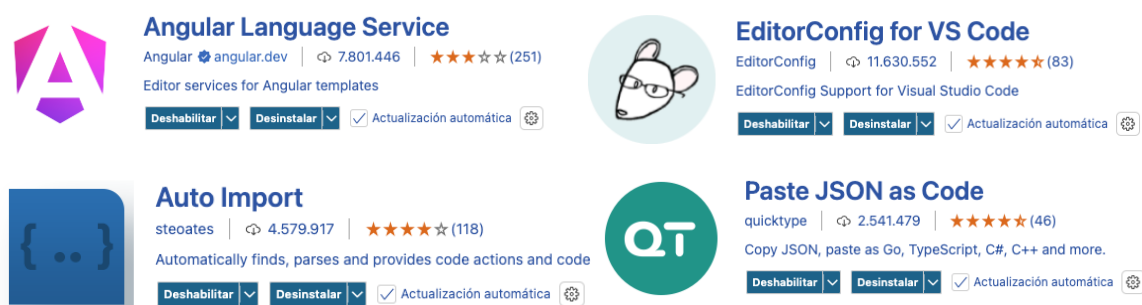
Para los siguientes temas seguiremos usando las herramientas previas: distintos navegadores, VS Code, Git y Postman.



NUEVO: Además, instala las siguientes extensiones para Google Chrome:

- Angular DevTools
<https://chromewebstore.google.com/detail/angular-devtools/ienfalfjdbdpbioblackkekamfmbnh>
- JSON Viewer Pro:
<https://chromewebstore.google.com/detail/json-viewer-pro/eifflpmocdbdmejbjaopkhhbfmdgiicc>
- Postman Interceptor:
<https://chromewebstore.google.com/detail/postman-interceptor/aicmkgpgakddgnaphhhpliifpcfhicfo?hl=es>

NUEVO: Y añade las siguiente extensiones a VS Code:





Para trabajar con Angular es necesario tener instalado NodeJS - npm - y NVM. Si ya lo tienes instalado, verifica tu versión:

```
node -v
```

```
npm -v
```

nvm el gestor de versiones de Node.js

Si tienes nvm, mucho mejor, aunque de momento no lo vamos a usar activamente en clase. Si lo tienes instalado:

```
nvm -v
```

nvm puede convivir con una instalación normal de Node.js en el sistema. En ese caso, tendrías dos instalaciones de Node.js: una instalada directamente en el sistema y otra gestionada por nvm. No obstante, lo más habitual en entornos profesionales es: Desinstalar Node.js (instalado manualmente), instalar nvm y finalmente instalar las versiones de Node necesarias usando nvm.

Comandos básicos de nvm

- Activar la versión LTS (Long Term Support) recomendada: `nvm use lts`
- Versión activa: `nvm current`
- Listar las versiones de Node disponibles o instaladas: `nvm ls`

Para windows: <https://github.com/coreybutler/nvm-windows>

Nota: Después de instalar Node.js, npm o nvm, es necesario cerrar el terminal y abrir uno nuevo para que los cambios se apliquen correctamente.

NUEVO: Y por supuesto hay que instalar Angular CLI: <https://angular.dev/tools/cli>

De manera GLOBAL:

```
npm install -g @angular/cli
```

```
ng version
```

Si quiere hacer una instalación local para un solo proyecto:

```
npm install @angular/cli@latest
```

```

$ ng version

Angular CLI: 19.2.0
Node: 20.17.0
Package Manager: npm 10.8.2
OS: darwin x64

Angular:
...

Package      Version
-----
@angular-devkit/architect    0.1902.0 (cli-only)
@angular-devkit/core        19.2.0 (cli-only)
@angular-devkit/schematics   19.2.0 (cli-only)
@schematics/angular         19.2.0 (cli-only)
  
```

⚠ Dependiendo de la versión de Angular que uses, el nombre de los ficheros cambia ligeramente: `app.module.ts` → `app-module.ts`, `app.component.ts` → `app.ts`, etc.

Tenlo en cuenta al probar los ejemplos en el aula o en casa.

0.1. Guía de atajos

Fernando Herrera es especialista en frameworks como Angular, Vue y otros, y es conocido por su estilo claro y práctico de enseñanza. Además, ofrece recursos gratuitos, como su chuleta de Angular, donde recopila conceptos clave y ejemplos útiles para desarrolladores.



Fernando Herrera
Klerith

Follow

Un simple desarrollador :)

13.8k followers · 3 following

Profesor y Full Stack Developer

Costa Rica

{dev/talles}

Repositorio de +Talento

Aquí les comparto todas las hojas de atajos y presentaciones que tengo sobre mis cursos.

Espero les sirva mucho en su camino!

Atte: Fernando y equipo de DevTalles

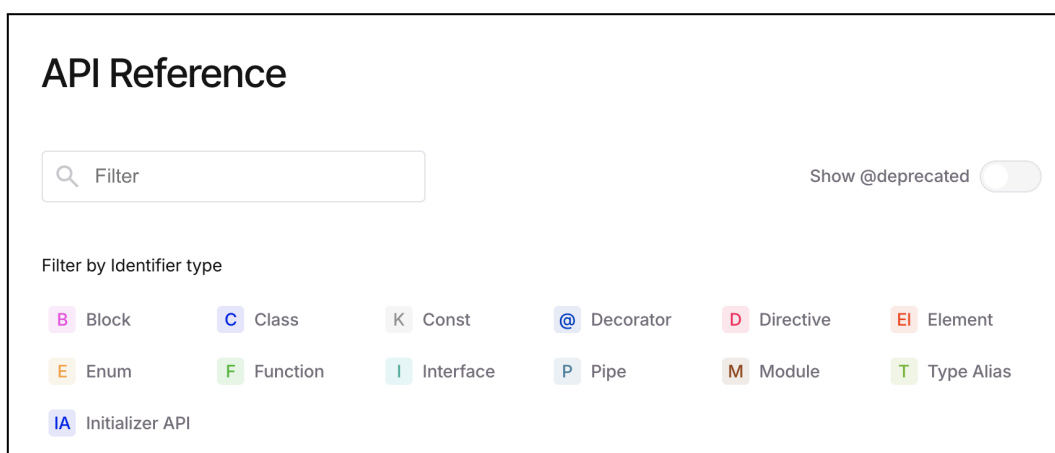
<https://github.com/Klerith/mas-talento/tree/main/angular>

<https://cursos.devtales.com/pages/mas-talento>

Chuleta del curso de Angular de Fernando Herrera

0.2. API Reference

<https://angular.dev/api>



0.3. LLM prompts and AI IDE setup

En este enlace se explica cómo darle instrucciones a una IA para que escriba Angular “bien”:

<https://angular.dev/ai/develop-with-ai>

Angular ha cambiado mucho (standalone, signals, control flow...) y las IAs pueden generar código de Angular antiguo. Si utilizas una herramienta de Inteligencia Artificial (ChatGPT, Copilot, etc.), debes indicarle explícitamente cómo generar el código.

Copia, pega y adapta este texto al inicio de la conversación con la IA:

```
Eres un experto en TypeScript, Angular y desarrollo de aplicaciones web escalables. Escribes código funcional, mantenible, eficiente y accesible siguiendo las buenas prácticas actuales de Angular y TypeScript.
```

```
## Buenas prácticas de TypeScript
- Usa tipado estricto.
- Prefiere la inferencia de tipos cuando el tipo sea evidente.
- Evita el tipo `any`; usa `unknown` cuando el tipo sea incierto.
```

```
## Buenas prácticas de Angular
```

```
- Usa siempre componentes standalone en lugar de NgModules.
- NO establezcas `standalone: true` en los decoradores de Angular (es el comportamiento por defecto en Angular moderno).
- Usa signals para la gestión del estado.
- Implementa lazy loading en las rutas de funcionalidades.
- NO uses los decoradores `@HostBinding` ni `@HostListener`. Usa la propiedad `host` del decorador `@Component` o `@Directive`.
- Usa `NgOptimizedImage` para todas las imágenes estáticas.
  - `NgOptimizedImage` no funciona con imágenes base64 inline.

## Requisitos de accesibilidad
- El código DEBE pasar todas las comprobaciones de AXE.
- Debe cumplir al menos el nivel AA de las WCAG, incluyendo gestión del foco, contraste de color y atributos ARIA.

### Componentes
- Mantén los componentes pequeños y con una única responsabilidad.
- Usa las funciones `input()` y `output()` en lugar de los decoradores.
- Usa `computed()` para estado derivado.
- Configura `changeDetection: ChangeDetectionStrategy.OnPush` en el decorador `@Component`.
- Prefiere plantillas inline para componentes pequeños.
- Prefiere formularios reactivos frente a formularios basados en plantillas.
- NO uses `ngClass`; usa bindings de `class`.
- NO uses `ngStyle`; usa bindings de `style`.
- Cuando uses plantillas o estilos externos, las rutas deben ser relativas al archivo TypeScript del componente.

## Gestión del estado
- Usa signals para el estado local del componente.
- Usa `computed()` para valores derivados.
- Mantén las transformaciones de estado puras y predecibles.
- NO uses `mutate()` sobre signals; usa `set()` o `update()`.

## Plantillas
- Mantén las plantillas simples y evita lógica compleja.
- Usa control de flujo nativo (`@if`, `@for`, `@switch`) en lugar de `*ngIf`, `*ngFor`, `*ngSwitch`.
- Usa el pipe `async` para manejar observables.
- No asumas la disponibilidad de objetos globales como `new Date()`.
- NO escribas funciones flecha en las plantillas.

## Servicios
- Diseña los servicios con una única responsabilidad.
- Usa la opción `providedIn: 'root'` para servicios singleton.
- Usa la función `inject()` en lugar de la inyección por constructor.

## Reglas adicionales obligatorias
- El código generado debe ser coherente con un proyecto creado mediante Angular CLI.
- Explica brevemente cada fragmento de código que generes.
```



1. Diferencias entre AngularJS y Angular moderno

Angular ha evolucionado mucho desde su primera versión (AngularJS 1.x). Aunque ambos sirven para crear aplicaciones web, hay cambios importantes que vale la pena entender:

1. Lenguaje

- AngularJS usaba JavaScript puro.
- Angular moderno usa TypeScript.

2. Arquitectura

- AngularJS se basaba en controladores y scope, algo parecido a tener rutas en Express donde manejas todo el flujo de datos desde una función.
- Angular moderno trabaja con componentes. Cada componente es como si fuera un módulo independiente de Node que maneja su propia lógica, vista y estilo.

3. Estructura del proyecto

- En AngularJS, la estructura del proyecto era más libre.
- Angular moderno, gracias al CLI, genera proyectos con una estructura fija y ordenada.

4. Módulos y organización

- En Node.js, usamos require o import para traer módulos y separar código.
- Angular moderno hasta la versión 14 se usa NgModules, que son como "supermódulos" donde agrupamos componentes y servicios para que trabajen juntos. Ahora también permite usar componentes standalone e inyección de dependencias de forma nativa.

5. Rendimiento y escalabilidad

- AngularJS se diseñó para proyectos pequeños o medianos.
- Angular moderno está hecho para aplicaciones grandes.

6. Soporte y actualizaciones

- AngularJS ya no recibe mejoras importantes (está en modo mantenimiento).
- Angular moderno sigue actualizándose todo el tiempo.

Curso básico de AngularJS: <https://www.w3schools.com/angularjs/default.asp>

Curso básico de Angular: <https://www.w3schools.com/angular/default.asp>

```
<!DOCTYPE html>
<html lang="es" ng-app="myShoppingList">
<head>
<meta charset="UTF-8">
<title>Mi Lista de Compras con AngularJS</title>
<!-- Cargando AngularJS desde CDN -->
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js"></script>
<style>
  body { font-family: Arial, sans-serif; margin: 20px; }
  ul { list-style-type: none; padding: 0; }
  li { margin: 5px 0; }
</style>
</head>
<body ng-controller="myCtrl">

  <h1>Lista de la compra</h1>

  <!-- Input para añadir productos -->
  <input type="text" ng-model="newProduct" placeholder="Añadir producto">
  <button ng-click="addItem()">Agregar</button>

  <!-- Lista de productos -->
  <ul>
    <li ng-repeat="x in products">
      {{x}}
      <button ng-click="removeItem($index)">Eliminar</button>
    </li>
  </ul>

  <p>Total de productos: {{products.length}}</p>

  <script>
    var app = angular.module("myShoppingList", []);

    app.controller("myCtrl", function($scope) {
      $scope.products = ["Milk", "Bread", "Cheese"];

      $scope.addItem = function() {
        if ($scope.newProduct) {
          $scope.products.push($scope.newProduct);
          $scope.newProduct = ""; // Limpiar input
        }
      };

      $scope.removeItem = function(index) {
        $scope.products.splice(index, 1);
      };
    });
  </script>

</body>
</html>
```

Lista de la compra

 Milk Bread Cheese

Total de productos: 3

2. Creación de un nuevo proyecto y estructura básica

Dentro del repositorio `fw_ejercicios_apellido1_nombre1_25-26`

Crear la siguiente carpeta que almacenará los distintos ejemplos vistos en el aula y los ejercicios que habrá que realizar:

- `fw_t02p02_apellido1_nombre1`

Antes de nada es importante conocer la versión actual de Angular por varios motivos que vamos a ver más adelante (uso de módulos o no, nombre de los ficheros, etc.):

```
✓ TERMINAL
~/workspace/dwec_24_25_angular ➤ ng --version
19.2.0
```

```
ng --help
ng generate --help
ng build --help
ng serve --help
```

2.1. Standalone Components

⚠ Desde Angular v14 en adelante y de forma oficial en v17, Angular introdujo una forma más simple de trabajar llamada "Standalone Components", donde ya no es obligatorio usar módulos para aplicaciones pequeñas o medianas.

Dado que hay multitud de aplicaciones desplegadas usando la filosofía de módulos es necesario conocer los dos tipos de aplicaciones.

En `fw_t02p02_apellido1_nombre1`, podemos crear un nuevo proyecto "Standalone Components" desde cero ejecutando:

```
ng new mi_app_01
[npm init @angular mi_app_01 - forma antigua]
```

```
Apple / ~ / workspace / dwec_24_25_angular ng new mi_app_01

Would you like to enable autocomplete? This will set up your terminal so pressing TAB while typing
Angular CLI commands will show possible options and autocomplete arguments. (Enabling autocomplete
will modify configuration files in your home directory.) Yes
Appended `source <(ng completion script)` to `~/.zshrc`. Restart your terminal or run the
following to autocomplete `ng` commands:

source <(ng completion script)

Would you like to share pseudonymous usage data about this project with the Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.dev/cli/analytics.

No
Global setting: disabled
Local setting: No local workspace configuration file.
Effective status: disabled
✓ Which stylesheet format would you like to use? CSS [
https://developer.mozilla.org/docs/Web/CSS ]
✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
CREATE mi_app_01/README.md (1470 bytes)
CREATE mi_app_01/.editorconfig (314 bytes)
```

A la pregunta de IA indica NONE

A la pregunta de SSR contesta NO para no añadir complejidad al proyecto. Server-Side Rendering (SSR) y Static Site Generation (SSG/Prerendering) son dos técnicas avanzadas que generan el contenido de la aplicación en el servidor, antes de enviarlo al navegador, mejorando el rendimiento y ayudando al SEO. Para este curso básico no son necesarias, ya que trabajaremos todo desde el cliente.

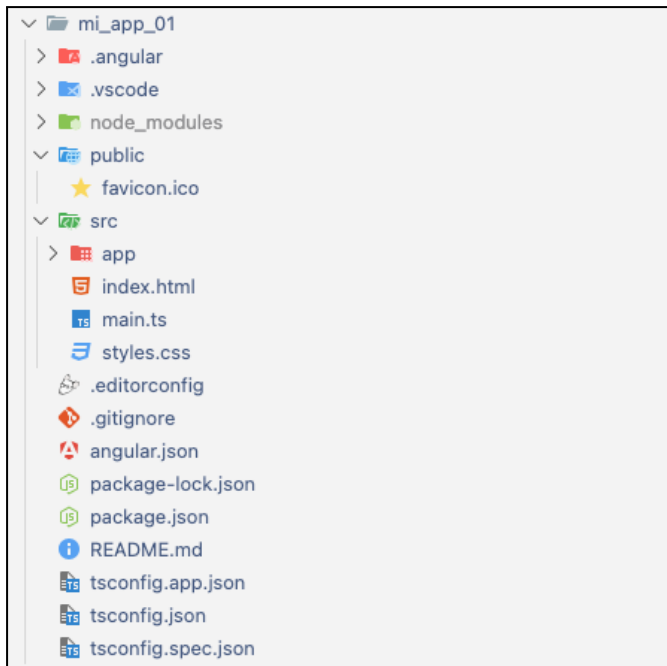
Esto generará automáticamente una estructura completa y organizada, lista para empezar a trabajar. El proyecto generado es de tipo **"Standalone Components"**.

Para arrancar el servidor de desarrollo y ver tu app en acción:

```
cd mi_app_01
ng serve
```

```
ng serve --open
ng serve -o
ng serve --help
```

La aplicación estará disponible en <http://localhost:4200/>



- `package.json`: Igual que en Node.js, gestiona las dependencias y scripts del proyecto.
- `tsconfig.json`: Configuración global de TypeScript.
- `src/index.html`: Archivo raíz HTML.
- `src/styles.css`: Estilos globales.
- `src/main.ts`: Punto de arranque del proyecto.
- `src/app/`: Aquí vive la lógica de la aplicación.
 - `app.component.ts` (o `app.ts`): La lógica del componente raíz. Aquí definimos datos y comportamiento.
 - `app.component.html` (o `app.html`): La vista del componente raíz. Contiene el HTML que se renderiza.
 - `app.component.css` (o `app.css`): Estilos específicos para este componente.
- `angular.json`: Configuración global del proyecto. **SÍ ES INTERESANTE.**

Nota: Los ficheros `spec` son ficheros relacionados con el testing.

Vemos el `main.ts` donde observamos que se usa el AppComponent y su configuración inicial dada por `appConfig`:

```
mi_app_01 > src > main.ts > ...  
1 import { bootstrapApplication } from '@angular/platform-browser';  
2 import { appConfig } from '../app/app.config';  
3 import { AppComponent } from '../app/app.component';  
4  
5 bootstrapApplication(AppComponent, appConfig)  
6   .catch((err) => console.error(err));
```

```
mi_app_01 > src > app > app.config.ts > ...  
1 import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';  
2 import { provideRouter } from '@angular/router';  
3  
4 import { routes } from '../app.routes';  
5  
6 export const appConfig: ApplicationConfig = {  
7   providers: [provideZoneChangeDetection({ eventCoalescing: true }), provideRouter(routes)]  
8 };
```

Nota: `bootstrapApplication` solo se utiliza en el AppModule y define el componente inicial que se debe cargar al iniciar la aplicación. No confundir con el framework.

Vemos `app.component.ts`, el componente principal que sirve de entrada a la app:

```
1 import { Component } from '@angular/core';  
2 import { RouterOutlet } from '@angular/router';  
3  
4 @Component({  
5   selector: 'app-root',  
6   standalone: true,  
7   imports: [RouterOutlet],  
8   templateUrl: '../app.component.html',  
9   styleUrls: '../app.component.css',  
10 })  
11 export class AppComponent {  
12   title = 'mi_app_01';  
13 }
```

Nota: Si ves un error sobre imports, agrega `standalone: true`

Observa que en Angular hay **dos tipos de import**:

```
1  import { Component } from '@angular/core';
2  import { RouterOutlet } from '@angular/router';
3
4  @Component({
5    selector: 'app-root',
6    standalone: true,
7    imports: [RouterOutlet],
8    templateUrl: './app.component.html',
9    styleUrls: ['./app.component.css'],
10  })
11  export class AppComponent {
12    title = 'mi_app_01';
13  }
```

- a. Imports en las primeras líneas de un componente: Se usan para importar **clases o funciones específicas** que se necesitan solo dentro de ese archivo. Sin estas líneas, TypeScript no sabría qué es Component o CommonModule y daría error de compilación. Es simplemente traer código externo a tu archivo para poder usarlo.
- b. Imports dentro del componente (en standalone true) en @component: Se usan para registrar **módulos o componente completos** en tu **Plantilla HTML** y habilitar sus funcionalidades en toda la aplicación. In esto, aunque hayas importado arriba con TypeScript, Angular no dejará usar esas directivas en el HTML y dará error en el navegador.

2.2. Módulos

Como hemos explicado, por defecto, las nuevas versiones de Angular trabajan sin módulos. Dado que hay multitud de aplicaciones desplegadas usando la filosofía de **MÓDULOS** es necesario tratar este concepto:

FUERA del repositorio `fw_ejercicios_apellido1_nombre1_25-26`

Crear la siguiente carpeta que almacenará los distintos ejemplos CON su propio repositorio individual:

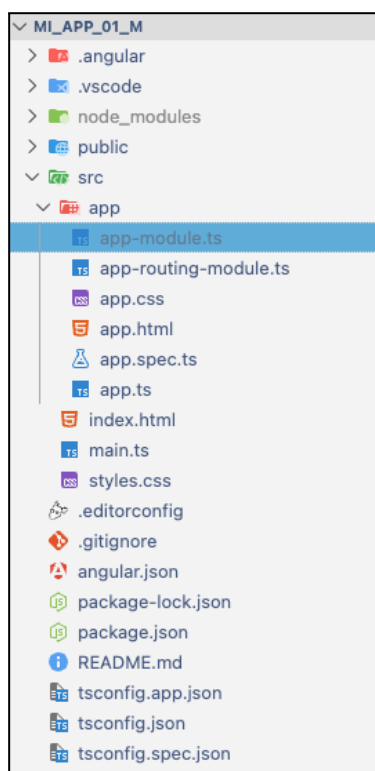
- `fw_t02p02_INDIVIDUAL_apellido1_nombre1`

En `fw_t02p02_INDIVIDUAL_apellido1_nombre1`

```
ng new mi_app_01_m --standalone false
```

El asistente te hará las mismas preguntas que antes. También verás inicia un repositorio con un commit inicia en la carpeta mi_app_01_m

```
cd mi_app_01_m
ng serve
```



```
src > ts main.ts > ...
1  import { platformBrowser } from '@angular/platform-browser';
2  import { AppModule } from './app/app-module';
3
4  platformBrowser().bootstrapModule(AppModule, {
5
6  })
7  .catch(err => console.error(err));
```

Observa los cambios en ficheros con respecto al proyecto anterior:

- app-module.ts
- app-routing.module.ts
 - Ya no tenemos el app.route.ts

- Ya no tenemos app.config.ts

```
src > app > TS app-module.ts > ...
1  import { NgModule, provideBrowserGlobalErrorListeners } from '@angular/core';
2  import { BrowserModule } from '@angular/platform-browser';
3
4  import { AppRoutingModule } from './app-routing-module';
5  import { App } from './app';
6
7  @NgModule({
8    declarations: [
9      App
10   ],
11   imports: [
12     BrowserModule,
13     AppRoutingModule
14   ],
15   providers: [
16     provideBrowserGlobalErrorListeners(),
17   ],
18   bootstrap: [App]
19 })
20 export class AppModule { }
```

```
src > app > TS app.ts > ...
1  import { Component, signal } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.html',
6    standalone: false,
7    styleUrls: ['./app.css']
8  })
9  export class App {
10    protected readonly title = signal('mi_app_01_m');
11  }
```

Observa que en Angular hay **dos tipos de import**:

- Imports en las primeras líneas de un componente: Se usan para importar **clases o funciones específicas** que se necesitan solo dentro de ese archivo. Sin estas líneas, TypeScript no sabría qué es Component o CommonModule y daría error de compilación. Es simplemente traer código externo a tu archivo para poder usarlo.
- Cuando trabajas con módulos (standalone: false), el array imports va

en `@NgModule` en lugar de en cada componente. Igual que antes, aquí registras las funcionalidades que estarán disponibles, pero ahora para todos los componentes declarados en ese módulo.

- ! **Con módulos:** Agrupamos todo en un único archivo para que Angular sepa qué usar.
- ! **Sin módulos:** Cada componente dice lo que necesita y se basta por sí mismo.

Con módulos (<code>--standalone false</code>)	Sin módulos (standalone por defecto)
Se crea el archivo <code>app.module.ts</code> .	No existe <code>app.module.ts</code> .
Los componentes se registran dentro del módulo (<code>@NgModule</code>).	Los componentes se registran solos, dentro del decorador <code>@Component</code> .
Los imports globales se configuran en el módulo.	Los imports se agregan directamente en cada componente que los necesita.
Estructura clásica y conocida.	Estructura más ligera y moderna.
Ideal para apps medianas/grandes.	Ideal para apps pequeñas o simples.

Antes de seguir, vamos a hacer unas pequeñas pruebas en el proyectos: `mi_app_01_m`

Vemos `app.ts` (antiguamente `app.component.ts`):

```
src > app > ts app.ts > ...
1  import { Component, signal } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.html',
6    standalone: false,
7    styleUrls: ['./app.css'],
8  })
9  export class App {
10   protected readonly title = signal('mi_app_01_m'); // ← Esto es una SEÑAL (signal)
11   public year = new Date().getFullYear(); // ← Esto es una variable normal
12 }
```

- `@Component ({...})`: Es un decorador que le dice a Angular que esta clase es un componente y le indica sus configuraciones básicas.
 - `'app-root'`: Define el nombre de la etiqueta que representa este componente (`<app-root>`).

- `export class App`: estamos creando y exportando una clase llamada App con dos propiedades. Angular usa esta clase para controlar lo que pasa en la parte principal de la aplicación.

El fichero *.ts o *.component.ts puede contener contenido HTML, aunque si el HTML tiene más de 4 líneas se recomienda tener un fichero externo:

```
3 @Component({
4   selector: 'app-root',
5   template: `
6     <span>The server is running</span>
7   `,
8   styles: `
9     span {
10       color: red;
11     }
12 `
13 })
```

Vemos que de momento este componente no tiene estilos en app.css. Añade el siguiente:

```
:host {
  color: #a144eb;
}
```

:host Es una pseudoclase especial de Angular que representa el propio componente. Cuando creas un componente en Angular, este se convierte en una etiqueta HTML personalizada (por ejemplo, <app-header>). La pseudoclase :host te permite aplicar estilos a esa etiqueta personalizada desde dentro del propio componente. Con :host puedes darle estilo al contenedor exterior del componente, como ponerle un borde, un color de fondo, o convertirlo en un flex container.

Ahora lo que vamos a hacer es borrar todo el contenido de app.html y dejarlo así:

```
<main class="main">
  <div class="content">
    <h1>Hello, {{ title() }}</h1>
    <p>Congratulations! Your app is running.</p>
    <p>{{ year }}</p>
    <p>{{ 1 + 1 }}</p>
  </div>
  <router-outlet />
```

En las nuevas versiones de Angular, title no es una variable, es un SIGNAL.

Un signal es una variable especial que avisa automáticamente a Angular cuando cambia su valor, para que actualice solo las partes del HTML que lo necesitan.

```
</main>
```

Vemos cómo se reflejan los cambios en pantalla automáticamente gracias al **binding**.

```
Los datos están directamente en el componente y los  
interpolamos con {{ }}.
```

Ahora vamos a instalar bootstrap usando npm y le vamos indicar a nuestro proyecto que lo use:

```
npm install bootstrap
```

Cuando instalas Bootstrap con `npm install bootstrap`, los archivos quedan integrados en tu proyecto y Angular los compila junto con tu aplicación. Esto significa que solo se incluyen los estilos que realmente usas. Con CDN, dependes de un servidor externo que puede fallar y cargas Bootstrap completo.

Ve al archivo `angular.json` y busca la sección "styles" y "scripts" dentro de "build":

```
"styles": [  
  "node_modules/bootstrap/dist/css/bootstrap.min.css",  
  "src/styles.css"  
],  
"scripts": [  
  "node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"  
]
```

```
29  
30 |  
31 |  
32 |  
33 |  
34 |  
35 |  
    "styles": [  
      "node_modules/bootstrap/dist/css/bootstrap.min.css",  
      "src/styles.css"  
    ],  
    "scripts": [  
      "node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"  
    ]
```

Nota: También existe la sección "styles" y "scripts" dentro de "test". **TEN CUIDADO.**

Edita el contenido de `app.html` y dejarlo así para probar que se engancha bien Bootstrap:

```
<main class="main container mt-5">
  <div class="content text-center">
    <h1 class="display-4">Hello, {{ title() }}</h1>
    <p class="lead">Congratulations! Your app is running.</p>
    <p>{{ year }}</p>
  </div>
  <router-outlet></router-outlet>
</main>
```

<https://material.angular.io/guide/getting-started>

Angular Material es una biblioteca de componentes UI basada en Material Design, el sistema de diseño creado por Google. Proporciona una colección de componentes preestilizados y accesibles, como botones, tarjetas, formularios y barras de navegación, que se integran de forma nativa con Angular. Similar a Bootstrap, pero diseñado específicamente para Angular.

Para instalar:

```
ng add @angular/material
```

Para empezar a usar los componentes de Angular Material solo tienes que importarlos en tus módulos:

```
import { MatButtonModule } from '@angular/material/button';
```

2.3. Compilando y desplegando la aplicación en producción

Para generar una versión optimizada de la aplicación lista para producción, usamos el siguiente comando:

```
ng build --configuration production
o
ng build
```

Esto compila y optimiza el código, reduciendo el tamaño de los archivos y mejorando el rendimiento mediante técnicas como:

- Minificación del código
- Tree-shaking (elimina código no usado)
- Optimización de bundle (agrupa archivos eficientemente)

El resultado se guarda en la carpeta `dist/tu-proyecto/`, que contiene los archivos listos para ser desplegados en un servidor.

Una vez generados los archivos en `dist/`, hay varias opciones para subirlos a un servidor:

- Servidor Estático (Ej: Apache, Nginx, Firebase Hosting).
- Desplegar en GitHub Pages si tienes el código en GitHub
- También puedes subir la carpeta `dist/` a servicios gratuitos que permiten hospedar aplicaciones estáticas con despliegue automático. Algunas de las plataformas más populares son: Netlify, Vercel, Render, Railway, etc.

Despliegue de prueba en Vercel.

Vamos a desplegar el proyecto `mi_app_01_m` a modo de prueba usando Vercel. Vercel permite automatizar el despliegue detectando automáticamente los cambios en el repositorio de GitHub. Por eso, hemos sacado este proyecto del repositorio general del curso para crear un repositorio independiente exclusivamente para él, facilitando así el despliegue continuo.

Pasos a seguir:

1. Crear cuenta en Vercel

Crea una cuenta en `vercel.com` haciendo login con tu cuenta de GitHub. Esto permitirá que Vercel acceda a tus repositorios.

2. Crear repositorio en GitHub

Crea un repositorio en GitHub PÚBLICO y sin contenido inicial (sin README, sin `.gitignore`, sin licencia). Nómbralo exactamente: `mi_app_01_m`

3. Subir el proyecto a GitHub

Angular ha debido crear un repositorio Git de forma automática al generar el proyecto con `ng new`. Ahora solo debes configurar la rama principal y subir el código:

```
git branch -M main
git add .
git commit -m "Hola Angular"
git remote add origin https://github.com/tu-usuario/mi_app_01_m.git
git push -u origin main
```

4. Desplegar en Vercel

Accede a Vercel y haz clic en "Add New Project". Selecciona "Import Git Repository" y elige tu repositorio `mi_app_01_m`. Vercel detecta automáticamente que es un proyecto Angular y configura:

- Build Command: `ng build`
- Output Directory: `dist/mi_app_01_m/browser`

Haz clic en "Deploy" y espera a que termine el proceso (1-3 minutos)

5. Accede a tu sitio

Una vez completado, Vercel te proporcionará una URL pública tipo `https://mi-app-01-m.xxx.vercel.app` donde estará desplegada tu aplicación. A partir de ahora, cada vez que hagas **git push**, Vercel detectará el cambio y desplegará automáticamente la nueva versión.

Todo eso lo podemos hacer desde el terminal con `npm install -g vercel`

Despliegue de prueba en servidor 'Tradicional'.

Dado que Angular no necesita un backend avanzado (solo requiere un servidor que sirva los archivos HTML, CSS y JavaScript), podemos realizar su despliegue en cualquier servicio de hosting básico.

- Puedes probar algunos servicios gratuitos como: InfinityFree, AwardSpace, Byet.host, FreeHosting.com, etc.

Pasos a seguir:

1. Crear cuenta y conectar por FTP

Crea una cuenta en un hosting gratuito y, usando las credenciales FTP proporcionadas, realiza la

conexión con FileZilla o incluso desde el propio servicio FTP vía web que ofrecen estos sitios.

Ejemplo con FileZilla:

- Servidor: ftp.tudominio.com
- Usuario: Tu usuario FTP
- Contraseña: Tu contraseña FTP
- Puerto: 21 (normalmente)

2. Compilar el proyecto Angular

En tu terminal, dentro de la carpeta del proyecto Angular, ejecuta: `ng build`

Esto genera la carpeta `dist/nombre-proyecto/browser/` con los archivos optimizados para producción.

3. Subir el contenido de la carpeta browser/

Con FileZilla (o el gestor de archivos del hosting):

- Navega a la carpeta `htdocs/` o `public_html/` (según el hosting)
- Borra todo el contenido que haya por defecto
- Busca en tu PC la carpeta `dist/nombre-proyecto/browser/`
- Selecciona todo el contenido de la carpeta `browser/` (NO subas la carpeta `browser/` en sí, solo su contenido)
- Arrastra los archivos al servidor.

Estructura final en el servidor (`htdocs/`):

Sitio remoto: /htdocs



Nombre de archivo ^	Tamaño de archivo	Tipo de archivo	Última modificación	Permisos
..				
.htaccess	285	Archivo	22/01/2026 2...	0644
favicon.ico	15.086	ico-archivo	22/01/2026 2...	0644
index.html	5.011	html-arch...	22/01/2026 2...	0644
main-ZNOOTCU3.js	204.819	js-archivo	22/01/2026 2...	0644
scripts-TTWY4XDY.js	80.449	js-archivo	22/01/2026 2...	0644
styles-JG7EAGFK.css	230.853	Text Docu...	22/01/2026 2...	0644

4. Crear el archivo .htaccess

Angular es una Single Page Application (SPA), por lo que necesita que todas las rutas redirijan a index.html. Para ello, crea un archivo llamado .htaccess en la misma carpeta donde está index.html (normalmente htdocs/ o public_html/) con el siguiente contenido:

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /

  # Si el archivo o directorio existe, sírvelo directamente
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d

  # Si no existe, redirige a index.html
  RewriteRule ^ index.html [L]
</IfModule>
```

Sitio remoto: /htdocs

✓

/

htdocs

Nombre de archivo	Tamaño de archivo	Tipo de archivo	Última modificación	Permisos
..				
.htaccess	285	Archivo	22/01/2026 2...	0644
favicon.ico	15.086	ico-archivo	22/01/2026 2...	0644
index.html	5.011	html-arch...	22/01/2026 2...	0644
main-ZNOOTCU3.js	204.819	js-archivo	22/01/2026 2...	0644
scripts-TTWY4XDY.js	80.449	js-archivo	22/01/2026 2...	0644
styles-JG7EAGFK.css	230.853	Text Docu...	22/01/2026 2...	0644

5. Acceder al sitio web

Accede a tu aplicación usando el dominio que te ha facilitado el sitio de hosting (por ejemplo: <https://tudominio.infinityfreeapp.com>).

- Si no carga correctamente:
 - Limpia la caché del navegador (Ctrl + Shift + R en Windows/Linux o Cmd + Shift + R en Mac)
 - Espera 2-3 minutos, ya que algunos hostings tardan en aplicar los cambios

3. Arquitectura de Angular

Angular es un framework pensado para organizar **aplicaciones de manera modular y escalable**. Todo gira en torno a piezas que se combinan para crear la aplicación.

- **COMPONENTES:** Los componentes son las piezas básicas de una aplicación Angular. Cada componente tiene tres partes:

- HTML (la vista).
- CSS (los estilos).
- TypeScript (la lógica).

Cada vez que queremos mostrar algo en pantalla o manejar una sección concreta, creamos un componente.

```
ng generate component [path/name]
ng g c [path/name]
```

- **DIRECTIVAS:** Las directivas son herramientas que permiten modificar el comportamiento o la apariencia de un elemento HTML. Tipos de Directivas en Angular:

- Directivas de Atributo: Modifican la apariencia o comportamiento de elementos (ejemplo: `ngClass`, `ngStyle`).
- Directivas Estructurales: Modifican el DOM (`*ngIf`, `*ngFor`, `*ngSwitch`,).
- Directivas Personalizadas.

```
ng generate directive [path/name]
ng g d [path/name]
```

- **SERVICIOS:** Los servicios son clases que se encargan de la lógica que queremos reutilizar, como por ejemplo:

- Llamadas a APIs.
- Gestión de datos Compartidos.
- Funciones de ayuda o manipulación de datos.

Gracias a los servicios, no repetimos código en cada componente y mantenemos la lógica centralizada.

```
ng generate service [path/name]
ng g s [path/name]
```

- **RUTAS:** En Angular, las rutas permiten navegar entre diferentes componentes según la URL del navegador, sin recargar la página. Gracias al sistema de enrutamiento, podemos crear aplicaciones con múltiples "pantallas" o "páginas" dentro de una SPA (Single Page Application). Por ejemplo: la ruta `/products`, carga el componente `ProductsComponent`.

Angular también permite el uso de **Guards**, que son como middlewares en Node, y permiten proteger rutas.

```
ng generate guard [path/name]
ng g g [path/name]
```

- **INYECCIÓN DE DEPENDENCIAS (DI):** es la forma que tiene Angular de entregar automáticamente los servicios (u otros recursos) donde los necesitemos. Cuando un componente necesita un servicio, Angular se encarga de crearlo y dárselo listo para usar, sin que tengamos que importarlo y construirlo manualmente. El propio framework los crea y los gestiona por ti.
- **MÓDULOS:** Los módulos son agrupaciones de componentes, servicios y otras funcionalidades. Cada aplicación tiene al menos un módulo principal llamado `AppModule`. El módulo se encarga de decirle a Angular:
 - Qué componentes existen.
 - Qué servicios están disponibles.
 - Qué otras funcionalidades queremos usar.

```
ng generate module [path/name]
ng g m [path/name]
```

⚠ Desde Angular v14 en adelante y de forma oficial en v17, Angular introdujo una forma más simple de trabajar llamada "**Standalone Components**", donde ya no es obligatorio usar módulos para aplicaciones pequeñas o medianas.

Antes de Angular 14-15, todo proyecto Angular necesitaba módulos (`NgModule`) para funcionar. Siempre había un módulo raíz (`AppModule`) que era obligatorio.

- **OTROS ELEMENTOS.** En Angular también encontramos interceptores y pipes.
 - Los interceptores son funciones que permiten modificar peticiones HTTP antes de enviarlas o después de recibirlas: Añadir un token JWT automáticamente a cada petición o manejar errores globales en respuestas HTTP.

- Los Pipes son herramientas que transforman datos en la vista, sin modificar la lógica del componente, por ejemplo poner todo en mayúscula, usar monedas, definir una precisión en la presentación de datos reales, etc.

```
ng generate interceptor [path/name]
ng g i [path/name]
```

```
ng generate pipe [path/name]
ng g p [path/name]
```

Angular CLI permite generar estructuras de TypeScript como clases, interfaces y enumeraciones. Estas entidades no son componentes en sí mismos, sino estructuras de TypeScript que ayudan a organizar el código y definir modelos de datos dentro de una aplicación Angular.

```
ng generate class [path/name]
ng g cl [path/name]

ng generate interface [path/name]
ng g i [path/name]

ng generate enum [path/name]
ng g e [path/name]
```

3.1. Guía rápida de Angular

En esta guía interactiva vamos a aprender los conceptos básicos que necesitas para empezar a construir aplicaciones modernas con Angular.

PREPARAR UN PROYECTO NUEVO PARA PROBAR

```
ng new mi_app_02_m --standalone false
```

Contesta a las preguntas como hemos venido haciendo hasta ahora.

```
cd mi_app_02_m
ng serve
```

Muy importante: Algunos detalles que vamos a ver a continuación son para un proyecto con módulos clásicos (standalone false) y esto cambia respecto a un proyecto moderno (standalone true).

TEN PRESENTE: Tenemos un sólo módulo que es AppModule y dentro de este módulo vamos a generar todos nuestros componentes. De momento, solo tenemos un componente: AppComponent (o App dependiendo de la versión de Angular).

Deja el fichero app.ts así:

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.html',
  standalone: false,
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('mi_app_02_m'); // SIGNAL <--NUEVO
}
```

Un signal es una variable especial que avisa automáticamente a Angular cuando cambia su valor, para que actualice solo las partes del HTML que lo necesitan

El fichero app.html así:

```
<main class="main">
```

```
<div class="content">
  <h1>Hello, {{ title() }}</h1>
</div>
<router-outlet />
</main>
```

Y el fichero app.component.css así:

```
:host {
  color: blue;
}
```

La pseudoclase `:host` te permite aplicar estilos a la etiqueta personalizada del componente. Es útil porque los estilos CSS normales solo afectan a lo que está dentro. Con `:host` puedes darle estilo al contenedor exterior del componente, como ponerle un borde, un color de fondo, o convertirlo en un flex container.

ANATOMÍA DE UN COMPONENTE

En Angular, los componentes son la base de cualquier aplicación. Todo lo que ves en pantalla está controlado por uno o varios componentes.

En el código anterior ya hemos actualizado una vista y los estilos.

USANDO DATOS DINÁMICOS CON INTERPOLACIÓN - UNA SOLA DIRECCIÓN

En Angular, toda la lógica y comportamiento de un componente se define en su clase TypeScript. Aquí es donde creamos las propiedades y funciones que luego podremos usar dentro del HTML del componente. Esto nos permite que la información que mostramos en pantalla no sea fija, sino que pueda ser dinámica, cambiando según los datos de la clase. La interpolación solo funciona en una dirección.

Ya hemos visto que la interpolación se realiza usando `{{ }}`

```
<p>Usuario: {{ user }}</p>

<p>{{ saludo() }}</p>
```

Evita código complejo dentro de `{{ }}`

Encadenamiento opcional (?.)

- Si employer es null, se detiene y devuelve undefined. No hay error
- Si es undefined, se detiene y devuelve undefined. No hay error
- En ambos casos el texto quedará vacío en lugar de lanzar un error y el HTML queda vacío.

```
<p>Employer: {{ employer?.companyName }}</p>
```

Operador AND lógico (&&)

- Es un filtro de seguridad clásico. Evalúa de izquierda a derecha.
- Si user es null, no mostrará nada en lugar de lanzar un error.
-

```
<p>{{ user && user.name }}</p>
```

Coalescencia nula (??)

- Si user?.country es null o undefined, mostrará 'No especificado'.
- A diferencia del operador ||, este no se activa con el número 0 ni con textos vacíos "". Solo reacciona ante null o undefined.

```
<p>País: {{ user?.country ?? 'No especificado' }}</p>
```

Asignación de coalescencia nula (??=)

- Si username ya tiene valor, lo mantiene. Si es null o undefined, le asigna 'Invitado'.

```
<p>Usuario: {{ username ??= 'Invitado' }}</p>
```

Cuidado: Si

```
userDescription = "<strong>Hola</strong>"
```

```
<p>{{ userDescription }}</p> → No interpreta HTML
```

```
<p [innerHTML]=userDescription></p> → Sí renderizará en negrita
```

Cuidado si lo que vamos a mostrar es una cadena escrita por el usuario (Vulnerabilidad XSS). Aunque Angular limpia el HTML peligroso por defecto.

COMPOSICIÓN DE COMPONENTES

Ahora que ya sabes crear y modificar un componente, llega el momento de componer componentes, es decir, usar un componente dentro de otro.

Esto permite dividir la aplicación en partes más pequeñas, reutilizables y fáciles de mantener.

Cada componente tiene un selector, que es el nombre que usamos para insertarlo dentro de otro componente, igual que si fuera una etiqueta HTML.

Primero vamos a crear el componente desde cli:

```
ng generate component user
```

Esto:

- Crea la carpeta `/src/app/user/`
- Genera automáticamente los cuatro archivos del componente
- Lo registra **automáticamente** en el AppModule dentro del array declarations

```
src > app > app-module.ts > ...
1  import { NgModule, provideBrowserGlobalErrorListeners } from '@angular/core';
2  import { BrowserModule } from '@angular/platform-browser';
3
4  import { AppRoutingModule } from './app-routing-module';
5  import { App } from './app';
6  import { User } from './user/user';
7
8  @NgModule({
9    declarations: [
10     App,
11     User
12   ],
13   imports: [
14     BrowserModule,
15     AppRoutingModule
16   ],
17   providers: [
18     provideBrowserGlobalErrorListeners(),
19   ],
20   bootstrap: [App]
21 })
22 export class AppModule { }
```

Al crear componentes en Angular con `ng generate component` podemos usar banderas (`--flags`) para modificar el comportamiento del comando.

```
--help: Muestra información sobre el comando y sus opciones.
--dry-run: Simula la ejecución sin modificar archivos, útil para verificar qué hará el comando antes de ejecutarlo realmente.
--flat: Crea el componente sin generar una nueva carpeta.
--inline-style / -s: Guarda los estilos directamente en el archivo .ts del componente en lugar de crear un archivo separado .css o .scss.
--inline-template / -t: Mantener el HTML dentro del archivo .ts
--skip-tests / -t: No generar archivos de prueba (.spec.ts)
```

Luego lo vamos a editar:

```
src > app > user >  user.ts > ...  
1  import { Component } from '@angular/core';  
2  
3  @Component({  
4    selector: 'app-user',  
5    standalone: false,  
6    templateUrl: './user.html',  
7    styleUrls: ['./user.css'],  
8  })  
9  export class User {  
10    public username = 'Yoda';  
11  }
```

```
src > app > user >  user.html >  br  
Go to component  
1  <span>Username: {{ username }}</span><br />
```

Puedes usarlo dentro de otro componente así:

```
src > app >  app.html >  div  
Go to component  
1  <main class="main">  
2    <div class="content">  
3      <h1>Hello, {{ title() }}</h1>  
4    </div>  
5    <div>  
6      <section>  
7        <app-user></app-user>  
8      </section>  
9      <ul>  
10       <li><app-user></app-user></li>  
11       <li><app-user></app-user></li>  
12     </ul>  
13   </div>  
14   <router-outlet />  
15 </main>
```

Observa el resultado:

ng serve

MOSTRAR CONTENIDO CONDICIONALMENTE - DIRECTIVA

En aplicaciones reales, muchas veces queremos mostrar o no mostrar algo en pantalla dependiendo de una condición.

En Angular clásico (sin standalone), usamos la directiva `*ngIf` para controlar qué aparece y qué no.

`@if` es la nueva sintaxis de Angular para mostrar contenido condicionalmente de manera más limpia y clara dentro de las plantillas. `@if` es compatible tanto en proyectos standalone como en proyectos con módulos.

```
src > app > TS app.ts > ...
1  import { Component, signal } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.html',
6    standalone: false,
7    styleUrls: ['./app.css'],
8  })
9  export class App {
10    protected readonly title = signal('mi_app_02_m');
11    public isServerRunning = false;
12  }
```

En `/src/app/app.html`:

```
<main class="main">
  <div class="content">
    <h1>Hello, {{ title() }}</h1>
  </div>
  <div>
    <section>
      <app-user></app-user>
    </section>
    <ul>
      <li><app-user></app-user></li>
      <li><app-user></app-user></li>
    </ul>
  </div>
  <div>@if (isServerRunning){
    <span>Yes, the server is running</span>
  }
  @else {
    <span>No, the server is not running</span>
  }
</div>
  <router-outlet />
</main>
```

También tenemos else if:

```
template: `
  @if (showHello) {
    <h2>Hello</h2>
  } @else if (showGoodbye) {
    <h2>Goodbye</h2>
  } @else {
    <h2>See you later</h2>
  }
`
```

<https://blog.angular-university.io/angular-if/>

También tenemos ngSwitch y @switch: <https://angular.dev/api/core/@switch>

```
@switch (condition) {
  @case (caseA) {
    Case A.
  }
  @case (caseB) {
    Case B.
  }
  @default {
    Default case.
  }
}
```

REPETIR ELEMENTOS - DIRECTIVA

En las aplicaciones web, muchas veces tenemos listas de datos que queremos mostrar en pantalla, como usuarios, productos o mensajes.

En Angular clásico (sin standalone), usamos la directiva *ngFor para esto.

Para esto, Angular 19 nos ofrece la nueva sintaxis @for, que nos permite recorrer arrays y repetir elementos de forma limpia dentro de las plantillas. En las aplicaciones web, muchas veces tenemos listas de datos que queremos mostrar en pantalla, como usuarios, productos o mensajes.

Para esto, Angular 19 nos ofrece la nueva sintaxis @for, que nos permite recorrer arrays y repetir elementos de forma limpia dentro de las plantillas.

Con @for podemos recorrer cualquier array y dibujar un bloque HTML por cada elemento del array.

Además, es obligatorio usar `track` para ayudar a Angular a identificar cada elemento con un ID único, lo que mejora el rendimiento y evita errores al actualizar la lista. La propiedad `track` le permite a angular poder seguir el elemento en caso de que el listado cambie.

En `/src/app/app.ts`

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.html',
  standalone: false,
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('mi_app_02_m');
  public isServerRunning = false;
  public characters = [
    { id: 0, name: 'Rachel' },
    { id: 1, name: 'Monica' },
    { id: 2, name: 'Phoebe' },
    { id: 3, name: 'Ross' },
    { id: 4, name: 'Chandler' },
    { id: 5, name: 'Joey' },
  ];
}
```

En `src/app/app.html`:

```
<main class="main">
  <div class="content">
    <h1>Hello, {{ title() }}</h1>
  </div>
  <div>
    <section>
      <app-user></app-user>
    </section>
    <ul>
      <li><app-user></app-user></li>
      <li><app-user></app-user></li>
    </ul>
  </div>
  <div>@if (isServerRunning){
    <span>Yes, the server is running</span>
  }
  @else {
    <span>No, the server is not running</span>
  }
</div>
```

```
</div>
<div>
  @for (character of characters; track character.id) {
    <p>{{ character.id }} - {{ character.name }}</p>
  }
</div>
<router-outlet />
</main>
```

`track` Es una directiva obligatoria que le dice a Angular cómo identificar cada elemento de la lista para optimizar el renderizado.

La nueva sintaxis `@for` en Angular permite recorrer listas de forma eficiente y utilizar variables de control internas como `$index`, `$count`, `$first`, `$last`, `$even` y `$odd`.

Cuando los elementos de un array no tienen un ID único, podemos usar `$index` para hacer el seguimiento en el track. Aunque no es recomendado:

```
<h2>Lista de Coches</h2>
<ul>
  @for (car of cars; track $index) {
    <li>{{ $index }} {{ car }}</li>
  }
</ul>
```

Podemos usar `@for` para recorrer un array sin imprimir los valores, solo para hacer cálculos.

```
<p>Total: @for (car of cars; track $index) { $count
}</p>
```

Nota: La directiva `@for` en Angular solo se puede usar para recorrer arrays u objetos iterables, no puedes usarla directamente por ejemplo para generar una secuencia de `li` con números del 1 al 10.

<https://blog.angular-university.io/angular-for/>

Asociado al `for` nos encontramos también `empty`:

```
template: `<ul>
  @for (item of items; track item) {
    <li>{{ item }}</li>
  }
`
```

```
@empty {  
  <li>No items found</li>  
}  
</ul>`,
```

ENLAZANDO PROPIEDADES DINÁMICAMENTE - UNA SOLA DIRECCIÓN

En Angular, usamos **property binding** para asignar valores dinámicos a propiedades de elementos HTML o de componentes.

Esto nos permite que las propiedades cambien automáticamente según los datos que definimos en la clase.

```
<input [value]="firstName">
```

Si `firstName = 'Juan'` se genera: `<input value="Juan">`

El enlace de propiedades no se limita solo a valores de elementos como `value`, `class`, o `style`. También se puede aplicar a otros atributos de HTML e incluso a atributos personalizados de componentes

```
<div [style.width.px]="mySize">...</div>
```

Si `mySize = 200` se genera: `<div style="width: 200px;">...</div>`

```
<a [href]="linkUrl">Haz clic aquí</a>
```

Si `linkUrl = 'https://angular.dev'` se genera:

```
<a href="https://angular.dev">Haz clic aquí</a>
```

```
<div [attr.role]="myAriaRole">...</div>
```

```
<div [class.extra-sparkle]="isDelightful">...</div>
```

```
<button [disabled]="isDisabled">Enviar</button>
```

```
src > app > TS app.ts > ...
1  import { Component, signal } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.html',
6    standalone: false,
7    styleUrls: ['./app.css'],
8  })
9  export class App {
10    protected readonly title = signal('mi_app_02_m');
11    public isServerRunning = false;
12    public characters = [
13      { id: 0, name: 'Rachel' },
14      { id: 1, name: 'Monica' },
15      { id: 2, name: 'Phoebe' },
16      { id: 3, name: 'Ross' },
17      { id: 4, name: 'Chandler' },
18      { id: 5, name: 'Joey' },
19    ];
20    public isEditable = true;
21  }
```

```
src > app > app.html > main.main
1  <main class="main">
25  </div>
26  <div [contentEditable]="isEditable">
27    Puede editar este texto si isEditable es true.
28  </div>
29  <router-outlet />
30  </main>
```

Nota: atributo contentEditable es una [propiedad de los elementos HTML](#) que permite hacerlos editables directamente en la página, sin necesidad de inputs o textareas.

Directivas ngStyle y ngClass

Las directivas ngStyle y ngClass permiten modificar dinámicamente los estilos y clases CSS de un elemento en función de variables o expresiones. Requieren importar CommonModule en el módulo donde se usen.

```
<p [ngStyle]="{'font-size': size + 'px', 'color': color}">Hola!</p>
<p [ngStyle]="styleObject">Hola!</p>
```

```
size = 20;
color = 'green';
styleObject = {
  'background-color':
    'gray',
  'border': '1px solid
```

	black'; };
<p [ngClass]="unaclase">Adiós!</p> <p [ngClass]="[unaclase, otraclase]">Adiós!</p> <p [ngClass]="{'text-danger': danger}">Adiós!</p>	unaclase = 'text-primary'; otraclase = 'font-weight-bold'; danger = false;

MANEJANDO EVENTOS

Con Angular, la gestión de eventos se hace fácilmente usando **event binding**, que permite ejecutar funciones cuando ocurren eventos. Usamos paréntesis () alrededor del nombre del evento y le indicamos qué función ejecutar.

```
<button (click)="saluda($event)">
<input (input)="almacena($any($event.target).value)">
```

\$event es el evento nativo del navegador (click, input, keydown, etc.).

Cuando queremos acceder directamente a una propiedad del elemento que ha reaccionado al evento tenemos que usar \$any es porque TS no sabe que event.target es un <input>, así que marca error de tipos.

En src/app/app.ts

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.html',
  standalone: false,
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('mi_app_02_m');
  public isServerRunning = false;
  public characters = [
    { id: 0, name: 'Rachel' },
    { id: 1, name: 'Monica' },
    { id: 2, name: 'Phoebe' },
    { id: 3, name: 'Ross' },
    { id: 4, name: 'Chandler' },
    { id: 5, name: 'Joey' },
  ];
  public isEditable = true;
  public message = '';
  public boxClass = 'box-class';
  public onMouseOverAction() {
    this.message = "Let's go!";
  }
}
```

```
}  
public onMouseOutAction() {  
    this.message = "";  
}  
  
}
```

En src/app/app.css

```
:host {  
    color: blue;  
}  
  
.caja-hover {  
    padding: 20px;  
    border: 2px solid green;  
}
```

En src/app/app.html

```
<main class="main">  
  <div class="content">  
    <h1>Hello, {{ title() }}</h1>  
  </div>  
  <div>  
    <section>  
      <app-user></app-user>  
    </section>  
    <ul>  
      <li><app-user></app-user></li>  
      <li><app-user></app-user></li>  
    </ul>  
  </div>  
  <div>@if (isServerRunning){  
    <span>Yes, the server is running</span>  
  }  
  @else {  
    <span>No, the server is not running</span>  
  }  
</div>  
  <div>  
    @for (character of characters; track character.id){  
      <p>{{ character.id }} - {{ character.name }}</p>  
    }  
  </div>  
  <div [contentEditable]="isEditable">
```



```
Puede editar este texto si isEditable es true.
</div>

<router-outlet />
</main>
```

En Angular, la **forma más común y recomendada** de manejar eventos es a través de la plantilla utilizando la sintaxis `(evento)="función()`. Esta metodología sigue el principio de "data binding" de Angular.

Para casos más avanzados, donde se necesita capturar eventos globales o manejar eventos sin modificar la plantilla, se puede usar `@HostListener` en el componente. Esta alternativa permite suscribirse a eventos del DOM de manera reactiva dentro del código TypeScript.

```
@HostListener('click', ['$event'])
onClick(event: Event) {
  console.log('Elemento clickeado:', event);
}
```

Nota importante: Aunque en JavaScript estamos acostumbrados a usar `addEventListener()` para gestionar eventos, Angular recomienda usar event binding con paréntesis `(evento)="función()` por varias razones. Usar `addEventListener()` en Angular es técnicamente posible pero va contra las buenas prácticas del framework y pierdes optimizaciones como la detección de cambios automática.

ENLACE BIDIRECCIONAL - DOS DIRECCIONES

Two-way Data Binding en Angular permite sincronizar automáticamente un valor entre el modelo (TypeScript) y la vista (HTML). Cuando el usuario actualiza el valor en la vista, el modelo se actualiza, y viceversa. Este mecanismo es especialmente útil en formularios y componentes personalizados, donde se necesita mantener sincronizados los datos entre la vista y el modelo.

```
<my-cmp [(elemento)]="userMail">
```

Nota: Esta sintaxis `[( )]` se llama "**banana in a box**".

Este código es equivalente a escribir manualmente:

```
<my-cmp
  [elemento]="userMail"
  (elementoChange)="userMail=$event">
```

En este caso [elemento] envía el valor del modelo (userMail) a la vista y (elementoChange) escucha los cambios en la vista y actualiza userMail en el modelo.

Más adelante usaremos este concepto en ejemplos prácticos, como en formularios dinámicos o componentes reutilizables.

COMUNICACIÓN CON @INPUT - PADRE -> HIJO

Los componentes se pueden anidar y un componente puede contener uno o varios componentes dentro. Cuando esto ocurre, muchas veces en nuestras apps necesitamos enviar información desde un componente padre (el que contiene) a un componente hijo (el que está dentro), para que el hijo reciba datos y los use en su vista o lógica.

Angular permite hacer esto usando el decorador @Input(), que marca una propiedad del componente hijo como "receptora" de datos desde fuera. Este concepto funciona de forma similar a las props en frameworks como React o Vue.

En src/app/user/user.ts - HIJO

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-user',
  standalone: false,
  templateUrl: './user.html',
  styleUrls: ['./user.css'],
})
export class User {
  public username = 'Yoda';
  @Input() name = '';
  @Input() specie = '';
  @Input() age = 0;
}
```

En src/app/user/user.html - HIJO

```
<span>Username: {{ username }}</span><br />
<span>The user's name: {{ name }}</span><br />
<span>The user's specie: {{ specie }}</span><br />
<span>The user's age: {{ age }}</span><br />
```

En src/app/app.html - PADRE

```
...
<div>
  <section>
    <app-user name="Master" specie="No human" [age]=25></app-user>
  </section>
</div>
...
</main>
```

Nota: Para recibir un string literal no es necesario usar [].
Con otro tipo de datos:

```
<app-user [age]="25">
<app-user [active]="true">
```

Si queremos pasar valores desde propiedades del padre:

En src/app/app.ts - PADRE

```
...
public faction = 'Republic';
...
```

En src/app/app.html - PADRE

```
...
<section>
  <app-user name="Master" specie="No human"
    [faction]="faction"></app-user>
</section>
...
```

En src/app/user/user.ts - HIJO

```
...
@Input() faction = '';
...
```

En src/app/user/user.html - HIJO

```
...
<span>The user's faction: {{ faction }}</span>
...
```

Indica que este valor debe ser proporcionado obligatoriamente por el componente padre:

```
@Input({ required: true }) bookTitle: string = '';
```

Si el componente padre no proporciona bookTitle, Angular mostrará un error en tiempo

de compilación.

Uso desde el componente padre:

```
<app-book-details  
[bookTitle]="varlibroElegido"></app-book-details>  
<app-book-details [bookTitle]='Star Wars'></app-book-details>
```

Modificar valores antes de asignarlos con setter. Puedes usar setter para transformar el valor que llega desde el padre antes de usarlo:

```
private _price: number = 0;  
@Input()  
set price(value: number) {  
  this._price = value * 1.21; // Aplica 21% de IVA  
}  
  
get price(): number {  
  return this._price;  
}
```

Uso desde el componente padre:

```
<app-book-details [price]="20"></app-book-details>
```

COMUNICACIÓN CON @OUTPUT - HIJO -> PADRE

Hasta ahora vimos cómo enviar datos del padre al hijo con @Input. pero, ¿qué pasa si queremos que el hijo avise al padre de que ocurrió algo? Por ejemplo: el usuario pulsó un botón, se añadió un elemento a una lista, se completó una tarea, etc.

Para esto usamos el decorador @Output junto con **EventEmitter**, que nos permite emitir eventos desde el hijo hacia el padre, y que el padre pueda "escucharlos" y reaccionar.

```
ng generate component child --skip-tests
```

En src/app/child/child.ts - HIJO

```
import { Component, Output, EventEmitter } from '@angular/core';  
  
@Component({  
  selector: 'app-child',  
  standalone: false,  
  templateUrl: './child.html',  
  styleUrls: ['./child.css'],  
})  
export class Child {  
  @Output() addItemChildEvent = new EventEmitter<string>();  
  
  public addItemChild() {  
    this.addItemChildEvent.emit('🐼');  
  }  
}
```

```
}
```

En `src/app/child/child.html` y `child.css` - HIJO

```
<button class="btn" (click)="addItemChild()">Añade tortuga (Btn Child)</button>

.btn { padding: 5px; }
```

En `src/app/app.ts` - PADRE

```
...

public itemsParent: string[] = [];
public addItemParent(newItem: string) {
  this.itemsParent.push(newItem);
}
}
```

En `src/app/app.html`

```
...
<div>
  <app-child (addItemChildEvent)="addItemParent($event)" />
  <p>Lista padre:</p>
  <ul>
    @for (item of itemsParent; track $index) {
      <li>{{item}}</li>
    }
  </ul>
</div>
<router-outlet />
</main>
```

CARGA DIFERIDA DE COMPONENTES **EN LA VISTA** (Deferrable Views)

En aplicaciones grandes, a veces hay componentes que no necesitamos cargar inmediatamente.

Por ejemplo:

- Secciones que están muy abajo en la página.
- Componentes pesados que no se usan hasta que el usuario interactúa.
- Recursos que queremos cargar solo cuando hagan falta.

Para ello, Angular ofrece una forma moderna de retrasar la carga **de partes de la plantilla** hasta que se cumpla alguna condición (como que el usuario haya llegado a esa parte de la página cuando haga clic en algo, o cuando el navegador esté libre).

Angular introduce las Deferrable Views con la nueva sintaxis **@defer**. Vinculado a esto haremos uso también de **@placeholder**, de un **@loading** y de un **@error**

- **@defer**. Aquí va el contenido que quieres cargar más tarde (por ejemplo, un componente pesado). Solo cuando se cumpla la condición de carga (como que entre al viewport o el navegador esté libre).
- **@placeholder**. Contenido que se muestra al principio, antes de que empiece la carga del bloque diferido. Justo al renderizar la página, mientras espera que empiece la carga.
- **@loading**. Contenido que aparece mientras se está cargando el bloque de **@defer**.
- **@error** (opcional) Muestra contenido si ocurre un error al cargar el bloque de **@defer**.

```
@defer {  
  <!-- Componente pesado que se carga después -->  
  <app-heavy-component />  
}  
@placeholder {  
  <!-- Contenido inicial ligero -->  
  <p>Este contenido aparece primero...</p>  
}  
@loading {  
  <!-- Mientras carga el componente pesado -->  
  <p>Cargando componente...</p>  
}  
@error {  
  <!-- Si falla la carga -->  
  <p>Error al cargar el componente</p>  
}
```

Primero vamos a crear el componente desde cli:

```
ng generate component comments --skip-tests
```

Vamos a `src/aap/comment.html` y añadimos algo de contenido.

```
<ul>  
  <li>Salido de la duda, libre de las tinieblas, cantando al Sol galopó hacia el  
amanecer, desnudando la espada.  
  Encendió una nueva esperanza, y murió esperanzado; fue más allá de la muerte,  
el miedo y el destino; dejó atrás la  
  ruina, y la vida, y entró en la larga gloria...</li>  
  <li>No es oro todo lo que reluce, ni toda la gente errante anda perdida; a las  
raíces profundas no llega la escarcha;  
  el viejo vigoroso no se marchita. De las cenizas subirá un fuego, y una luz</ul>
```

```
asomará en las sombras; el descoronado
  será de nuevo rey, forjarán otra vez la espada rota...</li>
<li>Galopaba con un furor demente, como si la fervorosa sangre guerrera de sus
antepasadosle corriera por las venas
  con un fuego nuevo; y transportado por Crinblanca parecía un dios de la
antigüedad, el propio Orome el Grande, se
  hubiera dicho, en la batalla de Valar, cuando el mundo era joven. El escudo
de oro resplandecía y centelleaba como
  una imagen del sol, y la hierba reverdecía alrededor de las patas del caballo.
Pues llegaba la mañana, la mañana y un
  viento del mar; y ya se disipaban las tinieblas; y los hombres de Mordor
gemían, y conocían el pánico, y huían y
  morían, y los cascos de la ira pasaban sobre ellos. Y de pronto los ejércitos
de Rohan rompieron a cantar, y
  cantaban mientras mataban, pues el júbilo de la batalla estaba en todos
ellos, y los sonidos de ese canto que era
  hermoso y terrible llegaron aun a la ciudad...</li>
</ul>
```

Ahora vamos a `src/app/app.html` y usamos `@defer`. Esto carga el componente `comments` cuando el navegador esté libre. El texto `@placeholder` aparece mientras esperamos que empiece la carga. Y mostramos texto de `@loading` durante la carga del componente.

```
...
<div>
  @defer (on viewport){
    <app-comments></app-comments>
  }@placeholder {
    <p>Comentarios futuros.</p>
  }@loading (minimum 2s) {
    <p>Cargando...</p>
  }@error {
    <p>Error en la cargar</p>
  }
</div>
<router-outlet />
</main>
```

*(on viewport): todo esto sucede **cuando hacemos scroll** y el bloque entra en pantalla. Es el disparador. Si reduces el tamaño de la ventana puedes forzar el scroll y ver el comportamiento.*

(minimum 2s): el mensaje de carga se mantiene al menos 2 segundos, aunque cargue antes.

Los disparadores más comunes de `@defer` que controlan cuándo se carga el contenido diferido son:

- **on viewport:** Carga el contenido cuando entra en la pantalla.
- **on interaction:** Carga el contenido cuando el usuario interactúa (clic, tecla, hover, etc.).
- **on idle:** Carga el contenido cuando el navegador está inactivo.
- **on timer(ms):** Carga el contenido después de un tiempo en milisegundos.

En Angular, `@defer` y **Lazy Loading** son conceptos relacionados, pero diferentes. Ambos optimizan el rendimiento evitando cargar componentes innecesarios al inicio, pero se usan en escenarios distintos.

@defer: Retrasa la carga de componentes específicos dentro de una plantilla (.html o template en `@Component`). Se activa cuando se cumplen ciertas condiciones (por ejemplo, cuando un usuario hace scroll o el navegador está ocioso). Se usa para componente no críticos.

Lazy Loading: Retrasa la carga de módulos y rutas completas hasta que el usuario navegue a ellas. Se usa por ejemplo para secciones enteras de la app que el usuario no visitará siempre como el Panel de administración.

```
const routes: Routes = [
  { path: '', component: HomeComponent }, // Carga inmediata
  { path: 'admin', loadChildren: () =>
    import('./admin/admin.module').then(m => m.AdminModule)
  }
  // Lazy Loading
];
```

MEJORA EL RENDIMIENTO DE IMÁGENES

Las imágenes son uno de los elementos que más afectan al rendimiento de una web. Si no las optimizamos bien, pueden hacer que la página cargue lenta, afectando al usuario y al SEO. Angular nos facilita este trabajo con la directiva **NgOptimizedImage**, que aplica buenas prácticas de forma automática.

NgOptimizedImage:

- Optimiza la carga de imágenes.
- Previene saltos de diseño (layout shift) al exigir tamaños definidos.
- Mejora los tiempos de carga y prioriza las imágenes importantes.
- Permite usar loaders para gestionar imágenes desde CDNs.

Para ello, importamos la directiva en `src/app/app-module.ts`

```
import { NgModule, provideBrowserGlobalErrorListeners } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

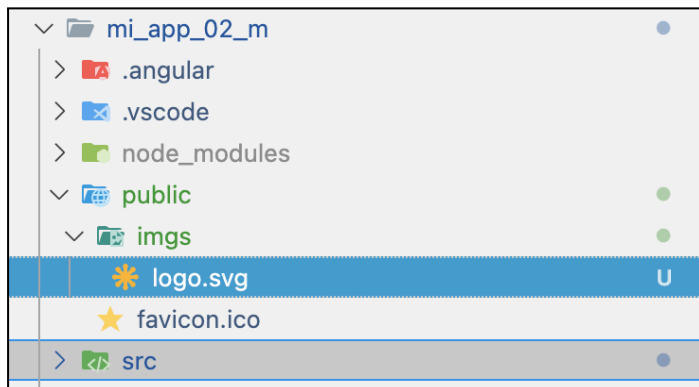
import { AppRoutingModule } from './app-routing-module';
import { App } from './app';
import { User } from './user/user';
import { Child } from './child/child';
import { Comments } from './comments/comments';
```



```
import { NgOptimizedImage } from '@angular/common';

@NgModule({
  declarations: [
    App,
    User,
    Child,
    Comments
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    NgOptimizedImage
  ],
  providers: [
    provideBrowserGlobalErrorListeners(),
  ],
  bootstrap: [App]
})
export class AppModule { }
```

Descarga el logo de angular, renombrarlo como logo.svg y guárdalo en una carpeta con el nombre imgs dentro de public:



[Descargar logo svg aquí.](#)

Tradicionalmente se usaba src/assets/, a partir de Angular 17 es totalmente válido trabajar con public/ y mantenerlo como tu carpeta de recursos.

Ahora vamos a modificar el componente **app**. Para ello usamos `ngSrc` en vez de `src`, tanto para imágenes estáticas como dinámicas. Es obligatorio indicar el tamaño de las imágenes para evitar problemas visuales al cargar (layout shift):

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-user',
  standalone: false,
  templateUrl: './user.html',
  styleUrls: ['./user.css'],
})
export class User {
  public username = 'Yoda';
  @Input() name = '';
  public logoUrl = '/imgs/logo.svg';
  public logoAlt = 'Angular logo';
}

<span>Username: {{ username }}</span><br />
<span>The user's name: {{ name }}</span>
<ul>
  <li>
    Static Image:
    <img ngSrc="/imgs/logo.svg" alt="Angular logo" width="32" height="32">
  </li>
  <li>
    Dinamic Image:
    <img [ngSrc]="logoUrl" [alt]="logoAlt" width="32" height="32">
  </li>
</ul>
```

Sugerencia 1. Para casos donde queremos que la imagen ocupe todo su contenedor usamos `fill`. Aunque el contenedor debe tener `position: relative;`, `fixed;` o `absolute;`

```
<div class="image-container">
  <img ngSrc="https://example.com/image.png" fill />
</div>
```

Sugerencia 2. Si tienes una imagen clave (como la de portada o una imagen muy grande al inicio), usa `priority` para priorizar imágenes importantes:

```
<img ngSrc="https://example.com/hero.png" width="800" height="600" priority />
```

Sugerencia 3. Si usas un CDN como `Imgix`, puedes configurar un loader para que Angular construya automáticamente las URLs:

```
import { provideImgixLoader } from '@angular/common';
```

```
providers: [  
  provideImgixLoader('https://my.base.url/')  
]
```

Y en la plantilla:

```
<img ngSrc="image.png" width="800" height="600" />
```

Angular generará la URL completa: <https://my.base.url/image.png>

GESTIONAR RUTAS EN ANGULAR CON MÓDULOS CLÁSICOS

En cualquier aplicación que tenga más de una página o vista, necesitamos un sistema de rutas que nos permita navegar entre diferentes componentes sin recargar la página.

Ahí es donde entra Angular Router.

Aunque en versiones recientes de Angular (17+) la configuración del router puede hacerse sin módulos (standalone: true), en nuestro caso usamos la arquitectura clásica con módulos (standalone: false), por lo que seguiremos el método tradicional.

Lo primero que vamos a hacer es crear un nuevo componente que se muestre cuando estemos en la ruta /:

```
ng generate component home --skip-tests
```

Después nos llevamos a ese home todo el contenido que teníamos en AppComponent y también le definimos un atributo tittleComp.

De este modo el **App** sería como el **layout principal de una web** (con menú y pie de página), y dentro de él cambiamos el contenido principal según la ruta gracias al <router-outlet>.

En el componente **home**:

```
import { Component } from '@angular/core';  
  
@Component({  
  selector: 'app-home',  
  standalone: false,  
  templateUrl: './home.html',  
  styleUrls: ['./home.css'],  
})  
export class Home {  
  public isServerRunning = false;  
  public characters = [  
    { id: 0, name: 'Rachel' },  
    { id: 1, name: 'Monica' },  
    { id: 2, name: 'Phoebe' },  
    { id: 3, name: 'Ross' },  
    { id: 4, name: 'Chandler' },  
    { id: 5, name: 'Joey' },  
  ]  
}
```

```
];
public isEditable = true;
public message = '';
public boxClass = 'box-class';
public onMouseOverAction() {
    this.message = "Let's go!";
}
public onMouseOutAction() {
    this.message = '';
}

public itemsParent: string[] = [];
public addItemParent(newItem: string) {
    this.itemsParent.push(newItem);
}
public titleComp = 'HOME';
}
```

```
<h2>{{ titleComp }}</h2>
<hr/>
<div>
    <section>
        <app-user></app-user>
    </section>
    <ul>
        <li><app-user></app-user></li>
        <li><app-user></app-user></li>
    </ul>
</div>
<div>@if (isServerRunning){
    <span>Yes, the server is running</span>
}
@else {
    <span>No, the server is not running</span>
}
</div>
<div>
    @for (character of characters; track character.id){
        <p>{{ character.id }} - {{ character.name }}</p>
    }
</div>
<div [contentEditable]="isEditable">
    Puede editar este texto si isEditable es true.
</div>
<div [class]="boxClass" (mouseover)="onMouseOverAction()"
(mouseout)="onMouseOutAction()">
    Mira: {{ message }}
</div>
<div>
    <section>
        <app-user name="Master"></app-user>
    </section>
</div>
<div>
    <app-child (addItemChildEvent)="addItemParent($event)" />
    <p>Lista padre:</p>
    <ul>
        @for (item of itemsParent; track $index) {
```

```

        <li>{{item}}</li>
      }
    </ul>
  </div>

  <div>
    @defer (on viewport){
      <app-comments></app-comments>
    }@placeholder {
      <p>Comentarios futuros.</p>
    }@loading (minimum 2s) {
      <p>Cargando...</p>
    }@error {
      <p>Error en la cargar</p>
    }
  </div>

```

También añadimos un atributo title al componente **user**:

```
import { Component, Input } from '@angular/core';
```

```

@Component({
  selector: 'app-user',
  standalone: false,
  templateUrl: './user.html',
  styleUrls: ['./user.css'],
})
export class User {
  public username = 'Yoda';
  @Input() name = '';
  public imageUrl = '/imgs/logo.svg';
  public logoAlt = 'Angular logo';
  public titleComp = 'USER';
}

```

```

<h2>{{titleComp}}</h2>
<hr />
<span>Username: {{ username }}</span><br />
<span>The user's name: {{ name }}</span>
<ul>
  <li>
    Static Image:
    <img ngSrc="/imgs/logo.svg" alt="Angular logo" width="32" height="32">
  </li>
  <li>
    Dinamic Image:
    <img [ngSrc]="imageUrl" [alt]="logoAlt" width="32" height="32">
  </li>
</ul>

```

Dejamos el **App** así y añadimos un menú de navegación simple:

```

import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-root',

```

```
templateUrl: './app.html',
standalone: false,
styleUrl: './app.css',
})
export class App {
  protected readonly title = signal('mi_app_02_m');
}
```

```
<main class="main">
  <div class="content">
    <h1>Hello, {{title()}}</h1>
  </div>
  <nav>
    <a routerLink="/">Home</a>
    |
    <a routerLink="/user">User</a>
  </nav>
  <router-outlet></router-outlet>
</main>
```

Aquí usamos **routerLink** en lugar de `<a href="...">`, para que Angular gestione las rutas sin recargar la página.

Hecho esto ahora comprobamos que el `app.module.ts` ya tiene el `AppRoutingModule` en los imports desde que se creó la plantilla del proyecto por defecto.

```
src > app > ts app-module.ts > ...
1  import { NgModule, provideBrowserGlobalErrorListeners } from '@angular/core';
2  import { BrowserModule } from '@angular/platform-browser';
3
4  import { AppRoutingModule } from './app-routing-module';
5  import { App } from './app';
6  import { User } from './user/user';
7  import { Child } from './child/child';
8  import { Comments } from './comments/comments';
9
10 import { NgOptimizedImage } from '@angular/common';
11 import { Home } from './home/home';
12
13 @NgModule({
14   declarations: [App, User, Child, Comments, Home],
15   imports: [BrowserModule, AppRoutingModule, NgOptimizedImage],
16   providers: [provideBrowserGlobalErrorListeners()],
17   bootstrap: [App],
18 })
19 export class AppModule {}
```

Vamos ya con las rutas en sí. Dado que estamos con `standalone` false, en vez de `app.routes.ts`, usamos un módulo específico para gestionar las rutas, llamado **app-routing-module.ts**.

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { User } from './user/user';
```

```
import { Home } from './home/home';

const routes: Routes = [
  { path: '', title: 'App Home', component: Home },
  { path: 'user', title: 'App User', component: User },
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule],
})
export class AppRoutingModule {}
```

standalone false → Rutas en Módulo: **app-routing-module.ts**
standalone true → Rutas en **app.routes.ts**

ROUTING CON PARÁMETROS EN ANGULAR

Un componente puede recibir un parámetro en la ruta porque sea necesario para poder mostrar información específica de un elemento. Consulta este ejemplo que haremos en el siguiente ejercicio prácticos: [ROUTING CON PARÁMETROS EN ANGULAR](#)

GUARDS EN ANGULAR

Podemos proteger las rutas bajo algún criterio, por ejemplo, ruta disponibles solo para usuarios autenticados. Consulta este ejemplo que haremos en el siguiente ejercicio prácticos: [LOGIN EN ANGULAR USANDO GUARD \(CON LOCALSTORAGE\)](#)

FORMS EN ANGULAR

Los formularios son fundamentales en cualquier aplicación porque permiten recoger información del usuario. En Angular, existen dos enfoques principales para manejar formularios:

- Template-driven forms (formularios basados en plantilla): son más declarativos y fáciles de usar, ya que la mayor parte de la lógica se define directamente en el HTML usando ngModel. Son ideales para formularios sencillos y de poco mantenimiento, pero ofrecen menos control sobre la validación y la gestión de datos.
- Reactive forms (formularios programáticos): permiten manejar los formularios desde TypeScript con FormGroup y FormControl, lo que facilita la validación, la gestión de estados y la actualización dinámica de campos. Son más escalables y adecuados para formularios complejos o dinámicos.

FORMS EN ANGULAR - Template-driven Forms

En esta sección, veremos el primer enfoque: **Template-driven Forms**.

Creemos un componente con nombre **forms** para hacer las pruebas:

```
ng generate component forms --skip-tests
```

Y preparamos una ruta para el mismo y una opción en el menú de navegación.

```
...
<nav>
  <a routerLink="/">Home</a>
  |
  <a routerLink="/user">User</a>
  |
  <a routerLink="/forms">Form</a>
</nav>
...

...
const routes: Routes = [
  { path: '', title: 'App Home', component: Home },
  { path: 'user', title: 'App User', component: User },
  { path: 'forms', title: 'App Form', component: Forms },
];
...
```

Hecho esto, como estamos usando módulos clásicos (standalone: false), debemos importar FormsModule en app-module.ts


```
import { NgModule, provideBrowserGlobalErrorListeners } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppRoutingModuleModule } from './app-routing-module';
import { App } from './app';
import { User } from './user/user';
import { Child } from './child/child';
import { Comments } from './comments/comments';

import { NgOptimizedImage } from '@angular/common';
import { Home } from './home/home';
import { Forms } from './forms/forms'; //Automáticamente

import { FormsModule } from '@angular/forms'; //Manualmente

@NgModule({
  declarations: [App, User, Child, Comments, Home, Forms],
  imports: [BrowserModule, AppRoutingModuleModule, NgOptimizedImage, FormsModule],
  providers: [provideBrowserGlobalErrorListeners()],
  bootstrap: [App],
})
export class AppModule {}
```

Ahora preparamos la vista y enlazamos el input con una variable del componente usando [(ngModel)]. Es una forma de Two-way Data Binding en Angular.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-forms',
  standalone: false,
  templateUrl: './forms.html',
  styleUrls: ['./forms.css'],
})
export class Forms {
  public titleComp = 'FORMS';
  public favoriteFramework = '';
  public module = {
    name: '',
    mark: '',
  };
}

<h2>{{titleComp}}</h2>
<hr>
<div>
  <h3>Template-driven Forms</h3>
  <form>
    <div>
      <label for="framework">
        Framework favorito:
      </label>
      <input id="framework" name="framework" type="text"
        [(ngModel)]="favoriteFramework">
```

```
</div>
<div>
  <label for="nameModule">Módulo:</label>
  <input id="nameModule" name="nameModule" type="text"
    [(ngModel)]="module.name">
</div>
<div>
  <label for="mark">Nota:</label>
  <input id="mark" name="mark" type="number"
    [(ngModel)]="module.mark">
</div>
<div>
  <p>Tu framework es: {{favoriteFramework}}</p>
  <p>{{module.name}} - {{module.mark}}</p>
</div>
</form>
</div>
```

Nota: Esta sintaxis [()] se llama "**banana in a box**".

FORMS EN ANGULAR - Reactive Forms

En algunos casos, queremos manejar los formularios desde el código TypeScript, sin depender tanto de la plantilla. Para eso, Angular ofrece **Reactive Forms**, donde controlamos los formularios de forma programática usando FormGroup y FormControl.

Recuerda que como trabajamos con módulos clásicos (standalone: false), no importamos ReactiveFormsModule en el componente, sino en el AppModule:

```
import { NgModule, provideBrowserGlobalErrorListeners } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppRoutingModule } from './app-routing-module';
import { App } from './app';
import { User } from './user/user';
import { Child } from './child/child';
import { Comments } from './comments/comments';

import { NgOptimizedImage } from '@angular/common';
import { Home } from './home/home';
import { Forms } from './forms/forms';

import { FormsModule } from '@angular/forms';
import { ReactiveFormsModule } from '@angular/forms';

@NgModule({
  declarations: [App, User, Child, Comments, Home, Forms],
  imports: [BrowserModule, AppRoutingModule, NgOptimizedImage,
    FormsModule, ReactiveFormsModule],
})
```

```
providers: [provideBrowserGlobalErrorListeners()],
bootstrap: [App],
}))
export class AppModule {}
```

Crear el FormGroup y los FormControl en el componente. El método handleSubmit() captura los valores al enviar el formulario.

```
import { Component } from '@angular/core';
import { FormGroup, FormControl } from '@angular/forms';

@Component({
  selector: 'app-forms',
  standalone: false,
  templateUrl: './forms.html',
  styleUrls: ['./forms.css'],
})
export class Forms {
  public titleComp = 'FORMS';
  public favoriteFramework = '';
  public module = {
    name: '',
    mark: '',
  };

  public profileForm = new FormGroup({
    name: new FormControl(''),
    email: new FormControl(''),
  });

  public handleSubmit() {
    alert(`Nombre: ${this.profileForm.value.name} (Email:
      ${this.profileForm.value.email})`);
  }
}

...
<div>
  <h3>Reactive Forms</h3>

  <form [formGroup]="profileForm" (ngSubmit)="handleSubmit()">
    <label for="name">Nombre</label>
    <input id="name" name="name" type="text" formControlName="name" />

    <label for="email">Email</label>
    <input id="email" name="email" type="email" formControlName="email" />
  </form>
</div>
```

```
<button type="submit">Submit</button>
</form>

<p>Name: {{ profileForm.value.name }}</p>
<p>Email: {{ profileForm.value.email }}</p>
</div>
```

VALIDACIÓN EN FORMULARIOS REACTIVOS

En los formularios, es esencial validar los datos antes de enviarlos para asegurarnos de que los usuarios introducen información correcta. Angular proporciona herramientas de validación integradas que facilitan esta tarea.

- Usamos Validators para definir reglas en cada campo.
- Controlamos la validez del formulario con .valid.

```
import { Component } from '@angular/core';
import { FormGroup, FormControl, Validators } from '@angular/forms';

@Component({
  selector: 'app-forms',
  standalone: false,
  templateUrl: './forms.html',
  styleUrls: ['./forms.css'],
})
export class Forms {
  public titleComp = 'FORMS';
  public favoriteFramework = '';
  public module = {
    name: '',
    mark: '',
  };

  public profileForm = new FormGroup({
    name: new FormControl('', [Validators.required,
                               Validators.minLength(3)]),
    email: new FormControl('', [Validators.required, Validators.email]),
  });

  public handleSubmit() {
    if (this.profileForm.valid) {
      alert(`Nombre: ${this.profileForm.value.name} (Email:
            ${this.profileForm.value.email})`);
    }
    else{
      alert('El formulario tiene errores.');
```

```
...
<div>
  <h3>Reactive Forms</h3>

  <form [formGroup]="profileForm" (ngSubmit)="handleSubmit()">
    <label for="name">Nombre</label>
    <input id="name" name="name" type="text" formControlName="name" />
    @if (profileForm.controls.name.invalid &&
      profileForm.controls.name.touched) {
      <span>Introduce un nombre de al menos 3 caracteres.</span>
    }
    <br />
    <label for="email">Email</label>
    <input id="email" name="email" type="email" formControlName="email"/>
    @if (profileForm.controls.email.invalid &&
      profileForm.controls.email.touched) {
      <span>Introduce un correo válido.</span>
    }
    <br />
    <button type="submit" [disabled]="!profileForm.valid">Submit</button>
  </form>

  <p>Name: {{ profileForm.value.name }}</p>
  <p>Email: {{ profileForm.value.email }}</p>
</div>
```

*touched: El usuario ha interactuado con el campo (hizo foco y luego lo perdió)

Validadores más comunes:

Validador	Descripción	Ejemplo
Validators.required	Campo obligatorio	new FormControl('', Validators.required)
Validators.email	Email válido	new FormControl('', Validators.email)
Validators.minLength(n)	Longitud mínima	Validators.minLength(3)
Validators.maxLength(n)	Longitud máxima	Validators.maxLength(50)
Validators.min(n)	Valor numérico mínimo	Validators.min(18)
Validators.max(n)	Valor numérico máximo	Validators.max(100)
Validators.pattern(regex)	Patrón personalizado	Validators.pattern(/^[0-9]+\$/)

Ahora vamos a introducir una validación **personalizada**:

```
import { Component } from '@angular/core';
import {
  FormGroup,
  FormControl,
  Validators,
```

```
AbstractControl,
ValidationErrors,
} from '@angular/forms';

@Component({
  selector: 'app-forms',
  standalone: false,
  templateUrl: './forms.html',
  styleUrls: ['./forms.css'],
})
export class Forms {
  public titleComp = 'FORMS';
  public favoriteFramework = '';
  public module = {
    name: '',
    mark: '',
  };

  public profileForm = new FormGroup({
    name: new FormControl('', [Validators.required, Validators.minLength(3)]),
    email: new FormControl('', [Validators.required, Validators.email,
      this.emailEduValidator]),
  });

  public handleSubmit() {
    if (this.profileForm.valid) {
      alert(`Nombre: ${this.profileForm.value.name} (Email:
        ${this.profileForm.value.email})`);
    }
    else {
      alert('El formulario tiene errores.');
```

```
    }
  }
  public emailEduValidator(control: AbstractControl): ValidationErrors | null {
    const email = control.value;
    return email && email.endsWith('@educastillalamancha.es')
      ? null : { emailEdu: true };
  }
}

...
<div>
  <h3>Reactive Forms</h3>

  <form [formGroup]="profileForm" (ngSubmit)="handleSubmit()">
    <label for="name">Nombre</label>
    <input id="name" name="name" type="text" formControlName="name" />
    @if (profileForm.controls.name.invalid && profileForm.controls.name.touched) {
    <span>Introduce un nombre.</span>
    }
    <br />
    <label for="email">Email</label>
    <input id="email" name="email" type="email" formControlName="email" />
    @if (profileForm.controls.email.invalid &&
      profileForm.controls.email.touched) {
    <span>Introduce un correo válido. </span>
    }
    @if (profileForm.controls.email.errors?.['emailEdu'] &&
```

```
        profileForm.controls.email.touched) {  
        <span>Introduce un correo @educastillalamancha.es.</span>  
        }  
        <br />  
        <button type="submit" [disabled]="!profileForm.valid">Submit</button>  
    </form>  
  
    <p>Name: {{ profileForm.value.name }}</p>  
    <p>Email: {{ profileForm.value.email }}</p>  
</div>
```

CREACIÓN DE SERVICIOS INYECTABLES

Uno de los principios clave en Angular es la Inyección de Dependencias (DI), que permite que los servicios sean inyectados automáticamente en cualquier componente que los necesite.

¿Para qué se usa un servicio en Angular?

- Para gestionar datos (como llamadas a APIs o almacenamiento local).
- Para compartir lógica entre diferentes componentes.
- Para mantener el código modular y limpio.

Antes de nada vamos a crear un nuevo componente: cars

ng generate **component cars** --skip-tests

Y preparamos una ruta para el mismo y una opción en el menú de navegación.

```
const routes: Routes = [  
  { path: '', title: 'App Home', component: Home },  
  { path: 'user', title: 'App User', component: User  
  },  
  { path: 'forms', title: 'App Form', component: Forms  
  },  
  { path: 'cars', title: 'App Car', component: Cars },  
];
```

```
<nav>  
  <a routerLink="/">Home</a>  
  |  
  <a routerLink="/user">User</a>  
  |  
  <a routerLink="/forms">Form</a>  
  |  
  <a routerLink="/cars">Car</a>  
</nav>
```

Después aquí creamos un servicio

ng generate **service services/car.service** --skip-tests

En Angular, un servicio necesita `@Injectable` para ser inyectable dentro de otros componentes. Por defecto también crea `@Injectable({ providedIn: 'root' })` que hace que el servicio esté disponible globalmente en la aplicación, sin necesidad de declararlo en providers dentro de un módulo.

Vamos a añadir un atributo y unos métodos para probar el servicio:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})

export class CarService {
  private _cars = ['Sunflower GT', 'Flexus Sport', 'Sprout Mach One'];

  public get cars(): string[] {
    return this._cars;
  }

  public set cars(newCars: string[]) {
    this._cars = newCars;
  }

  public addCar(car: string): void {
    this._cars.push(car);
  }

  public removeCar(id: number): void {
    this._cars.splice(id, 1);
  }
}
```

Ahora que tenemos el servicio, lo usamos en el componente `cars`

```
import { Component, inject } from '@angular/core';
import { CarService } from '../services/car.service';

@Component({
  selector: 'app-cars',
  standalone: false,
  templateUrl: './cars.html',
  styleUrls: ['./cars.css'],
})
export class Cars {
  public titleComp = 'Cars';
  public carService = inject(CarService);
}
```

```
}  
  
<h2>{{titleComp}}</h2>  
<hr />  
<p>{{ carService.cars }}</p>
```

Angular proporciona la función `inject()` para inyectar servicios directamente en una propiedad de clase. Esto nos permite llamar al getter en la plantilla directamente. **ESTA FORMA DE TRABAJAR ES 'MODERNA'.**

- Permite obtener servicios sin usar el constructor.
- Solo se puede usar `inject()` dentro del contexto de inyección de Angular (componentes, servicios, directivas, pipes).
- **No funciona** en funciones normales o métodos ejecutados después de la inicialización.

La forma clásica es usando un constructor:

```
export class UserComponent {  
  //carService = inject(CarService); Moderno  
  
  cars: string[] = [];  
  
  constructor(private carService: CarService) {  
    this.cars = this.carService.cars;  
  }  
  ...  
}
```

`constructor(private carService: CarService) {}` → Inyecta el servicio.

- El servicio se inyecta **como una propiedad de clase automáticamente**.

En el componente que queremos usar el servicio lo 'inyectamos' en:

- `ngOnInit()` → Ejecuta la lógica después de que Angular ha inicializado el componente.

Cuidado:

Si el servicio inyectado depende de una tarea costosa, esto podría fallar:

```
export class Cars {  
  public titleComp = 'Cars';  
  public carService = inject(CarService);  
  public display = this.carService.cars.join('|');  
}
```

Es mejor:

```
export class Cars {  
  public titleComp = 'Cars';  
  public carService = inject(CarService);  
  public display = '';  
  
  ngOnInit() {  
    this.display = this.carService.cars.join('|');  
  }  
}
```

FILTROS - PIPES

Los Pipes (Filtros) en Angular son funciones que transforman datos en las plantillas, de manera sencilla y sin afectar la lógica del componente. **Los Pipes NO modifican el valor original, solo lo transforman en la vista.**

```
<p>{{ 'Hola Mundo' | uppercase }}</p> <!-- Renderiza: "HOLA MUNDO" -->
<p>{{ miAtributo | lowercase }}</p>
<p>{{ 'bienvenidos a angular' | titlecase }}</p>
<p>{{ 0.25 | percent }}</p>
<p>{{ 1234567.89 | number }}</p>
<p>{{ {name: 'Angular', version: 19} | json }}</p>
```

Los Pipes también pueden recibir parámetros para personalizar su salida. Esto permite formatear números, fechas y monedas según nuestras necesidades.

```
<p>{{ today | date:'longDate' }}</p>
<p>{{ today | date:'short' }}</p>
<p>{{ 1500 | currency:'USD' }}</p>
<p>{{ 3.141592 | number:'1.2-2' }}</p>
...
```

Vamos a probar en nuestro componente **user**:

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-user',
  standalone: false,
  templateUrl: './user.html',
  styleUrls: ['./user.css'],
})
export class User {
  public username = 'Yoda';
  @Input() name = '';

  public logoUrl = '/imgs/logo.svg';
  public logoAlt = 'Angular logo';
  public titleComp = 'USER';

  public salary = 2100.21;
  public birth = new Date('1990-01-01');
}

<h2>{{titleComp}}</h2>
```

```
<hr />
<span>Username: {{ username | lowercase }}</span><br />
<span>The user's name: {{ name | uppercase }}</span>
<p>Salary: {{salary | currency:'EUR' }}</p>
<p>Bith year: {{ birth | date:'fullDate' | uppercase }} </p>
<ul>
  <li>
    Static Image:
    <img ngSrc="/imgs/logo.svg" alt="Angular logo" width="32" height="32">
  </li>
  <li>
    Dinamic Image:
    <img [ngSrc]="logoUrl" [alt]="logoAlt" width="32" height="32">
  </li>
</ul>
```

En aplicaciones standalone: true es necesario realizar estas importaciones para usar Pipes:

```
import {LowerCasePipe} from '@angular/common';
...
imports: [LowerCasePipe],
```

En aplicaciones con módulos tradicionales (standalone: false), los Pipes de Angular ya están disponibles y NO necesitan ser importados manualmente.

Para que Angular adapte las cantidades a euros con coma decimal y punto de miles es necesario configurar `LOCALE_ID` a 'es-ES' en `app-module.ts`:

```
import { LOCALE_ID, NgModule,
  provideBrowserGlobalErrorListeners } from '@angular/core';
...
import { registerLocaleData } from '@angular/common';
import localeEs from '@angular/common/locales/es'; //Default export --> sin {}
registerLocaleData(localeEs);

@NgModule({
  declarations: [App, User, Child, Comments, Home, Forms, Cars],
  imports: [BrowserModule, AppRoutingModule, NgOptimizedImage, FormsModule,
    ReactiveFormsModule],
  providers: [provideBrowserGlobalErrorListeners(),
    { provide: LOCALE_ID, useValue: 'es-ES' }
  ],
  bootstrap: [App],
})
export class AppModule {}
```

Y ya uso:

```
<span>Salary: {{salary | currency:'EUR' }}</span> <!-- Precio en Euros: 1.234,56 € -->
```

CREAR FILTROS - PIPES

Podemos crear Pipes personalizados que nos permiten transformar datos en la plantilla de forma específica cuando los Pipes integrados no son suficientes.

Lo primero es generar el Pipe desde la CLI:

```
ng generate pipe pipes/reverse --skip-tests
```

Lo siguiente es ver si se ha registrado automática al app-module.ts

Ahora definimos la lógica:

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'reverse',
  standalone: false,
})
export class ReversePipe implements PipeTransform {
  transform(value: string): string {
    return value.split('').reverse().join('');
  }
}
```

Y ahora lo usamos por ejemplo en el componente user:

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-user',
  standalone: false,
  templateUrl: './user.html',
  styleUrls: ['./user.css'],
})
export class User {
  public username = 'Yoda';
  @Input() name = '';

  public logoUrl = '/imgs/logo.svg';
}
```

```
public logoAlt = 'Angular logo';
public titleComp = 'USER';

public salary = 2100.21;
public birth = new Date('1990-01-01');
public desc = 'Angular Mola';
}
```

```
<h2>{{titleComp}}</h2>
<hr />
<span>Username: {{ username | lowercase }}</span><br />
<span>The user's name: {{ name | uppercase }}</span>
<p>Salary: {{salary | currency:'EUR' }}</p>
<p>Birth: {{ birth | date:'fullDate' | uppercase }} </p>
<p>Original: {{ desc }}</p>
<p>Invertido: {{ desc | reverse }}</p>
<ul>
  <li>
    Static Image:
    <img ngSrc="/imgs/logo.svg" alt="Angular logo" width="32" height="32">
  </li>
  <li>
    Dinamic Image:
    <img [ngSrc]="logoUrl" [alt]="logoAlt" width="32" height="32">
  </li>
</ul>
```

ASYNC/AWAIT

Como es lógico, en Angular, podemos hacer peticiones HTTP usando fetch con async/await para obtener información de servicios externos.

Vamos a hacer unas pruebas con la API pública de Star Wars.

En primer lugar vamos a crear una interfaz adaptada a nuestro problema para recoger solo la información que nos interesa de la API. Esto nos ayuda a tener autocompletado y detección de errores en TypeScript:

```
ng generate interface interfaces/i-film-sw.interface
```

```
export interface IFilmSW {  
  title: string;  
  director: string;  
  year: number;  
  episodeId: number;  
  openingCrawl: string;  
}
```

Ahora vamos a crear el servicio:

```
ng generate service services/apisw.service --skip-tests
```

```
import { Injectable } from '@angular/core';  
import { IFilmSW } from '../interfaces/i-film-sw.interface';  
  
@Injectable({  
  providedIn: 'root',  
})  
export class ApiswService {  
  private readonly API_URL = 'https://swapi.dev/api/films/';  
  
  private convertJsonToInterface(filmApi: Record<string, string>): IFilmSW {  
    return {  
      title: filmApi['title'],  
      director: filmApi['director'],  
      year: parseInt(filmApi['release_date'].split('-')[0]),  
      episodeId: Number(filmApi['episode_id']),  
      openingCrawl: filmApi['opening_crawl'],  
    };  
  }  
  
  async getFilms(): Promise<IFilmSW[]> {  
    try {  
      const response = await fetch(this.API_URL);  

```



```
if (!response.ok) {
  throw new Error(`HTTP Error: ${response.status}`);
}

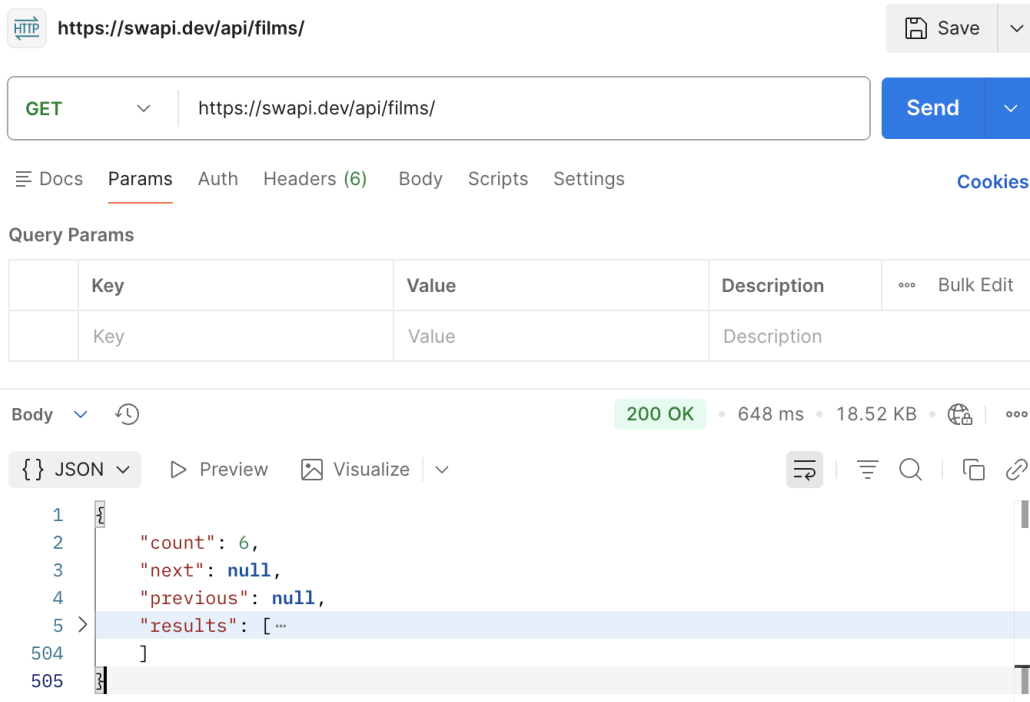
const data: Record<string, unknown> = await response.json();
//En la respuesta en el primer nivel no todos los valores son strings.

// La API devuelve las películas en data.results. Ahí sí es string.
const filmsApi: Record<string, string>[] =
  data['results'] as Record<string, string>[];

const films: IFilmSW[] = filmsApi.map((film: Record<string, string>) =>
  this.convertJsonToInterface(film),
);

return films;
} catch (error) {
  console.error('Error fetching films:', error);
  return [];
}
}
```

Observa lo que devuelve la api. Las películas están "results"



HTTP <https://swapi.dev/api/films/> Save

GET <https://swapi.dev/api/films/> Send

Docs Params Auth Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body 200 OK • 648 ms • 18.52 KB

{ } JSON Preview Visualize

```
1 {
2   "count": 6,
3   "next": null,
4   "previous": null,
5   "results": [ ...
504 ]
505 }
```

Se ha resuelto con `Record<string, string>` por probar algo diferente. Lo habitual es usar `any` en estos casos. El uso de `any` es aceptable en respuestas de APIs REST porque los datos JSON recibidos no tienen tipo en tiempo de compilación; la validación real debe hacerse en tiempo de ejecución mediante parsing y transformación a interfaces tipadas.

```
...
private convertJsonToInterfaceWithAny(filmApi: any): IFilmSW {
  return {
    title: filmApi.title,
    director: filmApi.director,
    year: parseInt(filmApi.release_date.split('-')[0]),
    episodeId: Number(filmApi.episode_id),
    openingCrawl: filmApi.opening_crawl,
  };
}

async getFilmsWithAny(): Promise<IFilmSW[]> {
  try {
    const response = await fetch(this.API_URL);
    if (!response.ok) {
      throw new Error(`HTTP Error: ${response.status}`);
    }
    const data: any = await response.json();
    // La API devuelve las películas en data.results
    const filmsApi: any[] = data['results'];
    const films: IFilmSW[] = filmsApi.map((film: any) =>
      this.convertJsonToInterfaceWithAny(film),
    );
    return films;
  } catch (error) {
    console.error('Error fetching films:', error);
    return [];
  }
}
...
```

Ahora vamos a crear un componente que use este servicio:

```
ng generate component filmsSW --skip-tests
```

Y preparamos una ruta para el mismo y una opción en el menú de navegación.

```
const routes: Routes = [
  { path: '', title: 'App Home', component: Home },
  { path: 'user', title: 'App User', component: User },
```

```
{ path: 'forms', title: 'App Form', component: Forms },
{ path: 'cars', title: 'App Car', component: Cars },
{ path: 'films', title: 'App Films', component: FilmsSW },
];
```

```
<nav>
  <a routerLink="/">Home</a>
  |
  <a routerLink="/user">User</a>
  |
  <a routerLink="/forms">Form</a>
  |
  <a routerLink="/cars">Car</a>
  |
  <a routerLink="/films">Film</a>
</nav>
```

También comprueba que el componente se ha registrado correctamente.

```
import { Component, OnInit, inject, ChangeDetectorRef } from
'@angular/core';
import { ApiswService } from '../services/apisw.service';
import { IFilmSW } from '../interfaces/i-film-sw.interface';

@Component({
  selector: 'app-films-sw',
  standalone: false,
  templateUrl: './films-sw.html',
  styleUrls: ['./films-sw.css'],
})
export class FilmsSW implements OnInit {
  public titleComp = 'FILM';

  private apiswService = inject(ApiswService);
  private cdr = inject(ChangeDetectorRef);
  private _filmsSW: IFilmSW[] = [];

  public loading = true;
  public error = '';

  get filmsSW(): IFilmSW[] {
    return this._filmsSW;
  }
}
```

```
set filmsSW(films: IFilmSW[]) {
  this._filmsSW = films;
}

removeFilmByEpisodeId(episodeId: number): void {
  this._filmsSW = this._filmsSW.filter((film)
    => film.episodeId !== episodeId);
  this.cdr.markForCheck(); //notifica a Angular un cambio.
}

async ngOnInit() {
  await this.loadFilms();
}

async loadFilms(): Promise<void> {
  this.loading = true;
  this.error = '';
  this.cdr.markForCheck(); //notifica a Angular un cambio.

  try {
    this._filmsSW = await this.apiswService.getFilms();
    this._filmsSW.sort((a, b) => a.episodeId - b.episodeId);
  } catch {
    this.error = 'Error loading films';
  } finally {
    this.loading = false;
    this.cdr.markForCheck(); //notifica a Angular un cambio.
  }
}
}
```

```
<h2>{{titleComp}}</h2>
<hr />
@if (loading) {
  <p>Cargando películas de Star Wars...</p>
}

@if (error) {
  <p class="error">{{ error }}</p>
}

@if (!loading && !error) {
  <table>
    <thead>
      <tr>
```

```
<th>Episodio</th>
<th>Título</th>
<th>Director</th>
<th>Año</th>
<th>Acciones</th>
</tr>
</thead>
<tbody>
  @for (film of filmsSW; track film.episodeId;) {
    <tr>
      <td>{{ film.episodeId }}</td>
      <td>{{ film.title }}</td>
      <td>{{ film.director }}</td>
      <td>{{ film.year }}</td>
      <td>
        <button type="button"
          (click)="removeFilmByEpisodeId(film.episodeId)">Borrar</button>
      </td>
    </tr>
  }
</tbody>
</table>

<p><strong>Total de películas:</strong> {{ filmsSW.length }}</p>
}
```

```
.error{
  color:red;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
}

th, td {
  border: 1px solid #ddd;
  padding: 12px;
  text-align: left;
}

th {
  background-color: #f4f4f4;
  font-weight: bold;
}
```

```
button {  
  background-color: #dc3545;  
  color: white;  
  border: none;  
  padding: 12px;  
  cursor: pointer;  
}
```

Alternativa Moderna de NOTIFICAR CAMBIOS: **Signals**

El código anterior utiliza la forma tradicional con `ChangeDetectorRef` para notificar a Angular de los cambios.

También podemos usar **Signals** para gestionar el estado reactivo de forma más simple y eficiente.

Como ya hemos comentado, los Signals son una nueva forma de manejar datos reactivos en Angular. Son variables especiales que:

- Notifican automáticamente a Angular cuando cambian su valor
- No necesitan `ChangeDetectorRef` ni `markForCheck()`
- Hacen el código más simple y legible
- Mejoran el rendimiento de la aplicación

Ahora lo hemos hecho así:

```
private _filmsSW: IFilmSW[] = [];  
private cdr = inject(ChangeDetectorRef);  
  
removeFilm(episodeId: number): void {  
  this._filmsSW = this._filmsSW.filter(  
    film => film.episodeId !== episodeId;  
  );  
  this.cdr.markForCheck(); //Necesitas avisar manualmente  
}
```

Con Signals sería así:

```
filmsSW = signal<IFilmSW[]>([]);  
  
removeFilm(episodeId: number): void {  
  this.filmsSW.update(  
    films =>  
    films.filter(film => film.episodeId !== episodeId)  
  ); //Angular detecta el cambio automáticamente.  
}
```

fetch() vs HttpClient en Angular

En Angular existen dos formas principales de hacer peticiones HTTP:

FORMA 1. fetch() (JavaScript estándar)

- Es la API nativa del navegador para hacer peticiones HTTP.
- Devuelve Promesas y no depende de Angular

FORMA 2. HttpClient (forma oficial de Angular)

- Es el servicio HTTP propio de Angular.
- Devuelve Observables (RxJS) y se integra con el sistema de Angular
- Permite:
 - Interceptores (tokens, auth, logs)
 - Cancelar peticiones
 - Testing más sencillo
 - Manejo centralizado de errores

```
this.http.get(url).subscribe(data => {  
  this.datos = data;  
});
```

LOCALSTORAGE

Ahora vamos ampliar el ejercicio anterior y vamos a hacer unas pruebas con la API LocalStorage.

En primer lugar vamos a crear una interfaz que representa un objeto literal que almacena información sobre un id de película y una puntuación. Esto nos ayuda a tener autocompletado y detección de errores en TypeScript:

```
ng generate interface interfaces/i-film-rating.interface
```

```
export interface IFilmRating {  
  episodeId: number;  
  rating: number;  
}
```

Ahora vamos a crear el servicio que se encarga de las operaciones con LocalStorage:

```
ng generate service services/localstorage.service --skip-tests
```

```
import { Injectable } from '@angular/core';
import { IFilmRating } from '../interfaces/i-film-rating.interface';

@Injectable({
  providedIn: 'root',
})
export class LocalStorageService {
  private static readonly KEY_MY_FILMS = 'my_films';

  public saveFilmsRating(films: IFilmRating[]): void {
    try {
      const jsonData = JSON.stringify(films);
      localStorage.setItem(LocalStorageService.KEY_MY_FILMS, jsonData);
    } catch (error) {
      console.error('Error saving films to localStorage:', error);
    }
  }

  public getFilmsRating(): IFilmRating[] {
    try {
      const jsonData =
        localStorage.getItem(LocalStorageService.KEY_MY_FILMS);
      return jsonData ? JSON.parse(jsonData) : [];
    } catch (error) {
      console.error('Error getting films from localStorage:', error);
      return [];
    }
  }

  public clearFilmsRating(): void {
    localStorage.removeItem(LocalStorageService.KEY_MY_FILMS);
  }
}
```

Ahora vamos a usar esto en nuestro componente **fims-sw.ts** y en **fims-sw.html**:

```
import { Component, OnInit, inject, ChangeDetectorRef } from '@angular/core';
import { ApiswService } from '../services/apisw.service';
import { IFilmSW } from '../interfaces/i-film-sw.interface';

import { IFilmRating } from '../interfaces/i-film-rating.interface';
import { LocalStorageService } from '../services/localstorage.service';

@Component({
```



```
selector: 'app-films-sw',
standalone: false,
templateUrl: './films-sw.html',
styleUrl: './films-sw.css',
})
export class FilmsSW implements OnInit {
  public titleComp = 'FILM';

  private apiswService = inject(ApiswService);
  private cdr = inject(ChangeDetectorRef);

  private _filmsSW: IFilmSW[] = [];
  public loading = true;
  public error = '';

  private localStorageService = inject(LocalStorageService);

  // Control de checkboxes y ratings
  public filmRatings = new Map<number, number>();

  get filmsSW(): IFilmSW[] {
    return this._filmsSW;
  }

  set filmsSW(films: IFilmSW[]) {
    this._filmsSW = films;
  }

  removeFilmByEpisodeId(episodeId: number): void {
    this._filmsSW = this._filmsSW.filter((film) => film.episodeId !== episodeId);
    this.filmRatings.delete(episodeId);
    this.cdr.markForCheck();
  }

  async loadFilms(): Promise<void> {
    this.loading = true;
    this.error = '';
    this.cdr.markForCheck(); //notifica a Angular un cambio.

    try {
      this._filmsSW = await this.apiswService.getFilms();
      this._filmsSW.sort((a, b) => a.episodeId - b.episodeId);
      this.loadSavedRatings();
    } catch {
      this.error = 'Error loading films';
    } finally {
      this.loading = false;
      this.cdr.markForCheck(); //notifica a Angular un cambio.
    }
  }

  async ngOnInit() {
    await this.loadFilms();
  }

  private loadSavedRatings(): void {
    const savedFilms = this.localStorageService.getFilmsRating();
```

```
savedFilms.forEach((saved) => {
  this.filmRatings.set(saved.episodeId, saved.rating);
});

this.cdr.markForCheck();
}

onCheckboxChange(episodeId: number, check: boolean): void {
  if (check) {
    this.filmRatings.set(episodeId, 0);
  } else {
    this.filmRatings.delete(episodeId);
  }
  this.cdr.markForCheck();
}

onRatingChange(episodeId: number, inputRat: string): void {
  const rating = Number(inputRat);
  this.filmRatings.set(episodeId, rating);
  this.cdr.markForCheck();
}

isSelected(episodeId: number): boolean {
  return this.filmRatings.has(episodeId);
}

getRating(episodeId: number): number {
  return this.filmRatings.get(episodeId) || 0;
}

getSelectedFilmsWithRatings(): IFilmRating[] {
  const selected: IFilmRating[] = [];

  this.filmRatings.forEach((rating, episodeId) => {
    selected.push({ episodeId, rating });
  });

  return selected;
}

saveSelectedFilms(): void {
  const selectedFilms = this.getSelectedFilmsWithRatings();
  this.localStorageService.saveFilmsRating(selectedFilms);
  alert(`Guardadas ${selectedFilms.length} películas en localStorage`);
}
}
```

```
<h2>{{titleComp}}</h2>
<hr />
@if (loading) {
<p>Cargando películas de Star Wars...</p>
}

@if (!error) {
<p class="error">{{ error }}</p>
}
```

```
@if (!loading && !error) {
<table>
  <thead>
    <tr>
      <th>Episodio</th>
      <th>Título</th>
      <th>Director</th>
      <th>Año</th>
      <th>Vista</th>
      <th>Calificación</th>
      <th>Acciones</th>
    </tr>
  </thead>
  <tbody>
    @for (film of filmsSW; track film.episodeId;) {
      <tr>
        <td>{{ film.episodeId }}</td>
        <td>{{ film.title }}</td>
        <td>{{ film.director }}</td>
        <td>{{ film.year }}</td>
        <td>
          <input type="checkbox" #checkbox [checked]="isSelected(film.episodeId) "
            (change)="onCheckboxChange(film.episodeId, checkbox.checked)">
        </td>
        <td>
          @if (isSelected(film.episodeId)) {
            <input type="number" min="0" max="10" #rating
              [value]="getRating(film.episodeId) "
              (input)="onRatingChange(film.episodeId, rating.value)">
          }
        </td>
        <td>
          <button type="button"
            (click)="removeFilmByEpisodeId(film.episodeId)">Borrar</button>
        </td>
      </tr>
    }
  </tbody>
</table>

<p><strong>Total de películas:</strong> {{ filmsSW.length }}</p>
<button type="button" (click)="saveSelectedFilms()">
  Guardar películas seleccionadas
</button>
}
```

En Angular, #checkbox o #rating no es un atributo HTML ni un id, sino una variable de plantilla. Sirve para tener una referencia directa al elemento dentro del propio template y poder acceder a sus propiedades, como el valor que escribe el usuario

Es decir, esto nos permite acceder al valor del input desde el template.

SIGNALS y HTTPCLIENT EN ANGULAR

Ahora vamos a practicar las soluciones modernas propuestas por Angular y que hemos ido comentando previamente.

Vamos a generar una interfaz para representar la información que nos interesa de las naves especiales de la API de Star Wars: <https://swapi.dev/documentation#starships>

```
ng generate interface interfaces/i-starship-sw
```

```
export interface IStarshipsSW {  
  name: string;  
  model: string;  
  manufacturer: string;  
}
```

Ahora vamos a activar **HttpClient** en app-module.ts

```
import { HttpClient } from '@angular/common/http';  
...  
providers: [  
  provideBrowserGlobalErrorListeners(),  
  { provide: LOCALE_ID, useValue: 'es-ES' },  
  provideHttpClient(),  
],  
...
```

Con Standalone sería en app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners }  
from '@angular/core';  
import { provideRouter } from '@angular/router';  
import { routes } from './app.routes';  
  
import { provideHttpClient } from '@angular/common/http';  
  
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideBrowserGlobalErrorListeners(),  
    provideRouter(routes),  
    provideHttpClient(),  
  ]  
}
```

```
};
```

HttpClient funciona con Observables. Un observable es un flujo de datos asíncrono que puede emitir múltiples valores a lo largo del tiempo. Es como una promesa mejorada que puede emitir más de un valor y además permite una suscripción a la que te puedes conectar y desconectar.

Actualiza el servicio ApiswService:

```
import { Injectable, inject } from '@angular/core';
...
import { HttpClient } from '@angular/common/http';
import { Observable, map, catchError, of } from 'rxjs';
import { IStarshipsSW } from '../interfaces/i-starship-sw';
...

export class ApiswService {
  private readonly API_STARSHIPS = 'https://swapi.dev/api/starships/';

  private http = inject(HttpClient);

  ...
  private convertJsonToInterfaceStarship(starship: any): IStarshipsSW {
    return {
      name: starship.name,
      model: starship.model,
      manufacturer: starship.manufacturer,
    };
  }
  //Devolvemos un observable que puede contener un Array de Naves
  public getStarships(): Observable<IStarshipsSW[]> {
    return this.http.get<any>(this.API_STARSHIPS).pipe(
      map((response) =>
        (response.results ?? []).map((ship: any) =>
          this.convertJsonToInterfaceStarship(ship)
        )
      ),
      catchError((err) => {
        console.error('Error fetching starships:', err);
        return of([] as IStarshipsSW[]);
        //of() crea un Observable
      })
    );
  }
  //pipe nos permite encadenar operadores
}
...
```

Recuerda cómo funciona la API y que en el json que recibimos las naves

```
están en "result".
```

Vamos a generar un componente que haga uso de este método del servicio. Además vamos a hacer uso de señales (**SIGNAL**) para notificar los cambios, de este modo Angular sabe exactamente qué cambió. Por tanto, la detección de cambios no la haremos con ChangeDetectorRef:

```
ng generate component starships-sw --skip-tests
```

```
starships-sw.ts
```

```
import { Component, inject, OnInit, signal } from '@angular/core';

import { ApiswService } from '../services/apisw.service';
import { IStarshipsSW } from '../interfaces/i-starship-sw';

@Component({
  selector: 'app-starships-sw',
  standalone: false,
  templateUrl: './starships-sw.html',
  styleUrls: ['./starships-sw.css'],
})
export class StarshipsSw implements OnInit {
  public titleComp = 'Starships';
  //public starships: IStarshipsSW[] = [];
  //public loading = true;
  //public error = '';
  public starships = signal<IStarshipsSW[]>([]);
  public loading = signal(true);
  public error = signal('');

  private apiswService = inject(ApiswService);

  ngOnInit(): void {
    this.loading.set(true);
    this.error.set('');
    //Para ejecutar un Observable y recibir sus datos,
    //debemos suscribirnos con .subscribe()
    this.apiswService.getStarships().subscribe({
      next: (data) => {
        this.starships.set(data);
        this.loading.set(false);
      },
      error: (err) => {
        console.error(err);
        this.error.set('No se pudieron cargar las naves.');
```

```
    },  
    });  
  }  
}
```

starships-sw.html

```
<h2>{{titleComp}}</h2>  
<hr />  
  
@if (loading()) {  
<p>Cargando naves...</p>  
} @else if (error()) {  
  <p>{{ error() }}</p>  
} @else {  
  
  @for (ship of starships(); track ship.name) {  
    <h3>{{ship.name }}</h3>  
    <ul>  
      <li>{{ ship.model }} </li>  
      <li>{{ ship.manufacturer }}</li>  
    </ul>  
  }  
<p>Total: {{ starships().length }}</p>  
}
```

Creamos una opción en el menú para usar el componente como ya hemos venido haciendo:

```
<nav>  
  <a routerLink="/">Home</a>  
  |  
  <a routerLink="/user">User</a>  
  |  
  <a routerLink="/forms">Forms</a>  
  |  
  <a routerLink="/cars">Cars</a>  
  |  
  <a routerLink="/films">Films</a>  
  |  
  <a routerLink="/starships">Starships</a>  
</nav>
```

```
const routes: Routes = [  
  { path: '', title: 'App Home', component: Home },  
  { path: 'user', title: 'App User', component: User },  
  { path: 'forms', title: 'App Forms', component: Forms },  
  { path: 'cars', title: 'App Cars', component: Cars },  
  { path: 'films', title: 'App Films', component: FilmsSW },  
  { path: 'starships', title: 'App Starships', component: StarshipsSw },  
];
```

CICLO DE VIDA DE COMPONENTES EN ANGULAR (LIFECYCLE HOOKS)

Angular llama automáticamente a ciertos métodos en momentos clave de la vida de un componente:

- **cuando se crea**
- **cuando cambian sus datos de entrada**
- **cuando el DOM está listo**
- **cuando se destruye**

Estos **métodos** se llaman Lifecycle Hooks y permiten:

- Inicializar datos
- Reaccionar a cambios
- Acceder al DOM de forma segura
- Liberar recursos (evitar fugas de memoria)

Ya hemos usados algunos a lo largo el curso, ahora los vamos a explicar con más detalle:

a. **ngOnInit** — Inicialización

Se ejecuta una vez, cuando el componente ya tiene sus inputs.

```
ngOnInit() {  
  this.loadData();  
}
```

Regla: No usar el constructor para lógica de inicialización. Usar ngOnInit.

b. **ngOnChanges** — Cambios en @Input

Se ejecuta cuando cambia un @Input desde el componente padre.

```
ngOnChanges(changes: SimpleChanges) {  
  if (changes['text']) {  
    console.log('Antes:', changes['text'].previousValue);  
    console.log('Ahora:', changes['text'].currentValue);  
  }  
}
```

Nota importante: Para que ngOnChanges se dispare correctamente, es mejor usar actualizaciones inmutables (reemplazar objetos/arrays, no mutarlos).

c. `ngAfterViewInit` — DOM listo

Se ejecuta cuando el template ya está renderizado.

```
@ViewChild('box') box!: ElementRef<HTMLInputElement>;

ngAfterViewInit() {
  this.box.nativeElement.focus();
}

...

<input #box>
```

Regla: Nunca acceder al DOM en el constructor ni en `ngOnInit`.

d. `ngOnDestroy` — Limpieza

Se ejecuta cuando el componente se destruye (por ejemplo con `*ngIf` o navegación).

Cuando un componente se muestra/oculta:

- Se crea → `ngOnInit`
- Se destruye → `ngOnDestroy`

```
ngOnDestroy() {
  clearInterval(this.intervalId);
  this.sub?.unsubscribe();
}
```

Regla clave: Todo lo que se crea, debe limpiarse en `ngOnDestroy`.

Buenas prácticas modernas

- Usar `ngOnInit` para llamadas HTTP

```
ngOnInit() {
  this.service.getData();
}
```

- Usar `ngAfterViewInit` para DOM

```
@ViewChild('input') input!: ElementRef;

ngAfterViewInit() {
  this.input.nativeElement.focus();
}
```



```
}
```

- Limpiar siempre en ngOnDestroy

```
ngOnDestroy() {  
  this.intervalId && clearInterval(this.intervalId);  
}
```



3.2. Build your first Angular app

'Build your first Angular app' es un tutorial oficial de Angular. Su objetivo es aprender Angular haciendo, no solo leyendo. El proyecto final es una aplicación que lista viviendas que permite ver el detalle de cada una.

<https://angular.dev/tutorials/first-app>

Además, haremos alguna ampliación propia para probar varios conceptos.

El proyecto, de momento, se crea dentro del repositorio
`fw_ejercicios_apellido1_nombre1_25-26` y dentro de la carpeta:
- `fw_t02p02_apellido1_nombre1`

CREAR EL PROYECTO

Crear un nuevo proyecto "Standalone Components" desde cero ejecutando:

```
ng new mi_app_03
```

Marca esta opciones en el asistente:

- ✓ Which stylesheet system would you like to use? `CSS` [<https://developer.mozilla.org/docs/Web/CSS>]
- ✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? `No`
- ✓ Which AI tools do you want to configure with Angular best practices? <https://angular.dev/ai/develop-with-ai> `None`

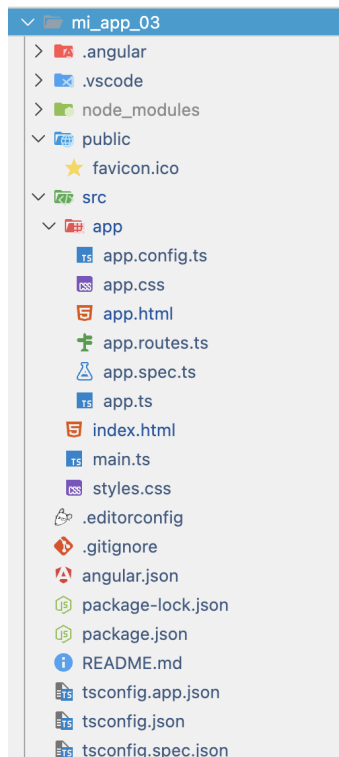
Entra en la carpeta creada y 'arranca' la aplicación:

```
ng serve
```

Accede a la aplicación:

```
http://localhost:4200
```

Analiza la estructura de carpetas:



Recuerda: Este proyecto es STANDALONE COMPONENTS y por tanto no vamos a usar módulos.

CÓDIGO AUTOGENERADO

Abre el fichero **src/index.html** y analiza su contenido.

Abre el fichero **src/styles.css** y añade este contenido.

```
:root {
  --primary-color: #605DC8;
  --secondary-color: #8B89E6;
  --accent-color: #e8e7fa;
  --shadow-color: #E8E8E8;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  padding: 20px;
}

button {
  padding: 10px 20px;
```

```
    cursor: pointer;
    background: var(--primary-color);
    color: white;
    border: none;
    border-radius: 4px;
  }

  button:hover {
    background: var(--secondary-color);
  }

  input {
    padding: 8px;
    border: 1px solid var(--shadow-color);
    border-radius: 4px;
  }
}
```

Abre el fichero **src/app/app.html** y reemplaza todo por:

```
<main>
  <h1>Hola Apellido1!</h1>
</main>
<router-outlet></router-outlet><!-- De momento no hace nada -->
```

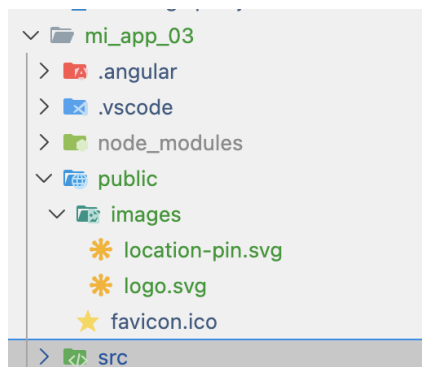
Recuerda: El navegador se actualiza solo mientras está el servidor corriendo.

Analiza el fichero **src/app/app.ts**:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('mi_app_03');
}
```

Para terminar nos vamos a descargar unas imágenes y las vamos a guardar dentro de la carpeta **public/images** para más adelante: 📁 **mi_app_03_recursos**



CREAR EL COMPONENTE HOME

Ahora vamos a crear un nuevo componente standalone, entender cómo se conecta un componente con otro y mostrar el componente Home dentro del componente raíz App.

Recuerda: Un componente en Angular es:

- Una clase TypeScript (lógica)
- Una plantilla HTML (vista)
- Un CSS (estilos)

Angular construye las aplicaciones uniando componentes, no páginas.

Lo primero desde la carpeta del proyecto:

```
ng generate component home --skip-tests
```

Angular crea automáticamente:

```
CREATE src/app/home/home.css (0 bytes)
CREATE src/app/home/home.spec.ts (517 bytes)
CREATE src/app/home/home.ts (178 bytes)
CREATE src/app/home/home.html (19 bytes)
```

Recuerda: En Angular actual los componentes ya son standalone por defecto. No usamos NgModules.

Revisa el fichero **home/home.ts**, **home/home.html** y **home/home.css**

Como sabes con esto ya existe el componente, pero NO se muestra todavía. Ahora vamos a usar Home dentro del componente App

En **app.ts**

```
import { Component, signal } from '@angular/core';
```

```
import { RouterOutlet } from '@angular/router';
import { Home } from './home/home';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Home],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('mi_app_03');
}
```

En app.html

```
<main>
  <header class="brand-name">
    
  </header>
  <section class="content">
    <app-home></app-home>
  </section>
</main>
<router-outlet></router-outlet><!-- De momento no hace nada -->
```

Arranca el servidor y comprueba el resultado

Ahora vamos a añadir estructura al componente Home y sus estilos

En home.html

```
<section>
  <form>
    <input type="text" placeholder="Filter by city">
    <button class="primary" type="button">Search</button>
  </form>
</section>
```

En home.css

```
.results {
  display: grid;
  column-gap: 14px;
  row-gap: 14px;
  grid-template-columns: repeat(auto-fill, minmax(400px, 400px));
  margin-top: 50px;
}
```

```
justify-content: space-around;
}
input[type="text"] {
  border: solid 1px var(--primary-color);
  padding: 10px;
  border-radius: 8px;
  margin-right: 4px;
  display: inline-block;
  width: 30%;
}
button {
  padding: 10px;
  border: solid 1px var(--primary-color);
  background: var(--primary-color);
  color: white;
  border-radius: 8px;
}
@media (min-width: 500px) and (max-width: 768px) {
  .results {
    grid-template-columns: repeat(2, 1fr);
  }
  input[type="text"] {
    width: 70%;
  }
}
@media (max-width: 499px) {
  .results {
    grid-template-columns: 1fr;
  }
}
```

Recuerda: Estos estilos solo afectan a Home, no al resto de la app.

CREAR EL COMPONENTE HOUSINGLOCATION

Ahora la aplicación tendrá un nuevo componente llamado HousingLocation. El componente se mostrará dentro del componente Home.

Crear el componente HousingLocation

```
ng generate component housing-location --skip-tests
```

Observa los ficheros creados. De momento no vamos a editar ninguno.

Ahora vamos a añadir el HousingLocation al componente Home

En `src/app/home/home.ts`

```
import { Component } from '@angular/core';
import { HousingLocation } from '../housing-location/housing-location';

@Component({
  selector: 'app-home',
  imports: [HousingLocation],
  templateUrl: './home.html',
  styleUrls: ['./home.css'],
})
export class Home {}
```

En `src/app/home/home.html`

```
<section>
  <form>
    <input type="text" placeholder="Filter by city" />
    <button class="primary" type="button">Search</button>
  </form>
</section>

<section class="results">
  <app-housing-location></app-housing-location>
</section>
```

CREA UNA 'INTERFACE' DE TYPESCRIPT

⚠ Recuerda: En Angular (TypeScript) una 'interface' se usa para definir la forma de los datos: qué propiedades tiene un objeto y de qué tipo son.

En Angular, las interfaces (TypeScript) son **tipos de datos personalizados** que se utilizan para describir la forma que deben tener los datos dentro de una aplicación. Recuerda que Angular utiliza TypeScript, lo que permite trabajar en un entorno fuertemente tipado.

Gracias a esto, se reduce la probabilidad de que una parte de la aplicación envíe datos con un formato incorrecto a otra. Los errores de tipo se detectan en tiempo de compilación y, en muchos casos, el propio IDE los señala antes incluso de ejecutar la aplicación.

En este capítulo se crea una interfaz para definir las propiedades que representan la información de una vivienda (housing location). Esta interfaz servirá como contrato para asegurar que todos los datos relacionados con una vivienda tengan siempre la estructura correcta.

En la raíz del proyecto Angular:

```
ng generate interface housinglocation
```

Esto crea el fichero `src/app/housinglocation.ts`

En dicho fichero añade:

```
export interface HousingLocationInfo {  
  id: number;  
  name: string;  
  city: string;  
  state: string;  
  photo: string;  
  availableUnits: number;  
  wifi: boolean;  
  laundry: boolean;  
}
```

Esto no afecta a rutas, CSS ni HTML.

Ahora vamos a usarla. En `src/app/home/home.ts`

```
import { Component } from '@angular/core';  
import { HousingLocation } from '../housing-location/housing-location';  
import { HousingLocationInfo } from '../housinglocation';  
  
@Component({  
  selector: 'app-home',  
  imports: [HousingLocation],  
  templateUrl: './home.html',  
  styleUrls: ['./home.css'],  
})  
export class Home {  
  readonly baseUrl = 'https://angular.dev/assets/images/tutorials/common';  
  
  housingLocation: HousingLocationInfo = {  
    id: 9999,  
    name: 'Test Home',  
    city: 'Test city',  
    state: 'ST',  
    photo: `${this.baseUrl}/example-house.jpg`,  
  }  
}
```

```
    availableUnits: 99,  
    wifi: true,  
    laundry: false,  
  };  
}
```

Nota: En este ejemplo, las imágenes vienen de una URL externa.

De momento este objeto aún no se muestra en pantalla. Solo sirve para comprobar tipado (TypeScript). Así que la app sigue mostrando: housing-location works!

INPUTS EN ANGULAR

⚠ En Angular, un Input es un mecanismo de comunicación entre componentes. Sirve para que un componente padre envíe datos a un componente hijo. No tiene relación directa con el elemento HTML `<input>`.

Los Inputs de Angular permiten que un componente hijo reciba datos desde un componente padre. En Angular, un componente no debería depender de datos externos “por sorpresa”. Los Inputs definen explícitamente qué datos necesita un componente para funcionar. Esto mejora la reutilización, la claridad del código y el control de errores.

Los pasos a seguir son los siguientes:

Paso 1. Importar la función `input()`

En `src/app/housing-location/housing-location.ts`:

```
import { Component, input } from '@angular/core';
```

Paso 2. Añadir la propiedad Input requerida: En el componente `HousingLocation`, añade una propiedad llamada `housingLocation` usando `input.required()` y el tipo `HousingLocationInfo`. Esto indica que el componente exige recibir ese dato y TypeScript lo tratará como no nulo.

En `src/app/housing-location/housing-location.ts`:

```
import { Component, input } from '@angular/core';  
import { HousingLocationInfo } from '../housinglocation';  
  
@Component({  
  selector: 'app-housing-location',  
  housingLocation: input.required() as HousingLocationInfo,  
})
```

```
imports: [],
templateUrl: './housing-location.html',
styleUrl: './housing-location.css',
}))
export class HousingLocation {
  housingLocation = input.required<HousingLocationInfo>();
  //No es una variable normal. Es una función.
}
```

Paso 3. Pasar datos desde el componente Home: Ahora enviamos datos desde el componente padre (Home) al hijo (HousingLocation) usando **property binding**.

En src/app/home/home.html observa que nos advierte de un error:

```
8 <section class="results">
9   <app-housing-location></app-housing-location> Required input 'housingLocation'
10 </section>
```

```
<section class="results">
  <app-housing-location [housingLocation]="housingLocation" />
</section>
```

- Lo que va entre [] es la propiedad del componente hijo
- Lo que va entre comillas es una variable del componente padre
- Angular mantiene el enlace de forma dinámica
- Si quitásemos los corchetes, Angular enviaría un string, no el objeto.

Paso 4. Añadimos interpolación: Ahora usamos los datos recibidos para renderizar información real usando interpolación.

En src/app/housing-location/housing-location.html se combinan binding + interpolación en el mismo componente:

```
<section class="listing">
  <img class="listing-photo" [src]="housingLocation().photo"
    alt="Exterior photo of {{ housingLocation().name }}"
    crossorigin />
  <h2 class="listing-heading">{{ housingLocation().name }}</h2>
  <p class="listing-location">
    {{ housingLocation().city }}, {{ housingLocation().state }}
  </p>
```

```
</section>
```

- `housingLocation()` → porque viene de `input()`
- `[src]` → property binding
- `{{ }}` → interpolación

Finalmente en `src/app/housing-location/housing-location.css` damos estilo

```
.listing {
  background: var(--accent-color);
  border-radius: 30px;
  padding-bottom: 30px;
}

.listing-heading {
  color: var(--primary-color);
  padding: 10px 20px 0 20px;
}

.listing-photo {
  height: 250px;
  width: 100%;
  object-fit: cover;
  border-radius: 30px 30px 0 0;
}

.listing-location {
  padding: 10px 20px 20px 20px;
}

.listing-location::before {
  content: url("/images/location-pin.svg") / "";
}

section.listing a {
  padding-left: 20px;
  text-decoration: none;
  color: var(--primary-color);
}

section.listing a::after {
  content: "\203A";
  margin-left: 5px;
}
```

USO DE @FOR

Ahora vamos a ver como un componente puede trabajar con colecciones de datos para repetir HTML dinámicamente usando @for. Cada iteración puede enviar datos distintos a un componente hijo

@for es un bloque de control de flujo de Angular.

```
@for (item of lista) {  
  <!-- HTML repetido -->  
}
```

Paso 1: Crear el array:

En src/app/home/home.ts vamos a cambiar la variable housingLocation que representa un objeto de tipo HousingLocationInfo por la variable housingLocationList un array de objetos HousingLocationInfo.

```
...  
export class Home {  
  readonly baseUrl = 'https://angular.dev/assets/images/tutorials/common';  
  
  housingLocationList: HousingLocationInfo[] = [  
    {  
      id: 0,  
      name: 'Acme Fresh Start Housing',  
      city: 'Chicago',  
      state: 'IL',  
      photo: `${this.baseUrl}/bernard-hermant-CLKGGwIBTaY-unsplash.jpg`,  
      availableUnits: 4,  
      wifi: true,  
      laundry: true,  
    },  
    {  
      id: 1,  
      name: 'A113 Transitional Housing',  
      city: 'Santa Monica',  
      state: 'CA',  
      photo: `${this.baseUrl}/brandon-griggs-wR11KBaB86U-unsplash.jpg`,  
      availableUnits: 0,  
      wifi: false,  
      laundry: true,  
    },  
    {  
      id: 2,  
      name: 'Warm Beds Housing Support',  
      city: 'Juneau',  
      state: 'AK',  
      photo: `${this.baseUrl}/i-do-nothing-but-love-lAyXd11-Wmc-unsplash.jpg`,  
    },  
  ],  
}
```

```
    availableUnits: 1,
    wifi: false,
    laundry: false,
  },
  {
    id: 3,
    name: 'Homesteady Housing',
    city: 'Chicago',
    state: 'IL',
    photo: `${this.baseUrl}/ian-macdonald-W8z6aiwfilE-unsplash.jpg`,
    availableUnits: 1,
    wifi: true,
    laundry: false,
  },
  {
    id: 4,
    name: 'Happy Homes Group',
    city: 'Gary',
    state: 'IN',
    photo: `${this.baseUrl}/krzysztof-hepner-978RAXoXnH4-unsplash.jpg`,
    availableUnits: 1,
    wifi: true,
    laundry: false,
  },
  {
    id: 5,
    name: 'Hopeful Apartment Group',
    city: 'Oakland',
    state: 'CA',
    photo: `${this.baseUrl}/r-architecture-JvQ0Q5IkeMM-unsplash.jpg`,
    availableUnits: 2,
    wifi: true,
    laundry: true,
  },
  {
    id: 6,
    name: 'Seriously Safe Towns',
    city: 'Oakland',
    state: 'CA',
    photo: `${this.baseUrl}/phil-hearing-IYfp2Ixe9nM-unsplash.jpg`,
    availableUnits: 5,
    wifi: true,
    laundry: true,
  },
  {
    id: 7,
    name: 'Hopeful Housing Solutions',
    city: 'Oakland',
    state: 'CA',
    photo: `${this.baseUrl}/r-architecture-GGupkreKwxA-unsplash.jpg`,
    availableUnits: 2,
```

```
wifi: true,
  laundry: true,
},
{
  id: 8,
  name: 'Seriously Safe Towns',
  city: 'Oakland',
  state: 'CA',
  photo: `${this.baseUrl}/saru-robert-9rP3mxf8qWI-unsplash.jpg`,
  availableUnits: 10,
  wifi: false,
  laundry: false,
},
{
  id: 9,
  name: 'Capital Safe Towns',
  city: 'Portland',
  state: 'OR',
  photo: `${this.baseUrl}/webaliser-_TPTXZd9mOo-unsplash.jpg`,
  availableUnits: 6,
  wifi: true,
  laundry: true,
},
],
}
```

En src/app/home/home.html vamos a recorrer ese array:

```
<section>
  <form>
    <input type="text" placeholder="Filtrar por ciudad">
    <button class="primary" type="button">Buscar</button>
  </form>
</section>
<section class="results">
  @for (housingLocation of housingLocationList; track $index) {
    <app-housing-location [housingLocation]="housingLocation" />
  }
</section>
```

En cada iteración se crea un <app-housing-location> y se le pasa un objeto distinto.

- **Cuidado:** Aquí housingLocation ya no es una propiedad del componente, es la variable del @for.

SERVICIOS EN ANGULAR

⚠ Un servicio en Angular es una clase que contiene lógica o datos compartidos y que puede ser usada por varios componentes. No pinta HTML ni tiene plantilla: su función es proporcionar datos o funcionalidades

Un servicio es una clase que permite separar los datos, la lógica de negocio y las funciones reutilizables de los componentes visuales.

Los servicios pueden ser utilizados por varios componentes, evitando duplicar código y facilitando el mantenimiento.

Para que un servicio pueda usarse en otros componentes, debe ser **inyectable**.

La **inyección de dependencias** es el mecanismo por el cual Angular crea instancias de los servicios y las proporciona automáticamente a los componentes que las necesitan. Un componente depende de un servicio cuando no puede funcionar correctamente sin él.

Lo primero que vamos a hacer es crear el servicio:

```
ng generate service housing --skip-tests
```

Esto nos crea el fichero `housing.service.ts` (o `housing.ts`):

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root',
})
export class HousingService {

}
```

Ahora vamos a mover los datos `housingLocationList` del `home.ts` al servicio. Para ello, en el fichero `housing.service.ts` (o `housing.ts`):

```
import { Injectable } from '@angular/core';
import { HousingLocationInfo } from './housinglocation';

@Injectable({
  providedIn: 'root',
})
export class HousingService {
  readonly baseUrl = 'https://angular.dev/assets/images/tutorials/common';
  protected housingLocationList: HousingLocationInfo[] = [
```

```
{
  id: 0,
  name: 'Acme Fresh Start Housing',
  city: 'Chicago',
  state: 'IL',
  photo: `${this.baseUrl}/bernard-hermant-CLKGGwIBTaY-unsplash.jpg`,
  availableUnits: 4,
  wifi: true,
  laundry: true,
},
{
  id: 1,
  name: 'All3 Transitional Housing',
  city: 'Santa Monica',
  state: 'CA',
  photo: `${this.baseUrl}/brandon-griggs-wR1lKBaB86U-unsplash.jpg`,
  availableUnits: 0,
  wifi: false,
  laundry: true,
},
{
  id: 2,
  name: 'Warm Beds Housing Support',
  city: 'Juneau',
  state: 'AK',
  photo: `${this.baseUrl}/i-do-nothing-but-love-lAyXdl1-Wmc-unsplash.jpg`,
  availableUnits: 1,
  wifi: false,
  laundry: false,
},
{
  id: 3,
  name: 'Homesteady Housing',
  city: 'Chicago',
  state: 'IL',
  photo: `${this.baseUrl}/ian-macdonald-W8z6aiwfilE-unsplash.jpg`,
  availableUnits: 1,
  wifi: true,
  laundry: false,
},
{
  id: 4,
  name: 'Happy Homes Group',
  city: 'Gary',
  state: 'IN',
  photo: `${this.baseUrl}/krzysztof-hepner-978RAXoXnH4-unsplash.jpg`,
  availableUnits: 1,
  wifi: true,
  laundry: false,
},
{
  id: 5,
  name: 'Hopeful Apartment Group',
  city: 'Oakland',
  state: 'CA',
  photo: `${this.baseUrl}/r-architecture-JvQ0Q5IkeMM-unsplash.jpg`,
  availableUnits: 2,
  wifi: true,
}
```

```
    laundry: true,
  },
  {
    id: 6,
    name: 'Seriously Safe Towns',
    city: 'Oakland',
    state: 'CA',
    photo: `${this.baseUrl}/phil-hearing-IYfp2Ixe9nM-unsplash.jpg`,
    availableUnits: 5,
    wifi: true,
    laundry: true,
  },
  {
    id: 7,
    name: 'Hopeful Housing Solutions',
    city: 'Oakland',
    state: 'CA',
    photo: `${this.baseUrl}/r-architecture-GGupkreKwxA-unsplash.jpg`,
    availableUnits: 2,
    wifi: true,
    laundry: true,
  },
  {
    id: 8,
    name: 'Seriously Safe Towns',
    city: 'Oakland',
    state: 'CA',
    photo: `${this.baseUrl}/saru-robert-9rP3mxf8qWI-unsplash.jpg`,
    availableUnits: 10,
    wifi: false,
    laundry: false,
  },
  {
    id: 9,
    name: 'Capital Safe Towns',
    city: 'Portland',
    state: 'OR',
    photo: `${this.baseUrl}/webaliser-_TPTXZd9mOo-unsplash.jpg`,
    availableUnits: 6,
    wifi: true,
    laundry: true,
  },
],
];
```

También vamos a añadir dos métodos:

```
import { Injectable } from '@angular/core';
import { HousingLocationInfo } from '../housinglocation';

@Injectable({
  providedIn: 'root',
})
export class HousingService {
  readonly baseUrl = 'https://angular.dev/assets/images/tutorials/common';
```

```
protected housingLocationList: HousingLocationInfo[] = [ ... ];
getAllHousingLocations(): HousingLocationInfo[] {
  return this.housingLocationList;
}

getHousingLocationById(id: number): HousingLocationInfo | undefined {
  return this.housingLocationList.find(
    (housingLocation) => housingLocation.id === id
  );
}
}
```

Hecho esto, ya podemos inyectar el servicio en el componente Home para usarlo.

Abre `src/app/home/home.ts`.

```
import { Component, inject } from '@angular/core';
import { HousingLocation } from '../housing-location/housing-location';
import { HousingLocationInfo } from '../housinglocation';
import { HousingService } from '../housing.service';

@Component({
  selector: 'app-home',
  imports: [HousingLocation],
  templateUrl: './home.html',
  styleUrls: ['./home.css'],
})
export class Home {
  //readonly baseUrl = 'https://angular.dev/assets/images/tutorials/common';
  housingLocationList: HousingLocationInfo[] = [];
  housingService: HousingService = inject(HousingService);

  constructor() {
    this.housingLocationList = this.housingService.getAllHousingLocations();
  }
}
```

Ahora Home no crea los datos. Home los solicita al servicio

ROUTING EN ANGULAR

El routing permite cambiar de vista sin recargar la página. En una SPA (Single Page Application) solo cambia el componente que se muestra, no el documento HTML completo.

Angular usa el Router para asociar URLs a componentes, cargar el componente correcto según la ruta y gestionar parámetros.

Para probarlo vamos a generar un componente DETAILS que mostrará la información detallada de una vivienda.

```
ng generate component details --skip-tests
```

```
CREATE src/app/details/details.css (0 bytes)
CREATE src/app/details/details.spec.ts (538 bytes)
CREATE src/app/details/details.ts (190 bytes)
CREATE src/app/details/details.html (22 bytes)
```

Ahora vamos a definir las rutas de la aplicación. Abre el fichero app.routes.ts:

```
import { Routes } from '@angular/router';

export const routes: Routes = [];
```

Completa el fichero para que:

```
import { Routes } from '@angular/router';
import { Home } from '../home/home';
import { Details } from '../details/details';

export const routes: Routes = [
  {
    path: '',
    component: Home,
    title: 'Home page',
  },
  {
    path: 'details/:id',
    component: Details,
    title: 'Details',
  },
];
```

- path: "" → ruta raíz (/)
- :id es un parámetro dinámico
- title cambia el <title> del navegador automáticamente

El siguiente paso es comprobar que el app.config.ts se creó correctamente con el asistente y que el router está registrado:

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from
 '@angular/core';
import { provideRouter } from '@angular/router';
```

```
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes)
  ]
};
```

Hecho esto vamos a preparar App para usar routing. El asistente ya añadió RouterOutlet por defecto en app.ts, así que solo vamos a añadir el RouterLink en app.ts:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('mi_app_03');
}
```

En app.htm adapta el contenido:

```
<main>
  <a [routerLink]="'/'">
    <header class="brand-name">
      
    </header>
  </a>

  <section class="content">
    <router-outlet></router-outlet> <!-- Ahora sí es útil -->
  </section>
</main>
```

- <router-outlet> es el hueco dinámico y Angular cargará ahí el componente según la URL. Por eso ya no se usa <app-home> directamente

ROUTING CON PARÁMETROS EN ANGULAR

Continuamos preparando el uso del componente details. Este componente recibe **un parámetro en la ruta** porque es necesario para poder mostrar información específica de un elemento (en este caso, una vivienda). Recuerda que en el archivo app.routes.ts, se definió un segmento dinámico utilizando la sintaxis de dos puntos (:): 'details/:id'

Lo primero que tenemos que hacer es pasar ese parámetro. Para generar el enlace en la plantilla del componente que lista las viviendas (housing-location.component.ts), se utiliza la directiva routerLink. RouterLink activa la creación dinámica de enlaces en la aplicación.

En src/app/housing-location/housing-location.ts añade:

```
import { Component, Input } from '@angular/core';
import { HousingLocationInfo } from '../housinglocation';
import { RouterLink } from '@angular/router';

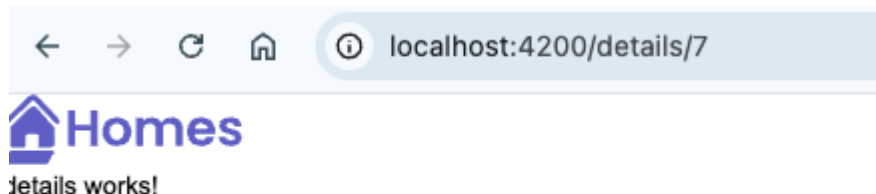
@Component({
  selector: 'app-housing-location',
  imports: [RouterLink],
  templateUrl: './housing-location.html',
  styleUrls: ['./housing-location.css'],
})
export class HousingLocation {
  housingLocation = Input.required<HousingLocationInfo>();
}
```

Y en src/app/housing-location/housing-location.html añade:

```
<section class="listing">
  <img class="listing-photo" [src]="housingLocation().photo" alt="Exterior
photo of {{ housingLocation().name }}"
  crossorigin />
  <h2 class="listing-heading">{{ housingLocation().name }}</h2>
  <p class="listing-location">
    {{ housingLocation().city }}, {{ housingLocation().state }}
  </p>
  <a [routerLink]="['/details', housingLocation().id]">Learn More</a>
</section>
```

- El valor asignado a routerLink es un array donde:
 - El primer elemento es el segmento estático de la ruta (/details).
 - Y el segundo elemento es el dato dinámico (el valor del id).

Guarda los cambios y comprueba qué ocurre:



Lo segundo que tenemos que hacer es obtener ese parámetro en el componente de destino (Details) para ello hay que hacer uso de ActivatedRoute e inject.

En src/app/details/details.ts

```
import { Component, inject } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { HousingService } from '../housing.service';
import { HousingLocationInfo } from '../housinglocation';

@Component({
  selector: 'app-details',
  imports: [],
  templateUrl: './details.html',
  styleUrls: ['./details.css'],
})
export class Details {
  route: ActivatedRoute = inject(ActivatedRoute);
  housingService = inject(HousingService);
  housingLocation: HousingLocationInfo | undefined;
  constructor() {
    const housingLocationId = Number(this.route.snapshot.params['id']);
    this.housingLocation =
      this.housingService.getHousingLocationById(housingLocationId);
  }
}
```

- ActivatedRoute: Servicio que contiene información sobre la ruta activa.
- snapshot: Una instantánea del estado de la ruta en el momento de la creación del componente. Se utiliza cuando no se espera que el parámetro cambie mientras el usuario permanece en el mismo componente.
- params['id']: Acceso al parámetro definido previamente en la configuración de rutas.
- Una vez obtenido el id, el componente solicita los datos completos al HousingService.

Ahora en src/app/details/details.html

```
<article>
  <img class="listing-photo" [src]="housingLocation?.photo"
```



```
        alt="Exterior photo of {{ housingLocation?.name }}"
        crossorigin />
<section class="listing-description">
  <h2 class="listing-heading">{{ housingLocation?.name }}</h2>
  <p class="listing-location">{{ housingLocation?.city }}, {{
    housingLocation?.state }}</p>
</section>
<section class="listing-features">
  <h2 class="section-heading">About this housing location</h2>
  <ul>
    <li>Units available: {{ housingLocation?.availableUnits }}</li>
    <li>Does this location have wifi: {{ housingLocation?.wifi }}</li>
    <li>Does this location have laundry: {{ housingLocation?.laundry }}</li>
  </ul>
</section>
</article>
```

- El Operador de Encadenamiento Opcional (**?.**) es crítico en Angular por dos razones:
 - Asincronía: Si el servicio tardará en devolver los datos, la propiedad `housingLocation` sería `undefined` inicialmente.
 - Seguridad de tipos: Evita errores de ejecución (Cannot read property 'xxxx' of undefined) al evaluar la expresión solo si el objeto existe.

Para concluir vamos a añadir unos estilos en `src/app/details/details.css`

```
.listing-photo {
  height: 600px;
  width: 50%;
  object-fit: cover;
  border-radius: 30px;
  float: right;
}
.listing-heading {
  font-size: 48pt;
  font-weight: bold;
  margin-bottom: 15px;
}
.listing-location::before {
  content: url('/images/location-pin.svg') / '';
}
.listing-location {
  font-size: 24pt;
  margin-bottom: 15px;
}
.listing-features > .section-heading {
  color: var(--secondary-color);
  font-size: 24pt;
  margin-bottom: 15px;
}
.listing-features {
  margin-bottom: 20px;
}
.listing-features li {
```

```
font-size: 14pt;
}
li {
  list-style-type: none;
}
.listing-apply .section-heading {
  font-size: 18pt;
  margin-bottom: 15px;
}
label, input {
  display: block;
}
label {
  color: var(--secondary-color);
  font-weight: bold;
  text-transform: uppercase;
  font-size: 12pt;
}
input {
  font-size: 16pt;
  margin-bottom: 15px;
  padding: 10px;
  width: 400px;
  border-top: none;
  border-right: none;
  border-left: none;
  border-bottom: solid .3px;
}
@media (max-width: 1024px) {
  .listing-photo {
    width: 100%;
    height: 400px;
  }
}
```

FORMS BÁSICOS EN ANGULAR - FORMULARIO BUSCAR

Ahora vamos a hacer que la búsqueda por texto permita filtrar los alojamientos mostrados en pantalla en función de la ciudad introducida por el usuario.

Vamos a necesitar dos listas:

- housingLocationList: lista completa (sin filtrar).
- filteredLocationList: lista filtrada que se muestra en pantalla.
- Al cargar la página, ambas listas deben contener todos los datos.

En src/app/home/home.ts

```
import { Component, inject } from '@angular/core';
import { HousingLocation } from '../housing-location/housing-location';
import { HousingLocationInfo } from '../housinglocation';
import { HousingService } from '../housing.service';

@Component({
  selector: 'app-home',
```

```
imports: [HousingLocation],
templateUrl: './home.html',
styleUrl: './home.css',
})
export class Home {
  housingLocationList: HousingLocationInfo[] = [];
  filteredLocationList: HousingLocationInfo[] = [];
  housingService: HousingService = inject(HousingService);

  constructor() {
    this.housingLocationList = this.housingService.getAllHousingLocations();
    this.filteredLocationList = this.housingLocationList;
  }

  filterResults(text: string) {
    if (!text) {
      this.filteredLocationList = this.housingLocationList;
      return;
    }
    this.filteredLocationList =
      this.housingLocationList.filter((housingLocation) =>
        housingLocation?.city.toLowerCase().includes(text.toLowerCase()),
      );
  }
}
```

- Si el texto está vacío → se muestran todos los resultados.
- Si hay texto → se filtran los alojamientos cuya ciudad contiene ese texto.

En src/app/home/home.html

```
<section>
  <form>
    <input type="text" placeholder="Filter by city" #filter />
    <button class="primary" type="button"
      (click)="filterResults(filter.value)">Search</button>
    </form>
  </section>
```

En Angular, **#filter** no es un atributo HTML ni un id, sino una variable de plantilla. Sirve para tener una referencia directa al elemento dentro del propio template y poder acceder a sus propiedades, como el valor que escribe el usuario

Es decir, esto nos permite acceder al valor del input desde el template.

En Angular no se trabaja directamente con eventos del DOM

(addEventListener, onsubmit, etc.).

Angular usa enlace a eventos (event binding):

```
(click)="filterResults(filter.value) "
```

Esto ya es un manejador de eventos, pero gestionado por Angular e Integrado con el ciclo de detección de cambios.

De nuevo en `src/app/home/home.html`

```
<section class="results">
  @for (housingLocation of filteredLocationList; track $index) {
    <app-housing-location [housingLocation]="housingLocation" />
  }
</section>
```

FORMS REACTIVOS EN ANGULAR - CONTACTO EN ANGULAR

En este apartado vamos a añadir un formulario reactivo en la página de detalles. El formulario recogerá datos del usuario y los enviará a un servicio, que por ahora solo los mostrará en la consola del navegador. No se usa API REST en este paso.

En `src/app/housing.service.ts` Primero ampliamos el servicio para que reciba los datos del formulario. Añade este método a la class `HousingService`:

```
submitApplication(firstName: string, lastName: string, email: string) {
  console.log(
    `Homes application received: firstName: ${firstName}, lastName:
                                     ${lastName}, email: ${email}.`,
  );
}
```

En `src/app/details/details.html` Añade el formulario html:

```
<section class="listing-apply">
  <h2 class="section-heading">Apply now to live here</h2>
  <form [formGroup]="applyForm" (submit)="submitApplication()">
    <label for="first-name">First Name</label>
    <input id="first-name" type="text" formControlName="firstName" />
    <label for="last-name">Last Name</label>
    <input id="last-name" type="text" formControlName="lastName" />
    <label for="email">Email</label>
    <input id="email" type="email" formControlName="email" />
    <button type="submit" class="primary">Apply now</button>
  </form>
</section>
```

- `[formGroup]="applyForm"` enlaza el formulario con Angular

- formControlName conecta cada input con su FormControl
- (submit)="submitApplication()" maneja el envío del formulario

En Angular no se trabaja directamente con eventos del DOM (addEventListener, onsubmit, etc.).

Angular usa enlace a eventos (event binding):

```
<form (submit)="submitApplication()">
```

Esto ya es un manejador de eventos, pero gestionado por Angular e Integrado con el ciclo de detección de cambios.

En src/app/details/details.ts:

- Importar las clases de formularios
- Registrar ReactiveFormsModule en el componente (en imports: [])
- Crear el formulario (FormGroup representa el formulario completo)
- Añadir el método de envío del formulario

```
import { Component, inject } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { HousingService } from '../housing.service';
import { HousingLocationInfo } from '../housinglocation';
import { FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-details',
  imports: [ReactiveFormsModule],
  templateUrl: './details.html',
  styleUrls: ['./details.css'],
})
export class Details {
  route: ActivatedRoute = inject(ActivatedRoute);
  housingService = inject(HousingService);
  housingLocation: HousingLocationInfo | undefined;

  applyForm = new FormGroup({
    firstName: new FormControl(''),
    lastName: new FormControl(''),
    email: new FormControl(''),
  });

  constructor() {
    const housingLocationId = Number(this.route.snapshot.params['id']);
    this.housingLocation =
      this.housingService.getHousingLocationById(housingLocationId);
  }

  submitApplication() {
    this.housingService.submitApplication(
      this.applyForm.value.firstName ?? '',
      this.applyForm.value.lastName ?? '',
    );
  }
}
```

```
this.applyForm.value.email ?? '',  
  );  
}  
}
```

- Usa ?? " para evitar valores null o undefined.
 - No es lo mismo que: valor || "".
 - || también reemplaza valores válidos como 0 o false.

LOCALSTORAGE EN ANGULAR

Como ya sabes, se puede usar esta API de almacenamiento en el navegador sin problemas en Angular.

Lo primero que vamos a hacer es crear una interface:

```
ng generate interface interfaces/interface-house-form
```

```
export interface InterfaceHouseForm {  
  id: string;  
  firstName: string;  
  lastName: string;  
  email: string;  
  housingLocationId: number;  
  consultaDate: Date;  
}
```

Y vamos a crear un servicio para gestionar todo lo relacionado con localStorage:

```
ng generate service services/localstorage.service --skip-tests
```

```
import { Injectable } from '@angular/core';  
import { InterfaceHouseForm } from '../interfaces/interface-house-form';  
  
@Injectable({  
  providedIn: 'root',  
})  
export class LocalStorageService {  
  private readonly APPLICATIONS_KEY = 'application';  
  
  saveApplication(application: InterfaceHouseForm): void {  
    const applications = this.getAllApplications();  
    applications.push(application);  
    localStorage.setItem(this.APPLICATIONS_KEY,  
      JSON.stringify(applications));  
  }  
}
```

```
getAllApplications(): InterfaceHouseForm[] {
  const data = localStorage.getItem(this.APPLICATIONS_KEY);
  if (!data) {
    return [];
  }

  const applications = JSON.parse(data);
  for (let i = 0; i < applications.length; i++) {
    applications[i].consultaDate = new
Date(applications[i].consultaDate);
  }

  return applications;
}
}
```

Y hacemos uso de este servicio en el HousingService

```
import { Injectable, inject } from '@angular/core';
...
import { LocalStorageService } from '../services/localstorage.service';
import { InterfaceHouseForm } from '../interfaces/interface-house-form';
...

export class HousingService {
  ...
  private localStorageService = inject(LocalStorageService);
  ...
  //Esto nos lo podemos llevar ya directamente al servicio LS
  //Aunque se ha dejado aquí para mantener la solución previa.
  submitApplication(
    firstName: string,
    lastName: string,
    email: string,
    housingLocationId: number,
  ): void {
    // Crear el objeto de consulta
    const apply: InterfaceHouseForm = {
      firstName: firstName,
      lastName: lastName,
      email: email,
```

```
        housingLocationId: housingLocationId,
        consultaDate: new Date(),
    };

    // Guardar en localStorage
    this.localStorageService.saveApplication(apply);

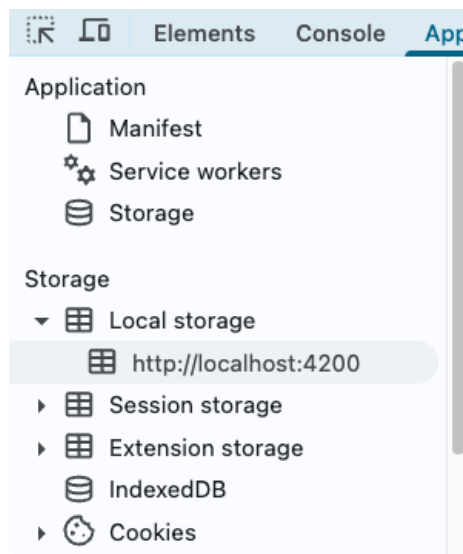
    console.log('Consulta guardada:', apply);
    alert('Appy sent successfully!');
}
...
}
```

Y finalmente actualizamos ligeramente el componente **details**:

```
...
export class Details {
...
    housingLocationId: number = 0;

    constructor() {
        this.housingLocationId = Number(this.route.snapshot.params['id']);
        ...
    }
...
    submitApplication() {
        this.housingService.submitApplication(
            this.applyForm.value.firstName ?? '',
            this.applyForm.value.lastName ?? '',
            this.applyForm.value.email ?? '',
            this.housingLocationId,
        );
    } //Esto nos lo podemos llevar ya directamente al servicio LS
    //Aunque se ha dejado aquí para mantener la solución previa.
...
}
```

Hacemos varias pruebas y consultamos directamente la información almacenada en el navegador



```
▼ [,...]  
  ▶ 0: {firstName: "Gon", lastName: "Freeccs", email: "gfreeccs@educastillalamancha.es", hou  
  ▼ 1: {firstName: "Leorio", lastName: "Paladiknight", email: "lpaladiknight@educastillalama  
    consultaDate: "2026-01-27T21:26:44.669Z"  
    email: "lpaladiknight@educastillalamancha.es"  
    firstName: "Leorio"  
    housingLocationId: 5  
    lastName: "Paladiknight"
```

COMUNICACIÓN HTTP EN ANGULAR

Hasta ahora el servicio HousingService leía datos de un array local. Ahora vamos a hacer una petición HTTP (GET) y recibir JSON, igual que en un proyecto real con API.

En housing.service.ts Elimina el array local y añade los métodos que se van a encargar de realizar las peticiones AJAX:

```
import { Injectable } from '@angular/core';  
import { HousingLocationInfo } from '../housinglocation';  
  
@Injectable({  
  providedIn: 'root',  
})  
export class HousingService {  
  readonly url = 'https://dwec-fw-gp.vercel.app/api/houses';  
  
  submitApplication(firstName: string, lastName: string, email: string) {  
    console.log(  
      `Homes application received: firstName: ${firstName}, lastName:  
        ${lastName}, email: ${email}.`,  
    );  
  }  
}
```

```
async getAllHousingLocations(): Promise<HousingLocationInfo[]> {
  const data = await fetch(this.url);
  return (await data.json()) ?? [];
}

async getHousingLocationById(id: number): Promise<HousingLocationInfo |
                                                                    undefined> {
  const data = await fetch(`${this.url}?id=${id}`);
  const locationJson = await data.json();
  return locationJson[0] ?? {};
}
}
```

Ahora en src/app/home/home.ts:

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { HousingLocation } from '../housing-location/housing-location';
import { HousingLocationInfo } from '../housinglocation';
import { HousingService } from '../housing.service';

@Component({
  selector: 'app-home',
  imports: [HousingLocation],
  templateUrl: './home.html',
  styleUrls: ['./home.css'],
})
export class Home {

  housingLocationList: HousingLocationInfo[] = [];
  filteredLocationList: HousingLocationInfo[] = [];
  housingService: HousingService = inject(HousingService);

  private readonly changeDetectorRef = inject(ChangeDetectorRef);

  constructor() {
    this.housingService
      .getAllHousingLocations()
      .then((housingLocationList: HousingLocationInfo[]) => {
        this.housingLocationList = housingLocationList;
        this.filteredLocationList = housingLocationList;
        this.changeDetectorRef.markForCheck();
        //notifica a Angular un cambio.
      });
  }

  filterResults(text: string) {
    if (!text) {
```

```
        this.filteredLocationList = this.housingLocationList;
        return;
    }
    this.filteredLocationList =
this.housingLocationList.filter((housingLocation) =>
    housingLocation?.city.toLowerCase().includes(text.toLowerCase()),
    );
    }
}
```

Ahora en `src/app/home/details.ts`:

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { HousingService } from '../housing.service';
import { HousingLocationInfo } from '../housinglocation';
import { FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-details',
  imports: [ReactiveFormsModule],
  templateUrl: './details.html',
  styleUrls: ['./details.css'],
})
export class Details {
  route: ActivatedRoute = inject(ActivatedRoute);
  housingService = inject(HousingService);
  housingLocation: HousingLocationInfo | undefined;

  private readonly changeDetectorRef = inject(ChangeDetectorRef);

  applyForm = new FormGroup({
    firstName: new FormControl(''),
    lastName: new FormControl(''),
    email: new FormControl(''),
  });

  constructor() {
    this.housingService.
    getHousingLocationById(housingLocationId)
    .then((housingLocation) => {
      this.housingLocation = housingLocation;
      this.changeDetectorRef.markForCheck();
      //notifica a Angular un cambio.
    });
  }
  submitApplication() {
    this.housingService.submitApplication(
      this.applyForm.value.firstName ?? '',
    );
  }
}
```

```
this.applyForm.value.lastName ?? '',
this.applyForm.value.email ?? '',
);
}
}
```

Recuerda: Esto en Angular moderno también se puede hacer con **SIGNALS y HttpClient**

LOGIN EN ANGULAR (CON LOCALSTORAGE)

Vamos a generar un componente de login. La información de los usuarios la vamos a recuperar del localStorage.

Lo primero que vamos a hacer es crearnos una interface para los usuarios:

ng generate interface interfaces/interface-user

```
export interface InterfaceUser {
  username: string;
  password: string;
}
```

Y vamos a actualizar nuestro servicio de LocalStorage:

```
import { Injectable } from '@angular/core';
import { InterfaceHouseForm } from '../interfaces/interface-house-form';
import { InterfaceUser } from '../interfaces/interface-user';

@Injectable({
  providedIn: 'root',
})
export class LocalStorageService {
  private readonly APPLICATIONS_KEY = 'application';
  private readonly USERS_KEY = 'users';
  private readonly AUTH_KEY = 'currentUser';

  constructor() {
    this.initializeTestUsers();
  }

  saveApplication(application: InterfaceHouseForm): void {
    const applications = this.getAllApplications();
    applications.push(application);
    localStorage.setItem(this.APPLICATIONS_KEY,
```

```
JSON.stringify(applications));
}

getAllApplications(): InterfaceHouseForm[] {
  const data = localStorage.getItem(this.APPLICATIONS_KEY);
  if (!data) {
    return [];
  }

  const applications = JSON.parse(data);
  for (let i = 0; i < applications.length; i++) {
    applications[i].consultaDate = new
Date(applications[i].consultaDate);
  }

  return applications;
}

getApplicationsByMonthAndYear(month: number, year: number):
InterfaceHouseForm[] {
  const allApplications = this.getAllApplications();

  return allApplications.filter((application) => {
    const date = new Date(application.consultaDate);
    return date.getMonth() === month - 1 && date.getFullYear() === year;
  });
}

private initializeTestUsers(): void {
  const currentUsers = this.getAllUsers();

  if (currentUsers.length === 0) {
    const testUsers: InterfaceUser[] = [
      { username: 'gon', password: 'gon' },
      { username: 'killua', password: 'killua' },
    ];
    localStorage.setItem(this.USERS_KEY, JSON.stringify(testUsers));
  }
}

private getAllUsers(): InterfaceUser[] {
  const data = localStorage.getItem(this.USERS_KEY);
  return data ? JSON.parse(data) : [];
}

verifyUser(username: string, password: string): boolean {
  const users = this.getAllUsers();
  const user = users.find((u) =>
    u.username === username && u.password === password);
  if (user) {
    return true;
  }
}
```

```
    } else {
        return false;
    }
}

saveAuthenticatedUser(username: string): void {
    localStorage.setItem(this.AUTH_KEY, username);
}

getAuthenticatedUser(): string | null {
    return localStorage.getItem(this.AUTH_KEY);
}

isAuthenticated(): boolean {
    return this.getAuthenticatedUser() !== null;
}

logout(): void {
    localStorage.removeItem(this.AUTH_KEY);
}
}
```

Generamos un componente para hacer el login

ng generate component login --skip-tests

```
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';
import { LocalStorageService } from '../services/localstorage.service';

@Component({
  selector: 'app-login',
  imports: [],
  templateUrl: './login.html',
  styleUrls: ['./login.css'],
})
export class Login {
  private localStorageService = inject(LocalStorageService);
  private router = inject(Router);

  login(username: string, password: string): void {
    if (this.localStorageService.verifyUser(username, password)) {
      this.localStorageService.saveAuthenticatedUser(username);
      this.router.navigate(['/home']);
    } else {
      alert('Usuario o contraseña incorrectos');
    }
  }
}
```

```
ngOnInit(): void {  
  // Si ya está autenticado, redirigir a home  
  if (this.localStorageService.isAuthenticated()) {  
    this.router.navigate(['/home']);  
  }  
}  
}
```

```
<section class="login-container">  
  <form #loginForm>  
    <input type="text" placeholder="Username" #username />  
    <input type="password" placeholder="Password" #password />  
    <button type="button" (click)="login(username.value, password.value)">  
      Entrar  
    </button>  
  </form>  
  <p class="hint">Usuarios de prueba:</p>  
  <ul class="hint">  
    <li>gon/gon</li>  
    <li>killua/killua</li>  
  </ul>  
</section>
```

```
.login-container {  
  max-width: 400px;  
  margin: 50px auto;  
  padding: 30px;  
}  
  
.login-container form {  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
}  
  
.login-container input {  
  width: 100%;  
}  
  
.login-container button {  
  width: 100%;  
  margin-bottom: 10px;  
}  
  
.login-container .hint {  
  text-align: center;  
  font-size: 12px;  
  color: #666;  
  list-style-type: none;  
}
```

Ahora vamos a actualizar las rutas en `app.routes.ts`. La página de inicio cargará el componente Login. Y ahora le componente Home se cargará en la ruta `.../home`

```
import { Routes } from '@angular/router';
import { Home } from '../home/home';
import { Details } from '../details/details';
import { Login } from '../login/login';

export const routes: Routes = [
  {
    path: '',
    component: Login,
    title: 'Login',
  },
  {
    path: 'home',
    component: Home,
    title: 'Home page',
  },
  {
    path: 'details/:id',
    component: Details,
    title: 'Home details',
  },
];
```

Prueba el comportamiento del sitio web con el código implementado hasta ahora. Para cerrar de sesión, borra manualmente desde el navegador la key `currentUser`

LOGIN EN ANGULAR USANDO GUARD (CON LOCALSTORAGE)

El siguiente paso es proteger las rutas usando un Guard:

```
ng generate guard guards/auth --skip-tests
```

Los Guards son protecciones que controlan el acceso a las rutas. Angular pregunta al Guard "¿puedo pasar?" antes de cargar un componente.

Cuando te pregunte, selecciona: `CanActivate`

Tipos principales de Guards:

- a. `CanActivate`: Antes de entrar a una ruta. Por ejemplo, para verificar login antes

de acceder a nuestro /home

- b. CanActivateChild: Antes de entrar a rutas hijas. Por ejemplo, para proteger todas las subrutas de /admin
- c. CanDeactivate: Antes de salir de una ruta. Por ejemplo, para avisar si hay cambios sin guardar en un formulario
- d. CanMatch: Antes de que la ruta coincida. Se usar para cargar diferentes componentes según el rol del usuario.

```
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { LocalStorageService } from '../services/localstorage.service';

export const authGuard: CanActivateFn = (route, state) => {
  const localStorageService = inject(LocalStorageService);
  const router = inject(Router);

  if (localStorageService.isAuthenticated()) {
    return true; // Allow access
  }

  // Redirect to login
  router.navigate(['/']);
  return false; // Block access
};
```

Ahora actualizamos las rutas aplicando el Guard en app.routers.ts.

```
import { Routes } from '@angular/router';
import { Home } from '../home/home';
import { Details } from '../details/details';
import { Login } from '../login/login';
import { authGuard } from '../guards/auth-guard';

export const routes: Routes = [
  {
    path: '',
    component: Login,
    title: 'Login',
  },
  {
    path: 'home',
```

```
    component: Home,
    title: 'Home page',
    canActivate: [authGuard],
  },
  {
    path: 'details/:id',
    component: Details,
    title: 'Details',
    canActivate: [authGuard],
  },
];
```

JWT EN LA AUTENTICACIÓN:

- El backend devuelve un JWT al hacer login
- Se guarda el token en localStorage (en lugar del username)
- Se envía el token en el header Authorization: Bearer <token> en cada petición
- Se decodifica el token para obtener info del usuario

A. Pruebas en Angular

INTRODUCCIÓN

En este anexo veremos de forma básica el concepto de pruebas automáticas en aplicaciones Angular. En este contexto, las pruebas pueden ser de distintos tipos:

- **Pruebas unitarias:** Verifican el funcionamiento de una parte concreta de la aplicación de forma aislada, como una función, un servicio o un componente. No dependen de servicios externos ni de otros módulos de la aplicación. Son rápidas de ejecutar y permiten detectar errores en fases tempranas del desarrollo.
- **Pruebas de integración:** Comprueban que varias partes de la aplicación funcionan correctamente juntas, por ejemplo, un componente que utiliza un servicio o la comunicación entre varios componentes. Aunque son más completas que las pruebas unitarias, siguen ejecutándose en un entorno controlado.
- **Pruebas de extremo a extremo (E2E):** Simulan el comportamiento real de un usuario en la aplicación, interactuando con la interfaz (clics, formularios, navegación). Validan flujos completos desde el inicio hasta el final, pero son más lentas y costosas de mantener.

En esta parte, **nos centraremos exclusivamente en las pruebas unitarias**, ya que permiten verificar el correcto funcionamiento de componentes, servicios y lógica pura de forma aislada, rápida y sin depender de servicios externos ni del navegador real.

El objetivo no es cubrir todos los casos posibles, sino comprender cómo validar que una parte

concreta de la aplicación se comporta como se espera.

Cambios en el sistema de pruebas en Angular moderno - **Novedad**

NOTE: This guide covers the default testing setup for new Angular CLI projects, which uses Vitest. If you are migrating an existing project from Karma, see the [Migrating from Karma to Vitest guide](#). Karma is still supported; for more information, see the [Karma testing guide](#).

Las versiones recientes de Angular han actualizado el sistema de pruebas por defecto, pasando a utilizar **Vitest** en los proyectos nuevos creados con Angular CLI. Este cambio simplifica la ejecución de tests y reduce la configuración necesaria, especialmente en aplicaciones basadas en componentes standalone.

El sistema anterior, basado en **Karma y Jasmine**, sigue siendo compatible y puede encontrarse en muchos proyectos existentes, existiendo además guías oficiales de migración: <https://angular.dev/guide/testing/migrating-to-vitest>

PRINCIPIOS BÁSICOS DE TESTING UNITARIOS EN ANGULAR

- a. Lógica pura
 - Funciones que no dependen de Angular.
 - Se prueban directamente usando describe, it y expect.
- b. Componentes y servicios
 - Se prueban de forma aislada.
 - En componentes se comprueba también lo que se renderiza en el HTML (DOM).
- c. TestBed (configuración ligera)
 - TestBed crea un Angular pequeño solo para el test que permite crear componentes y servicios para los tests.
 - Solo se importan el componente y las dependencias necesarias.
- d. Mocks (simulación)
 - Se simulan comportamientos para controlar el resultado del test.
 - Las peticiones HTTP no deben llamar a Internet.
 - El router no debe navegar realmente.

CARACTERÍSTICAS DE UNA BUENA PRUEBA UNITARIA

Una prueba unitaria debe cumplir una serie de características que faciliten su uso y mantenimiento a lo largo del tiempo. En esta práctica se tendrán en cuenta las siguientes:

- a. Fáciles de escribir: Una prueba debe poder escribirse sin una configuración compleja. Si una prueba requiere demasiado código o demasiadas dependencias, es probable que no esté bien planteada.

- b. Fáciles de leer: El código de una prueba debe entenderse casi como una explicación de lo que se está comprobando. Al leer una prueba debería quedar claro qué se está probando y cuál es el resultado esperado.
- c. Rápidas: Las pruebas unitarias deben ejecutarse en poco tiempo. Por este motivo no deben depender de Internet, del sistema de archivos ni de temporizadores reales.
- d. Flexibles (fáciles de mantener): Una prueba debe fallar solo cuando el comportamiento que se está probando cambia realmente. Cambios menores en la implementación no deberían obligar a reescribir todas las pruebas.

Una forma sencilla de escribir pruebas claras y legibles es seguir el patrón **AAA**, que divide cada prueba en tres partes:

- **Arrange (Preparar):** Se prepara el entorno del test: datos de entrada, objetos necesarios o mocks.
- **Act (Actuar):** Se ejecuta la acción que se quiere probar (llamar a una función, método o evento).
- **Assert (Comprobar):** Se verifica que el resultado obtenido es el esperado mediante `expect(...)`.

CONCEPTO DE COBERTURA (COVERAGE)

La cobertura indica qué parte del código ha sido ejecutada al lanzar las pruebas automáticas. Se expresa normalmente en porcentajes y **sirve como orientación, no como garantía de calidad**. La cobertura ayuda a detectar qué partes del código no se están probando.

Tipos habituales de cobertura:

- a. Cobertura de líneas: Porcentaje de líneas de código que se han ejecutado al menos una vez durante los tests.
- b. Cobertura de ramas: Porcentaje de decisiones (if, else, switch) que se han probado en sus distintos caminos.
- c. Cobertura de funciones: Porcentaje de funciones o métodos que han sido invocados en las pruebas.
- d. Cobertura de condiciones: Porcentaje de condiciones lógicas que se han evaluado tanto como verdaderas como falsas.

Importante:

- Una alta cobertura no garantiza que el código esté bien probado.
- Una cobertura baja no implica que el código sea incorrecto.
- La cobertura es una herramienta de apoyo, no un objetivo en sí mismo.

PRIMER TEST

Vamos a hacer unas pruebas sobre la aplicación **MI_APP_03**

Primero comprueba que tu proyecto Angular está configurado por defecto para usar VITEST:

```
package.json > ...
35   "devDependencies": {
36     "@angular/build": "^21.1.0",
37     "@angular/cli": "^21.1.0",
38     "@angular/compiler-cli": "^21.1.0",
39     "jsdom": "^27.1.0",
40     "typescript": "~5.9.2",
41     "vitest": "^4.0.8"
42   }
43 }
```

```
tsconfig.spec.json > ...
2   /* To learn more about Angular compilation
3   {
4     "extends": "./tsconfig.json",
5     "compilerOptions": {
6       "outDir": "./out-tsc/spec",
7       "types": [
8         "vitest/globals"
9       ]
10    },
11    "include": [
12      "src/**/*.d.ts",
13      "src/**/*.ts"
14    ]
15  }
16
```

Después borra todos los ficheros **.spect.ts** **menos el fichero app.spect.ts** Analiza su contenido por defecto y modificarlo para que quede así:

app.spect.ts

```
import { TestBed } from '@angular/core/testing';
import { App } from './app';

describe('App', () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App],
    }).compileComponents();
  });
});
```

```
});

it('should be true', () => {
  expect(true).toBe(true);
});
});
```

- **describe** se utiliza para agrupar un conjunto de pruebas relacionadas, normalmente asociadas a un componente o servicio concreto; su primer parámetro es una descripción textual que ayuda a identificar qué se está probando.
- **beforeEach** (lo genera el asistente por defecto) se ejecuta antes de cada prueba, y en él se prepara el entorno de testing mediante TestBed, que crea una versión ligera de Angular donde se cargan los componentes necesarios. Se encarga de preparar un escenario limpio antes de cada test, garantizando que lo que ocurra en una prueba no contamine a la siguiente. Sin el beforeEach, si un test modifica una variable global o una propiedad del componente, el siguiente test heredaría ese estado.
- **it** define una prueba individual, que comprueba una condición concreta mediante una aserción con **expect(...)**.

Nota: El enfoque varía un poco si usamos standalone true o standalone false

```
beforeEach(async () => {
  await
  TestBed.configureTestingModule({
    imports:
    [RouterModule.forRoot([])],
    declarations: [App],
  }).compileComponents();
});
```

```
beforeEach(async () => {
  await
  TestBed.configureTestingModule({
    imports: [AppComponent],
  }).compileComponents();
});
```

Finalmente lanza los test de prueba de la aplicación con: `ng test`

```

~/.work/fw_ejercicios_m/mi_app_03 > npm run test
Initial chunk files | Names | Raw size
chunk-XSGC6WRG.js | - | 4.05 kB
spec-app-app.js | spec-app-app | 2.32 kB
init-testbed.js | init-testbed | 1.20 kB
styles.css | styles | 584 bytes
| Initial total | 8.16 kB

Application bundle generation complete. [0.588 seconds] - 2026-02-08T17:11:08.466Z

Watch mode enabled. Watching for file changes...

DEV v4.0.16 /Users/alberto/workspace/fw_ejercicios_montoro_alberto_25-26/mi_app_03
✓ mi_app_03 src/app/app.spec.ts (1 test) 1ms
  ✓ App (1)
    ✓ should be true 1ms

Test Files 1 passed (1)
Tests 1 passed (1)
Start at 18:11:08
Duration 433ms (transform 22ms, setup 89ms, import 29ms, tests 1ms, environment 251ms)

PASS Waiting for file changes...
press h to show help, press q to quit

```

Al ejecutarlo, el proceso de testing queda en ejecución continua y se mantiene a la espera de cambios. Esto permite ir definiendo o modificando pruebas y, al guardar los ficheros, ver automáticamente los resultados por pantalla sin necesidad de relanzar el comando.

```

PASS Waiting for file changes...
press h to show help, press q to quit

Watch Usage
press a, return to rerun all tests
press r to rerun current pattern tests
press f to rerun only failed tests
press u to update snapshot
press p to filter by a filename
press t to filter by a test name regex pattern
press w to filter by a project name
press b to start the browser server if not started yet
press q to quit

```

Modifica el test:

```

...
it('should be true', () => {
  expect(false).toBe(true);
});

```

```
> mi_app_03 src/app/app.spec.ts (1 test | 1 failed) 3ms
> App (1)
  ✖ should be true 3ms

----- Failed Tests 1 -----

FAIL mi_app_03 src/app/app.spec.ts > App > should be true
AssertionError: expected false to be true // Object.is equality

- Expected
+ Received

- true
+ false

> src/app/app.spec.ts:12:19
   10|
   11|   it('should be true', () => {
   12|     expect(false).toBe(true);
      |                     ^
   13|   });
   14| });
```

```
Test Files  1 failed (1)
Tests       1 failed (1)
Start at    18:18:23
Duration    128ms
```

```
FAIL Tests failed. Watching for file changes...
press h to show help, press q to quit
```

Añade dos nuevos test:

```
...
it('should be 4', () => {
  expect(2+2).toBe(4);
});

it('should not divide by 0', () => {
  let a = 2;
  let b = 1;
  let result = a / b;
  expect(result).not.toBe(Infinity);
});
```



```
> mi_app_03 src/app/app.spec.ts (3 tests | 1 failed) 4ms
> App (3)
  ✗ should be true 3ms
  ✓ should be 4 0ms
  ✓ should not divide by 0 0ms

----- Failed Tests 1 -----

FAIL mi_app_03 src/app/app.spec.ts > App > should be true
AssertionError: expected false to be true // Object.is equality
```

Añade otro test:

```
...
it('should stop test execution when an expect fails', () => {
  const a = 2;
  const b = 2;

  // Este expect es correcto
  expect(a - b).toBe(0);

  // Este expect FALLA
  expect(a + b).toBe(5);

  // Este expect NO se ejecuta nunca
  expect(true).toBe(true);
});
```

```
> mi_app_03 src/app/app.spec.ts (4 tests | 2 failed) 4ms
> App (4)
  ✗ should be true 3ms
  ✓ should be 4 0ms
  ✓ should not divide by 0 0ms
  ✗ should stop test execution when an expect fails 0ms

----- Failed Tests 2 -----
```

Nota: En algunos casos, tras modificar un test, aunque se pulse 'a' (ejecutar todos) o 'r' (repetir), el comportamiento puede no ser el esperado. Si ocurre esto, pulsa 'q' para salir del modo interactivo y vuelve a lanzar de nuevo todos los tests ejecutando otra vez: `ng test`

TEST BÁSICOS EN COMPONENTES

Vamos a probar el componente app y para ello vamos a ampliarlo ligeramente.

```
<main>
  <a [routerLink]="['/']">
    <header class="brand-name">
      
    </header>
  </a>

  <section class="content">
    <router-outlet></router-outlet> <!-- Ahora sí es útil -->
  </section>
  <section class="aux" [ngStyle]="auxStyle">
    <button class="btn-test" (click)="incrementClicks()">Click me</button>
    <button class="btn-test" (click)="incrementClicks(2)">Click me +2</button>

    <p class="click-counter">
      Clicks: {{ clicks }}
    </p>
  </section>
</main>
```

```
import { Component, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [CommonModule, RouterOutlet, RouterLink],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('mi_app_03');
  public subTitle = signal('');
  public year = 2025;
  public clicks = 0;

  public auxStyle: Record<string, string> = {
    textAlign: 'center',
    marginTop: '100px',
  };

  constructor() {
    this.subTitle.set('DWEC & Frameworks');
    this.year = new Date().getFullYear();
  }

  public showTitleAndsubTitle() {
    return `${this.title()} - ${this.subTitle()} [${this.year}]`;
  }

  public incrementClicks(inc: number = 1) {
    this.clicks=this.clicks+inc;
  }
}
```

```
}  
}
```

En el mismo fichero `app.spect.ts`, primero comenta los test anteriores y luego añade estos test:

```
// El componente app importa RouterOutlet y usas routerLink  
// En los tests conviene añadir provideRouter([])  
  
import { provideRouter } from '@angular/router';  
  
...  
describe('App', () => {  
  beforeEach(async () => {  
    await TestBed.configureTestingModule({  
      imports: [App],  
      providers: [provideRouter([])],  
    }).compileComponents();  
  });  
  
  ...  
  it('should create the app', () => {  
    const fixture = TestBed.createComponent(App);  
    const app = fixture.componentInstance;  
    expect(app).toBeTruthy();  
  });  
});
```

- Se instancia el componente y devuelve un fixture, que representa el contenedor del componente en el test.
 - Se ejecuta el constructor del componente. NO se ejecuta `ngOnInit` automáticamente en este test.
- A partir de este fixture se accede a `componentInstance`, que es la instancia real de la clase `App`.
- Crear el componente no es lo mismo que renderizarlo.
- La aserción `expect(app).toBeTruthy()` verifica que dicha instancia existe y no es `null` ni `undefined`.

Ahora añade este test:

```
...
it('should keep the title and subTitle unchanged', () => {
  const fixture = TestBed.createComponent(App);
  const app = fixture.componentInstance;

  app.subTitle.set('IES Gregorio Prieto');

  console.log('>>>Privado:', app['title']());
  console.log('>>>Publico:', app.subTitle());
  console.log('>>>Publico:', app.year);
  console.log('>>>Publico:', app.showTitleAndsubTitle());

  expect(app.year).toBe(new Date().getFullYear());
  expect(app['title']()).toBe('mi_app_03');
  expect(app.subTitle()).toBe('DWECC & Frameworks');
});
```

```
stdout | src/app/app.spec.ts > App > should keep the title and subTitle unchanged
>>>Privado: mi_app_03
>>>Publico: IES Gregorio Prieto
>>>Publico: 2026
>>>Publico: mi_app_03 - [Signal: IES Gregorio Prieto] [2026]
```

```
> mi_app_03 src/app/app.spec.ts (2 tests | 1 failed) 20ms
> App (2)
  ✓ should create the app 15ms
  ✗ should keep the title and subTitle unchanged 5ms
```

Failed Tests 1

```
FAIL mi_app_03 src/app/app.spec.ts > App > should keep the title and subTitle unchanged
AssertionError: expected 'IES Gregorio Prieto' to be 'DWECC & Frameworks' // Object.is equality
```

```
Expected: "DWECC & Frameworks"
Received: "IES Gregorio Prieto"
```

Durante la ejecución de los tests de componentes, Angular no utiliza un navegador real (Chrome, Firefox, etc.), sino un entorno simulado basado en JSDOM.

JSDOM es una implementación en JavaScript del DOM que permite crear y manipular HTML, atributos, estilos y eventos como si estuviéramos en un navegador, pero sin interfaz gráfica.

Gracias a JSDOM, en los tests podemos acceder a cualquier elemento, usar `querySelector`, leer clases CSS, atributos o texto renderizado, y comprobar el comportamiento de la vista, todo ello de forma rápida, aislada y sin depender del navegador real del usuario.

Arreglar el test anterior y ahora añade otro test nuevo:

```
...
it('should return formatted title, subtitle and year', () => {
  const fixture = TestBed.createComponent(App);
  const app = fixture.componentInstance;

  const result = app.showTitleAndsubTitle();

  expect(result).toContain('DWEK & Frameworks');
});
```

Ahora prueba este test:

```
...
it('should render a router-outlet inside section.content', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const app = fixture.componentInstance;
  const compiled = fixture.nativeElement as HTMLElement;

  console.log('>>>Render:', compiled.innerHTML);

  // router-outlet existe
  const routerOutlet = compiled.querySelector('router-outlet');
  expect(routerOutlet).toBeTruthy();

  // router-outlet está dentro de section.content
  const contentSection = compiled.querySelector('section.content');
  console.log('>>>Render section.content:', contentSection?.innerHTML);
  console.log('>>>Render class section.content:',
    contentSection?.classList.value);

  expect(contentSection).toBeTruthy();
  expect(contentSection?.querySelector('router-outlet')).toBeTruthy();
});
```

- **fixture.detectChanges()** se usa siempre que
 - accedas al HTML
 - busques elementos con `querySelector`
 - esperes que se ejecute `ngOnInit`
 - dependas de bindings (`{{ }}`, `@if`, `@for`, `signals` en `template`)

Si pruebas la clase, no hace falta `detectChanges()`.

Si pruebas la vista, sí hace falta.

```
fixture.detectChanges():
- se llama JUSTO ANTES de comprobar el HTML.
- después de un cambio.
```

Y otros dos más:

```
...
it('should render header with class brand-name', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const compiled = fixture.nativeElement as HTMLElement;

  const header = compiled.querySelector('header');
  expect(header?.classList).toContain('brand-name');
});

it('should render brand logo and link to "/"', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const app = fixture.componentInstance;
  const compiled = fixture.nativeElement as HTMLElement;

  // existe el logo dentro del header y tiene la imagen correcta
  const logo = compiled.querySelector(
    'header.brand-name>img.brand-logo',
  ) as HTMLImageElement | null;
  expect(logo).toBeTruthy();
  expect(logo?.src).toContain('/images/logo.svg');

  // existe el enlace que envuelve header/logo
  const header = compiled.querySelector('header.brand-name');
  const link = header?.parentElement as HTMLAnchorElement | null;

  expect(link).toBeTruthy();
  expect(link?.getAttribute('href')).toBe('/');
});

it('should apply auxStyle on aux section', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();

  const compiled = fixture.nativeElement as HTMLElement;
  const section = compiled.querySelector('section.aux') as HTMLElement;

  expect(section.style.textAlign).toBe('center');
  expect(section.style.marginTop).toBe('100px');
});
```

Finalmente probamos un test negativo

```
...
it('should not render footer element', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const compiled = fixture.nativeElement as HTMLElement;

  const footer = compiled.querySelector('footer');
  expect(footer).toBeNull();
});
```

Vamos a probar ahora un evento:

```
...
it('should increase clicks when buttons are clicked', () => {
  const fixture = TestBed.createComponent(App);
  fixture.detectChanges();
  const app = fixture.componentInstance;
  const compiled = fixture.nativeElement as HTMLElement;
  const buttons = compiled.querySelectorAll('button.btn-test');
  expect(buttons).toBeTruthy();
  expect(buttons.length).toBe(2);

  const button1=buttons[0] as HTMLButtonElement;
  const button2=buttons[1] as HTMLButtonElement;

  button1.click(); //+1
  button1.click(); //+1
  button2.click(); //+2
  button1.click(); //1
  fixture.detectChanges();

  const counter = compiled.querySelector('.click-counter') as HTMLElement;
  expect(counter.textContent).toContain('5');
  expect(app.clicks).toBe(5);
});
```

Recuerda en Angular, `@Input()` y `@Output()` se utilizan para comunicar componentes padre e hijo.

En testing, los `@Input()` se pueden establecer con `fixture.componentRef.setInput(...)` y los `@Output()` se verifican suscribiéndose al `EventEmitter` y simulando la interacción (por ejemplo, un click).

@input

```
it('should render the input label', () => {
  const fixture = TestBed.createComponent(CompInput);

  fixture.componentRef.setInput('label', 'Nuevo');
  fixture.detectChanges();

  const el: HTMLElement = fixture.nativeElement;
  expect(el.textContent).toContain('Nuevo');
});
```

@output

```
it('should emit increment value on click', () => {
  const fixture = TestBed.createComponent(CompOutput);
```

```
fixture.detectChanges();

const comp = fixture.componentInstance;

const received: number[] = [];
comp.increment.subscribe((v) => received.push(v));

const button: HTMLButtonElement =
  fixture.nativeElement.querySelector('button');
button.click();
button.click();

expect(received).toEqual([1, 2]);
});
```

Puedes suscribirte al EventEmitter directamente. No necesitas crear un componente padre para comprobar el @Output().

COBERTURA DE PRUEBAS

Prueba en la terminal: `ng test --coverage`

```
~/.work/fw_ejercicios_m/mi_app_03 > ng test --coverage
The following packages are required but were not found:
- @vitest/coverage-v8
Please install the missing packages and rerun the test command.
~/.work/fw_ejercicios_m/mi_app_03 >
```

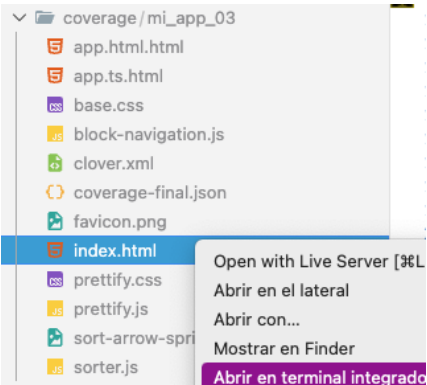
Instala los paquetes necesarios: `npm install @vitest/coverage-v8 --save-dev`

```
~/.work/fw_ejercicios_m/mi_app_03 > npm install @vitest/coverage-v8 --save-dev
added 11 packages, changed 10 packages, and audited 510 packages in 8s
108 packages are looking for funding
run `npm fund` for details
~/.work/fw_ejercicios_m/mi_app_03 >
```

Vuelve a ejecutar la cobertura: `ng test --coverage`

No se queda esperando: `ng test --coverage --no-watch`

Al terminar, se crea una carpeta `coverage/` con un informe HTML. Abre el fichero `coverage/index.html` en el navegador para verlo.



All files

100% Statements 38/38 85.71% Branches 12/14 100% Functions 5/5 100% Lines 22/22

Press n or / to go to the next uncovered block, b, p or k for the previous block.

Filter:

File		Statements	Branches	Functions	Lines				
app.html		100%	16/16	100%	0/0	100%	1/1	100%	12/12
app.ts		100%	14/14	85.71%	12/14	100%	4/4	100%	10/10

All files app.ts

100% Statements 14/14 85.71% Branches 12/14 100% Functions 4/4 100% Lines 18/18

Press n or j to go to the next uncovered block, b, p or k for the previous block.

```
1 import { Component, signal } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { RouterOutlet, RouterLink } from '@angular/router';
4 import { Home } from './home/home';
5
6 @Component({
7   selector: 'app-root',
8   imports: [CommonModule, RouterOutlet, RouterLink],
9   templateUrl: './app.html',
10  styleUrls: ['./app.css'],
11 })
12 export class App {
13   protected readonly title = signal('mi_app_03');
14   public subTitle = signal('');
15   public year = 2025;
16   public clicks = 0;
17
18   public auxStyle: Record<string, string> = {
19     textAlign: 'center',
20     marginTop: '100px',
21   };
22
23   constructor() {
24     this.subTitle.set('DWECC & Frameworks');
25     this.year = new Date().getFullYear();
26   }
27
28   public showTitleAndsubTitle() {
29     return `${this.title()} - ${this.subTitle()} ${this.year}`;
30   }
31
32   public incrementClicks() {
33     this.clicks++;
34   }
35 }
36
```

TEST BÁSICOS EN SERVICIOS

Vamos a crear un servicio y vamos a hacer distintos tipos de pruebas para ir entendiendo lo más importante. Para que algunas pruebas sean más interesantes, también vamos a añadir algunas modificaciones.

Construye el proyecto con `ng serve` y usando el navegador elimina las key de localStorage asociadas a las explicaciones previas.

Crea los siguientes elementos en la terminal:

```
ng generate component apply --skip-tests
ng generate service services/apply.service
ng generate enum enums/week-day
```

Añade las siguientes rutas:

```
{
  path: 'apply',
  component: Apply,
  title: 'Apply',
}
```

```
    canActivate: [authGuard],
  },
  ...
```

Amplía la interfaz: Amplía la interfaz InterfaceHouseForm:

interface-house-form.ts

```
import { WeekDay } from '../enums/week-day';

export interface InterfaceHouseForm {
  firstName: string;
  lastName: string;
  email: string;
  housingLocationId: number;
  consultaDate: Date;
  id: number;
  assignedDay?: WeekDay;
}
```

Amplía el servicio LocalStorageService y comenta el método saveApplication

localStorage.service

```
...

/*   saveApplication(application: InterfaceHouseForm): void {
    const applications = this.getAllApplications();
    applications.push(application);
    localStorage.setItem(this.APPLICATIONS_KEY,
JSON.stringify(applications));
  } */

  setAllApplications(applications: InterfaceHouseForm[]): void {
    localStorage.setItem(this.APPLICATIONS_KEY,
JSON.stringify(applications));
  }
  saveV2Application(application: InterfaceHouseForm): void {
    const applications = this.getAllApplications();

    //siempre insertamos al final:
    const nextId = applications.length === 0 ? 1 :
applications[applications.length - 1].id + 1;

    application.id = nextId;
    applications.push(application);
    localStorage.setItem(this.APPLICATIONS_KEY,
JSON.stringify(applications));
  }

..
```

Guarda cambio, compila y soluciona los errores para que se invoque ahora ***saveV2Application()***:

```
...
    id: 0,
  };

  // Guardar en localStorage
  this.localStorageService.saveV2Application(apply);
...

```

Creamos el enum:

week-day.ts

```
export enum WeekDay {
  Monday = 'Lunes',
  Tuesday = 'Martes',
  Wednesday = 'Miércoles',
  Thursday = 'Jueves',
  Friday = 'Viernes',
}
```

Completa el componente creado

apply.ts

```
import { Component, inject } from '@angular/core';

import { LocalStorageService } from '../services/localstorage.service';
import { ApplyService } from '../services/apply.service';
import { WeekDay } from '../enums/week-day';

@Component({
  selector: 'app-apply',
  imports: [],
  templateUrl: './apply.html',
  styleUrls: ['./apply.css'],
})
export class Apply {
  public applyService = inject(ApplyService);

  public weekDays: WeekDay[] = [
    WeekDay.Monday,
    WeekDay.Tuesday,
    WeekDay.Wednesday,
    WeekDay.Thursday,
    WeekDay.Friday,
  ];

  constructor() {

```

```
this.applyService.load();
}
}
```

apply.html

```
<section class="resultados">

  <ul>
    @for (day of weekDays; track day) {
      <li>
        Citas del {{ day }}:
        {{ applyService.getAppointmentsByDay(day).length }}
      </li>
    }
  </ul>
  <br>
  <p> <strong>Citas sin asignar:
    {{ applyService.getUnassignedAppointments().length }}
  </strong>
</p>

</section>
```

Define los métodos del servicio creado:

apply.service.ts

```
import { inject, Injectable } from '@angular/core';

import { InterfaceHouseForm } from '../interfaces/interface-house-form';
import { WeekDay } from '../enums/week-day';
import { LocalStorageService } from '../localStorage.service';

@Injectable({
  providedIn: 'root',
})
//Trabajamos con las citas en memoria hasta guardar.
export class ApplyService {
  private localStorageService = inject(LocalStorageService);

  private working: InterfaceHouseForm[] = [];

  private dirty = false; //Cambios pendientes
  constructor() {
    this.initializeTestData();
  }

  load(): void {
    this.working = this.localStorageService.getAllApplications();
    this.dirty = false;
  }
}
```

```
getAppointments(): InterfaceHouseForm[] {
    return this.working;
}

hasPendingChanges(): boolean {
    return this.dirty;
}

saveChanges(): void {
    this.localStorageService.setAllApplications(this.working);
    this.dirty = false;
}

discardChanges(): void {
    this.load();
}

addAppointment(app: InterfaceHouseForm): void {
    //crear una COPIA del objeto, no guardar la referencia
    this.working.push({ ...app });
    this.dirty = true;
}

deleteAppointmentById(id: number): boolean {
    const index = this.working.findIndex((a) => a.id === id);

    if (index === -1) {
        return false;
    }

    this.working.splice(index, 1);
    this.dirty = true;
    return true;
}

assignDay(id: number, day: WeekDay): boolean {
    const appointment = this.working.find((a) => a.id === id);
    if (!appointment) return false;

    const appointmentsThatDay = this.working.filter((a) => a.assignedDay === day
    && a.id !== id);

    if (appointmentsThatDay.length >= 2) {
        return false;
    }

    appointment.assignedDay = day;
    this.dirty = true;
    return true;
}

getAppointmentsByDay(day: WeekDay): InterfaceHouseForm[] {
    return this.working.filter((a) => a.assignedDay === day);
}

getUnassignedAppointments(): InterfaceHouseForm[] {
    return this.working.filter((a) => !a.assignedDay);
}
```

```
clear(): void {
    this.working = [];
    this.dirty = false;
}

private initializeTestData(): void {
    // Si ya hay datos, no hacemos nada
    const existing = this.localStorageService.getAllApplications();
    if (existing.length > 0) {
        return;
    }

    const baseDate = new Date(2026, 2, 6);

    const testApplications: InterfaceHouseForm[] = [
        {
            id: 1,
            firstName: 'Gon',
            lastName: 'Freeccs',
            email: 'gon@hunter.com',
            housingLocationId: 1,
            consultaDate: baseDate,
            assignedDay: WeekDay.Monday,
        },
        {
            id: 2,
            firstName: 'Killua',
            lastName: 'Zoldyck',
            email: 'killua@hunter.com',
            housingLocationId: 1,
            consultaDate: baseDate,
            assignedDay: WeekDay.Monday,
        },
        {
            id: 3,
            firstName: 'Kurapika',
            lastName: 'Kurta',
            email: 'kurapika@hunter.com',
            housingLocationId: 2,
            consultaDate: baseDate,
            assignedDay: WeekDay.Friday,
        },
        {
            id: 4,
            firstName: 'Leorio',
            lastName: 'Paradinight',
            email: 'leorio@hunter.com',
            housingLocationId: 2,
            consultaDate: baseDate,
        },
        {
            id: 5,
            firstName: 'Hisoka',
            lastName: 'Morrow',
            email: 'hisoka@hunter.com',
            housingLocationId: 3,
            consultaDate: baseDate,
        },
    ],
```

```
    },  
  ],  
  
  this.localStorageService.setAllApplications(testApplications);  
}  
}
```

Comprueba que funciona: `ng serve` y navega a: <http://localhost:4200/apply> tras autenticarte

Ahora vamos a probar paso a paso el servicio. Fichero: **apply.service.spec.ts**

ESTRATEGIA DE TESTING: Confiamos en que LocalStorageService ya está probado y funciona.

Esto simplifica nuestras pruebas y nos permite centrarnos en la lógica de ApplyService.

```
import { TestBed } from '@angular/core/testing';  
import { ApplyService } from '../apply.service';  
import { LocalStorageService } from '../localStorage.service';  
import { WeekDay } from '../enums/week-day';  
import { InterfaceHouseForm } from '../interfaces/interface-house-form';  
  
describe('ApplyService', () => {  
  //Declaramos variables que vamos a usar en cada test  
  //y que no son necesario re-crear en cada test  
  let service: ApplyService;  
  let localStorageService: LocalStorageService;  
  
  beforeEach(() => {  
    // Limpiar localStorage para evitar contaminación entre tests  
    // No confundir con localStorageService  
    localStorage.clear();  
  
    TestBed.configureTestingModule({  
      providers: [ApplyService, LocalStorageService],  
    });  
  
    localStorageService = TestBed.inject(LocalStorageService);  
    service = TestBed.inject(ApplyService);  
  
    // Recuerda: Todo ocurre sobre DOM virtual  
    localStorageService.setAllApplications([]);  
    service.clear();  
  });  
  
  it('should be created', () => {  
    console.log(service.getAppointments());  
    expect(service).toBeTruthy();  
    expect(service).toBeInstanceOf(ApplyService);  
  });  
});
```



```
it('should start with empty appointments after clear', () => {
  expect(service.getAppointments()).toHaveLength(0);
  expect(service.hasPendingChanges()).toBe(false);
});

});
```

Vamos a probar el load y que en cada prueba eliminamos los datos de ejemplo:

```
it('should load appointments from localStorage after load()', () => {
  localStorageService.setAllApplications([
    {
      id: 1,
      firstName: 'Gon',
      lastName: 'Freeccs',
      email: 'gon@hunter.com',
      housingLocationId: 1,
      consultaDate: new Date(2026, 2, 6),
      assignedDay: WeekDay.Monday,
    },
  ]);

  service.load();
  console.log(service.getAppointments());
  expect(service.getAppointments()).toHaveLength(1);
  expect(service.hasPendingChanges()).toBe(false);
});
```

Uso de FACTORY y variable fija para hacer los test más legibles:

```
describe('ApplyService', () => {
  //Declaramos variables que vamos a usar en cada test
  //y que no son necesario re-crear en cada test
  let service: ApplyService;
  let localStorageService: LocalStorageService;

  const baseDate = new Date(2026, 2, 6);

  function makeApp(id: number, assignedDay?: WeekDay): InterfaceHouseForm {
    return {
      id,
      firstName: `User${id}`,
      lastName: 'Test',
      email: `user${id}@test.com`,
      housingLocationId: 1,
      consultaDate: baseDate,
      assignedDay,
    };
  }

  beforeEach(() => {
    ...
  })
});
```

Vamos a usar esa Factory para probar una inserción en memoria:

```
it('should add appointment to working array', () => {
  const newApply = makeApp(1);
  service.addAppointment(newApply);

  const appointments = service.getAppointments();
  expect(appointments).toHaveLength(1);
  expect(appointments[0]).toEqual(newApply);
});

it('should mark dirty flag as true when adding appointment', () => {
  expect(service.hasPendingChanges()).toBe(false);
  service.addAppointment(makeApp(1));
  expect(service.hasPendingChanges()).toBe(true);
});
```

Ahora vamos a probar varias inserciones en memoria, la inserción en LocalStorage y la carga:

```
it('should allow adding multiple appointments', () => {
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));
  service.addAppointment(makeApp(3));

  expect(service.getAppointments()).toHaveLength(3);
  expect(service.getAppointments().map((a) => a.id)).toEqual([1, 2, 3]);
});

it('should load appointments from LocalStorageService', () => {
  const savedData = [makeApp(1), makeApp(2, WeekDay.Monday), makeApp(3,
    WeekDay.Friday)];
  localStorageService.setAllApplications(savedData);

  service.load();

  expect(service.getAppointments()).toHaveLength(3);
  expect(service.getAppointments()[0].id).toBe(1);
  expect(service.getAppointments()[1].assignedDay).toBe(WeekDay.Monday);
  expect(service.getAppointments()[2].assignedDay).toBe(WeekDay.Friday);
});
```

Ahora vamos a probar la asignación de días:

```
it('should return false when assigning day to non-existent appointment', () => {
  const result = service.assignDay(999, WeekDay.Monday);
  expect(result).toBe(false);
});

it('should assign day when there are fewer than 2 appointments on that day', () => {
  service.addAppointment(makeApp(1)); // Sin día asignado
  service.addAppointment(makeApp(2, WeekDay.Monday)); // 1 cita en lunes

  const result = service.assignDay(1, WeekDay.Monday);

  expect(result).toBe(true);
});
```

```
expect(service.getAppointmentsByDay(WeekDay.Monday)).toHaveLength(2);

const modified = service.getAppointments().find(a => a.id === 1);
expect(modified?.assignedDay).toBe(WeekDay.Monday);
});

it('should NOT assign day when there are already 2 appointments on that day',
() => {
  service.addAppointment(makeApp(1, WeekDay.Monday));
  service.addAppointment(makeApp(2, WeekDay.Monday));
  service.addAppointment(makeApp(3)); // Sin asignar

  const result = service.assignDay(3, WeekDay.Monday);

  expect(result).toBe(false);
  expect(service.getAppointmentsByDay(WeekDay.Monday)).toHaveLength(2);

  const unchanged = service.getAppointments().find(a => a.id === 3);
  expect(unchanged?.assignedDay).toBeUndefined();
});

it('should return only unassigned appointments', () => {
  service.addAppointment(makeApp(1, WeekDay.Friday));
  service.addAppointment(makeApp(2)); // Sin asignar
  service.addAppointment(makeApp(3)); // Sin asignar
  service.addAppointment(makeApp(4, WeekDay.Monday));

  const unassigned = service.getUnassignedAppointments();

  expect(unassigned).toHaveLength(2);
  expect(unassigned.map((a) => a.id)).toEqual([2, 3]);
  expect(unassigned.every((a) => a.assignedDay === undefined)).toBe(true);
});
```

Ahora vamos a probar el borrado:

```
it('should return false when deleting non-existent id', () => {
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));

  const result = service.deleteAppointmentById(999);
  expect(result).toBe(false);
  expect(service.getAppointments()).toHaveLength(2);
});

it('should delete appointment and return true when id exists', () => {
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));
  service.addAppointment(makeApp(3));

  const result = service.deleteAppointmentById(2);

  expect(result).toBe(true);
  expect(service.getAppointments()).toHaveLength(2);
  expect(service.getAppointments().map((a) => a.id)).toEqual([1, 3]);
});
```

```
expect(service.hasPendingChanges()).toBe(true);
});

it('should handle deleting from single-element list', () => {
  service.addAppointment(makeApp(1));
  const result = service.deleteAppointmentById(1);

  expect(result).toBe(true);
  expect(service.getAppointments()).toHaveLength(0);
});
```

Estas son una serie de pruebas esenciales, pero aún nos quedan aspectos del servicio sin probar:

- Más verificaciones de que el cambio de dry se hace correctamente: `hasPendingChanges()`
- Caso límite: eliminar el primero de varios
- Caso límite: eliminar el último de varios
- Reasignar al mismo día no debe rechazarse
- Cambiar una cita de lunes a viernes
- Distribuir citas en múltiples días
- Probar asignación a cada día de la semana
- `getAppointmentsByDay` con múltiples citas
- `getAppointmentsByDay` cuando no hay nada
- Guardar múltiples tipos de cambios
- Hacer cambios y descartarlos
- `clear()` no debe tocar el storage
- Flujo completo de usuario
- ...

Ahora vamos a probar el uso de espías que vigilan si se llamó a un método. El spy NO impide que el método real se ejecute, solo lo observa.

```
it('should call setAllApplications when saveChanges is executed', () => {
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));

  // vi.spyOn(objeto, 'nombreMétodo') crea el espía
  const spy = vi.spyOn(localStorageService, 'setAllApplications');

  service.saveChanges();

  expect(spy).toHaveBeenCalled(); // ¿Se llamó?
  expect(spy).toHaveBeenCalledTimes(1); // ¿Cuántas veces?

  // IMPORTANTE: Siempre restaurar el spy después de usarlo
  spy.mockRestore();
});

it('should call setAllApplications with correct data', () => {
  const app1 = makeApp(1);
  const app2 = makeApp(2, WeekDay.Monday);
  service.addAppointment(app1);
```

```
service.addAppointment(app2);

const spy = vi.spyOn(localStorageService, 'setAllApplications');

service.saveChanges();

// Verificar CON QUÉ argumentos se llamó
expect(spy).toHaveBeenCalledWith([app1, app2]);

// Otra forma con matchers:
expect(spy).toHaveBeenCalledWith(
  expect.arrayContaining([
    expect.objectContaining({ id: 1 }),
    expect.objectContaining({ id: 2, assignedDay: WeekDay.Monday }),
  ]),
);

spy.mockRestore();
});

it('should NOT call setAllApplications when only adding appointments', () => {
  const spy = vi.spyOn(localStorageService, 'setAllApplications');

  // Solo agregar en memoria, NO guardar
  service.addAppointment(makeApp(1));
  service.addAppointment(makeApp(2));

  expect(spy).not.toHaveBeenCalled();

  spy.mockRestore();
});
```

Ahora vamos a probar otra utilidad de spy:

- Vamos a interceptar un método real de una dependencia (en este caso LocalStorageService) y controlar qué devuelve, sin ejecutar su implementación original ([combinando uso de spy y uso básico de mock](#)).

De esta forma, aislamos completamente ApplyService y podemos comprobar cómo reacciona ante distintos escenarios simulados, como recibir datos concretos o un array vacío, sin depender del comportamiento real del almacenamiento.

```
it('should load mocked data using spyOn (simulated return)', () => {
  const fakeData = [makeApp(100, WeekDay.Monday), makeApp(200,
    WeekDay.Friday)];

  // Espiamos el método real y forzamos su retorno
  const spy = vi.spyOn(localStorageService,
    'getAllApplications').mockReturnValue(fakeData);

  service.load();
```

```
expect(spy).toHaveBeenCalled();

expect(service.getAppointments()).toHaveLength(2);
expect(service.getAppointments().map((a) => a.id)).toEqual([100, 200]);
spy.mockRestore();
});

it('should simulate empty storage using spy', () => {
  const spy = vi.spyOn(localStorageService,
    'getAllApplications').mockReturnValue([]);

  service.load();

  expect(service.getAppointments()).toHaveLength(0);
  spy.mockRestore();
});
```

MOCKS

En este apartado vamos a probar el servicio `HousingService`. Este servicio realiza peticiones asíncronas mediante `fetch`, por lo que vamos a utilizar mocks para simular todas las dependencias externas (en este caso, la llamada HTTP).

Nota: No vamos a probar: `submitApplication()`

El objetivo es aislar completamente el servicio, evitando depender de la red o de APIs reales. Los **mocks nos permiten controlar exactamente qué devuelve fetch, simular errores y verificar cómo responde nuestro código en cada escenario**.

De esta forma, probamos únicamente la lógica interna del servicio. En pruebas anteriores podríamos haber aplicado la misma técnica para simular `LocalStorageService`, en lugar de asumir que ya funcionaba correctamente.

No queremos probar que `fetch` funciona. Queremos probar que:

- Se llama a `fetch`.
- Se procesa correctamente el `.json()`.
- Se devuelve `[]` cuando la API devuelve `null`.
- Se maneja correctamente la respuesta.

Por eso simulamos fetch. Recuerda un Mock:

- REEMPLAZA completamente el servicio/método
- **No ejecuta el código real**
- Tú controlas qué retorna

¿Cuándo se usan?

- Cuando la dependencia NO está probada
- Cuando quieres simular errores
- Cuando la dependencia es muy lenta (BD, API)
- Cuando quieres AISLAR completamente el servicio

Crea el correspondiente fichero `housing.service.spec.ts` con este contenido:

```
import { TestBed } from '@angular/core/testing';
import { HousingService } from '../housing.service';
import { HousingLocationInfo } from '../housinglocation';

describe('HousingService', () => {
  //Declaramos variables que vamos a usar en cada test
  //y que no son necesario re-crear en cada test

  let service: HousingService;

  // Helper: crea una Response "fake" mínima para fetch
  // fetch() devuelve un objeto Response que distintos campos
  // Aquí estamos construyendo una versión mínima simulada
  function mockFetchResponse<T>(opts: { ok?: boolean; status?: number; json: T })
  {
    const { ok = true, status = 200, json } = opts;

    return {
      ok,
      status,
      json: vi.fn().mockResolvedValue(json),
    } as unknown as Response;
  } //mockResolvedValue simula una promesa resuelta

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [HousingService],
    });

    service = TestBed.inject(HousingService);
  });

  afterEach(() => {
    vi.restoreAllMocks();
    //Muy importante en tests con mocks:
    //Restauramos todos los spies después de cada test
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
    expect(service).toBeInstanceOf(HousingService);
  });

  it('getAllHousingLocations() should return locations from fetch().json()',
  async () => {
    const locationsFake: HousingLocationInfo[] = [
```

```
    { id: 1, name: 'House 1' } as any,
    { id: 2, name: 'House 2' } as any,
  ];

  //estamos reemplazando un fetch REAL por uno simulado.
  //globalThis.fetch = la función fetch nativa del navegador

  const fetchSpy = vi
    .spyOn(globalThis, 'fetch')
    .mockResolvedValue(mockFetchResponse({ json: locationsFake }));

  const result = await service.getAllHousingLocations();

  expect(fetchSpy).toHaveBeenCalledTimes(1);
  expect(result).toEqual(locationsFake);
});

it('getAllHousingLocations() should return [] when json is null/undefined',
  async () => {
    vi.spyOn(globalThis, 'fetch').mockResolvedValue(
      mockFetchResponse({ whatever: string } | null>({
        json: null,
      })),
    );

    const result = await service.getAllHousingLocations();

    expect(result).toEqual([]);
  });

it('getAllHousingLocations() should throw error when fetch fails', async () => {
  // Simular error de red
  vi.spyOn(globalThis, 'fetch').mockRejectedValue(new Error('Network error'));

  await expect(service.getAllHousingLocations()).rejects.toThrow('Network
error');
});

it('getHousingLocationById() should call fetch with ?id= and return first
element', async () => {
  const id = 7;
  const apiResult: HousingLocationInfo[] = [{ id, name: 'Only one' } as any];

  const fetchSpy = vi
    .spyOn(globalThis, 'fetch')
    .mockResolvedValue(mockFetchResponse({ json: apiResult }));

  const result = await service.getHousingLocationById(id);

  expect(fetchSpy).toHaveBeenCalledTimes(1);
  expect(String(fetchSpy.mock.calls[0][0])).toContain(`?id=${id}`);
  //calls: Es un array que guarda TODAS las llamadas al spy.
  //fetchSpy.mock.calls[0][0] = primer argumento de la primera llamada

  expect(result).toEqual(apiResult[0]);
});

it('getHousingLocationById() should return {} when API returns empty array',
  async () => {
```



```
vi.spyOn(globalThis, 'fetch').mockResolvedValue(
  mockFetchResponse<HousingLocationInfo[]>({ json: [] }),
);

const result = await service.getHousingLocationById(999);

expect(result).toEqual({});
});

it('getHousingLocationById() should build URL with id parameter', async () => {
  const fetchSpy = vi
    .spyOn(globalThis, 'fetch')
    .mockResolvedValue(mockFetchResponse({ json: [] }));

  await service.getHousingLocationById(42);

  expect(fetchSpy).toHaveBeenCalledWith('https://dwec-fw-gp.vercel.app/api/houses?id=42');
});
});
```

Cuando probamos código asíncrono debemos usar `async/await` en el test. Si no usamos `await`, el test puede finalizar antes de que la promesa se resuelva y dar falsos positivos.

Finalmente si queremos probar un método que tiene `try / catch` en función del tipo de respuesta:

```
it('getAllHousingLocations2() should return [] when response is not ok', async () => {
  vi.spyOn(globalThis, 'fetch').mockResolvedValue(
    mockFetchResponse({
      ok: false,
      status: 500,
      json: null,
    }),
  );

  const result = await service.getAllHousingLocations2();

  expect(result).toEqual([]);
});
```

Tipos de mocks en Vitest

a. `mockReturnValue` → Funciones síncronas. Para funciones normales (no `async`):

```
vi.spyOn(service, 'metodoNormal').mockReturnValue('resultado');
```

b. `mockResolvedValue` → Funciones async que resuelven. Para funciones que devuelven `Promise.resolve(...)`:

```
vi.spyOn(service, 'metodoAsync').mockResolvedValue('resultado');
```

c. `mockRejectedValue` → Funciones async que fallan. Para simular errores en promesas:

```
vi.spyOn(service, 'metodoAsync').mockRejectedValue(new Error('Falló'));
```

Si el servicio usara Angular `HttpClient` en lugar de `fetch`, no mockearíamos `fetch`. En Angular existe una herramienta específica para esto:

```
HttpClientTestingModule  
HttpTestingController
```



4. Creación de componentes

Generar componentes (ng generate component)
Comunicación entre componentes (Input y Output)
Ciclo de vida de los componentes

5. Templates y Data Binding

Interpolación ({{ }})
Property Binding ([])
Event Binding (())
Two-way Data Binding ([({}]))

6. Directivas

Estructurales (*ngIf, *ngFor)
Atributo (ngClass, ngStyle)
Crear directivas personalizadas (opcional)

7. Servicios e Inyección de Dependencias

Crear servicios (ng generate service)
Uso de @Injectable
Inyectar servicios en componentes
Comunicación entre componentes con servicios

8. HTTP y consumo de APIs

HttpClientModule
Realizar peticiones GET y POST
Manejo de Observables
Mostrar datos en pantalla

9. Ruteo

Configuración básica de rutas
Navegación entre componentes
Rutas con parámetros
Lazy Loading (opcional)

10. Formularios

Formularios reactivos
Validaciones básicas
Envío de formularios